PWLLHELI SAILING CLUB

Pwllheli Race Officer

Reference Manual

Installation, hardware integration and every admin function

Covers: Windows setup & deployment · horn, audio & RTSP camera hardware · weather station · Cloudflare remote access · every Settings and admin page

Covers app version 1.003

Internal race-office / IT-volunteer reference · human-supervised prototype

ABOUT THIS MANUAL

About this manual

This is the complete reference manual for Pwllheli Race Officer: every admin screen and setting, plus the hardware and services it integrates with — the start-hut horn, VHF/audio announcements, an RTSP IP camera for finish video, a weather station feed, and Cloudflare for remote/public access. It is written for whoever sets up and maintains the system: a race officer, a club IT volunteer, or club committee member.

If you only need to run a series of races day-to-day, the shorter **Race Officer's Guide** (*Pwllheli_Race_Officer_Series_Guide.pdf*) is a faster read. If you're a competitor wanting to know what the public pages show, see the **Competitor's Guide** (*Pwllheli_Competitor_Guide.pdf*). This manual is the one to reach for when installing the system, wiring up hardware, or looking up what a specific setting does.

> **NOTE**
>
> Pwllheli Race Officer is a **human-supervised** tool. It automates timing, hardware signals and publishing, but the race officer and race committee remain responsible for every official decision — verify what the app shows against what is actually happening at the venue.

How this manual is organised:

- **Part I — Getting started:** installing on Windows, production deployment, first login and the Dashboard.
- **Part II — Boats, marks and courses:** the boat database and the club's fixed marks/courses.
- **Part III — Running a series of races:** the full race-day workflow from series setup to publishing results.
- **Part IV — Hardware integration:** the horn, manual horn-input sensing, central VHF audio, the weather station, off-grid hut power monitoring, and RTSP camera/video recording.
- **Part V — Remote access:** Cloudflare Tunnel, Cloudflare R2 public video/image publishing, and the public live camera stream.
- **Part VI — Administration and reference:** rating sources, polars, branding, backup/restore, environment variables, status codes and a good-practice checklist.

CONTENTS

Contents

PART I — GETTING STARTED

Installing on Windows

Pwllheli Race Officer is a Python/Flask application served by Waitress. This chapter covers a clean install on a Windows PC; Python 3.10+ is required.

1. Unpack the release

Create a normal, writable folder such as **C:\RaceOfficer**, place the release ZIP there, and extract it — you should end up with a folder like **C:\RaceOfficer\pwllheli_race_officer_v1_003**.

> **NOTE**
>
> Do not run the app directly from the ZIP file, an email attachment, Downloads, or a OneDrive-synced folder — it needs a normal local folder it can write a database into. If Windows offered to **disable the path length limit** during Python install, accept it.

2. Install Python

Download the 64-bit Windows installer from python.org. On the first installer screen, tick **"Add python.exe to PATH"**, then click **Install Now**. Verify from a new Command Prompt:

```
py --version
py -m pip --version
```

3. Create a virtual environment and install dependencies

From the app folder, in Command Prompt (not PowerShell — see note below):

```
cd /d C:\RaceOfficer\pwllheli_race_officer_v1_003
py -m venv .venv
.venv\Scripts\activate.bat
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

> **NOTE**
>
> The beginner guide uses **Command Prompt** rather than PowerShell specifically to avoid PowerShell's execution-policy prompts when activating a virtual environment.

4. First run

```
python app.py
```

This starts Waitress on **0.0.0.0:5050** by default. Open **http://localhost:5050/admin** for the race-office dashboard, or **http://localhost:5050** for the public read-only current-race page. Leave the window open; **Ctrl+C** stops the app.

5. First login

Username **admin**. If you haven't set a password via the environment variable below, the app generates one and writes it to **runtime/initial_admin_password.txt** the first time it runs.

To set your own password before the database is first created:

```
set RO_INITIAL_ADMIN_PASSWORD=choose-a-good-password-here
python app.py
```

This only affects first-run account creation. To change a password later, use Settings → Users (Chapter 3). Any pre-existing default **admin/admin** credential is automatically replaced on startup unless **RO_ALLOW_DEFAULT_ADMIN=1** is explicitly set.

Upgrading to a new version

1. Stop the running app (or the Scheduled Task — see Chapter 2).
2. Back up the **data/** folder (Backup / restore in the app, or copy the folder directly).
3. Unzip the new version into a **new** folder — don't overwrite the old one in place.
4. Copy your existing **data/** folder into the new version's folder.
5. Start the app and check Settings — serial ports, camera URL, weather station address and FFmpeg path carry over via **data/race_officer.db**, but are worth a quick visual check after any upgrade.

> **NOTE**
>
> After an upgrade, existing browser sessions are treated as stale and everyone is asked to log in again — this is deliberate, not a bug. A same-version restart does not force re-login.

Production deployment on Windows

For the race-hut PC, the app should start automatically at login rather than being run from a Command Prompt window left open. The **deploy\windows** folder provides a Windows Scheduled Task set up for this, registered under the task name **“Pwllheli Race Officer”**.

The five deployment scripts

FIELD / CONTROL	WHAT IT DOES
install_startup_task	Prepares the virtual environment (via start_race_officer -SetupOnly), then registers the Scheduled Task: trigger “at logon” for the current user, hidden window, auto-restart up to 3 times, and starts it immediately.
start_race_officer	The actual worker script, used both by the task and for manual/diagnostic runs. Creates runtime folders, creates the venv if missing, reinstalls dependencies only when requirements.txt has changed (hash-checked), sets default RO_HOST/RO_PORT/RO_THREADS, and logs to runtime\logs\race_officer.log.
restart_startup_task	Stops and restarts the Scheduled Task — use after an upgrade or a hardware settings change that needs a fresh process.
status_startup_task	Reports task state, last run result, does a live HTTP check against /admin, and tails the last 20 lines of the log file.
uninstall_startup_task	Stops and removes the Scheduled Task. Does not delete the app folder or data — safe to run before installing a new version's task.

Typical order of use: **install** once, **status** to confirm it's healthy, **restart** after changes, **uninstall** before moving to a new version folder (then **install** again from the new folder, since the task points at the folder it was installed from).

> **NOTE**
>
> The task deliberately runs **non-elevated**, in the signed-in user's own interactive session (not as a Windows Service). This is required for desktop audio (VHF announcements), USB/serial horn hardware, and cameras — services have restricted access to interactive audio and USB devices. If a black console window is left open after login, that means an old-style task/script is installed; fix by re-running uninstall then install.

Environment variables

FIELD / CONTROL	WHAT IT DOES
RO_HOST	Bind address for Waitress. Default 0.0.0.0 (all network interfaces).
RO_PORT	Listening port. Default 5050.
RO_THREADS	Waitress worker thread count. Default 8.

FIELD / CONTROL	WHAT IT DOES
RO_SECRET_KEY	Flask session signing key. If unset, a random key is generated once and persisted to runtime/secret_key.txt so restarts don't log everyone out. Set a strong value explicitly for any internet-facing deployment (see Chapter 17).
RO_INITIAL_ADMIN_PASSWORD	Sets the first-run admin password. Only effective before the database/first admin user exists.
RO_ALLOW_DEFAULT_ADMIN	Set to 1 to stop the app auto-replacing an existing insecure admin/admin credential pair.
RO_WEATHER_ALLOWED_HOSTS	Comma-separated hostname allow-list further restricting which weather-station addresses are accepted (see Chapter 14).

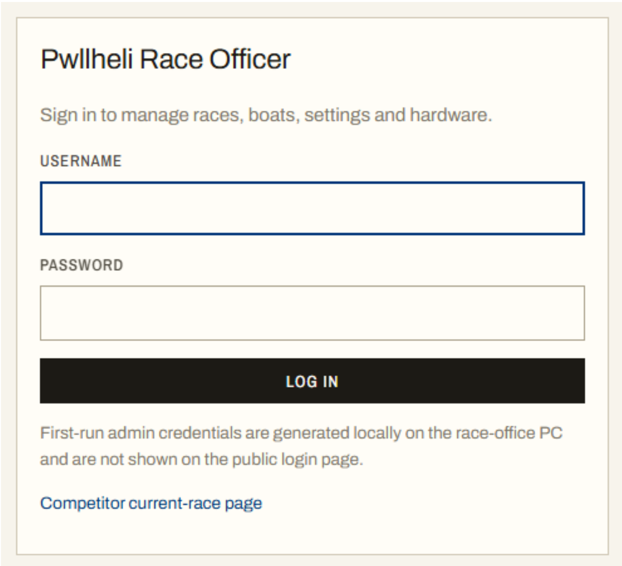
IMPORTANT

Windows-specific gotchas: check for **case-only duplicate paths** before zipping a release (Windows treats them as the same file); keep the race-office PC's clock **NTP or GPS disciplined** — every signal and finish time depends on it; and don't let the PC sleep during racing.

First login, users and the Dashboard

Logging in

Open **http://<pc-address>:5050/admin** and sign in. The public competitor pages (**Competitor's Guide**) never require a login — only the race-office side does.



The sign-in page.

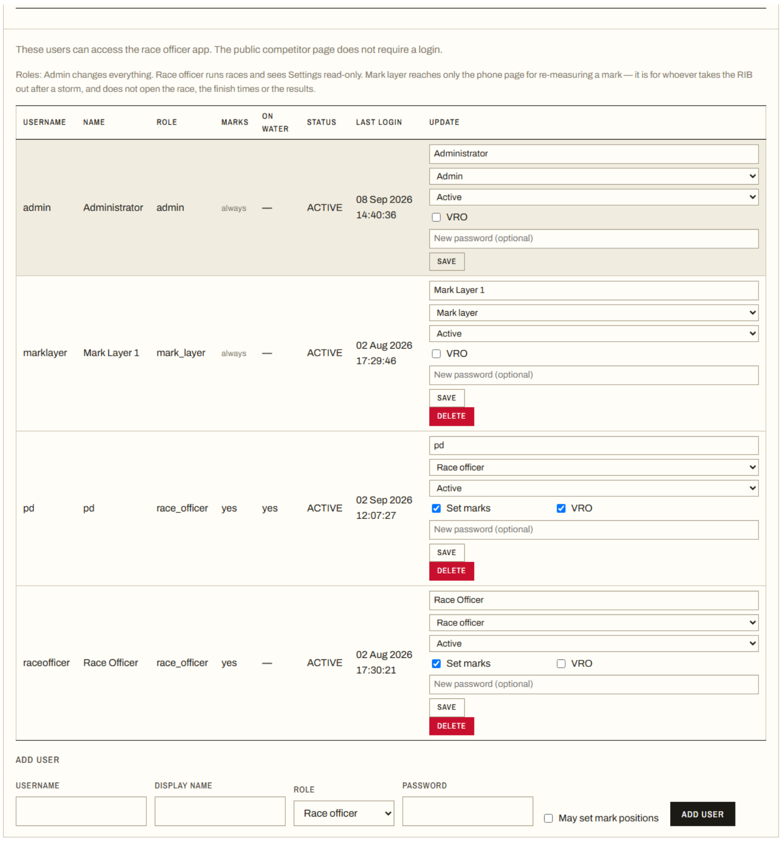
User roles

Every login account has one of two roles:

- **Admin** — full access to everything, including Settings and restoring a backup.
- **Race officer** — can run races and use everything needed on race day, but **Settings is read-only** and **restoring a backup is not allowed** (downloading one is fine). Opening Settings as a race officer shows every value greyed out, with a “Read-only · administrator access required” note in place of the Save button. The limit is enforced on the server too, not just hidden in the page.

Managing users (Settings → Users)

Users management is admin only. Every race-office login is a row here: username, display name, role (an **Admin**/**Race officer**/**Mark layer** dropdown), whether it may set mark positions, status (Active/Inactive), and last login. Each row has its own inline Save (edit display name, role, status, or set a new password) and a Delete button — except for the account you're currently signed in as. Add a user with the form at the bottom: Username, Display name, Role and Password. At least one active administrator must always remain, so the app refuses to demote, deactivate or delete the last admin.



Settings → Users: existing accounts and the add-user form.

The three roles

Admin changes everything, including Settings and restoring a backup. **Race officer** runs races and uses everything race day needs, but sees Settings read-only and cannot restore a backup (they can still download one). **Mark layer** is much smaller than either: it reaches the phone page for re-measuring a mark and nothing else. Signing in lands straight on that page, and any other page sends them back to it.

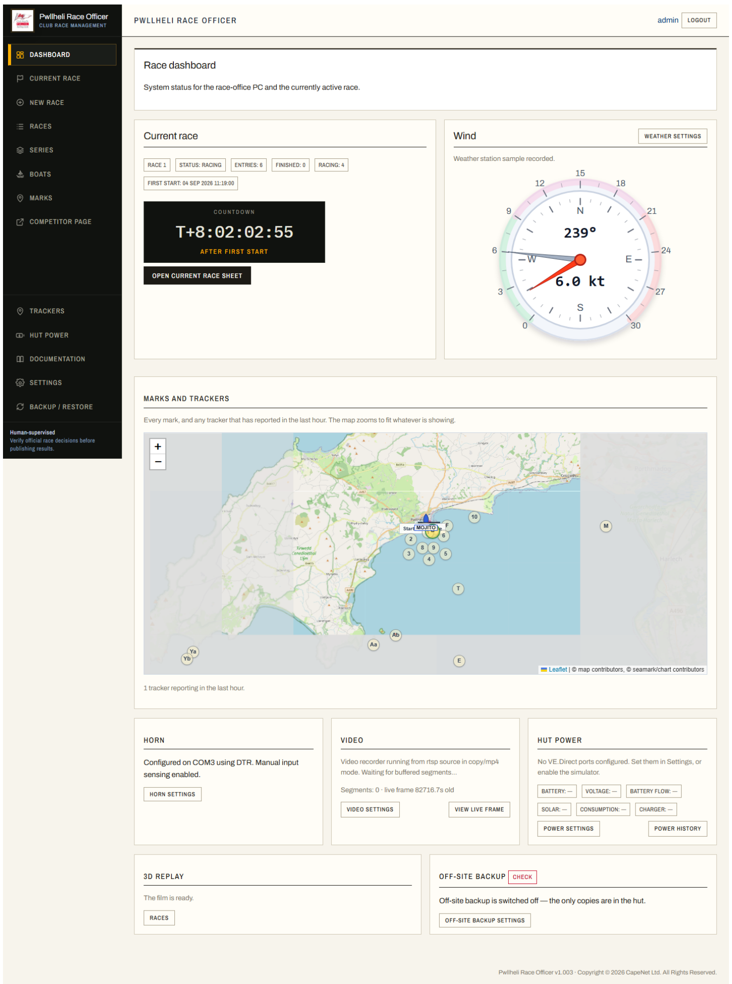
The reason for a third role is that the person who takes a RIB out after a storm is often neither an administrator nor the duty race officer, and that job needs one button on a phone — not the start sequence, the finish times and the results as well. A race officer who should *also* be able to do it gets the **Set marks** tickbox instead, so they do not need a second account. Administrators and mark layers always may, so the tickbox is not shown on their rows.

Changing your own password

Because Settings is read-only for race officers, every signed-in user has a **My account** page for changing their *own* password — open it from the username in the top bar (or go to **/account**). It asks for the current password to confirm the change, enforces a minimum length, and keeps you signed in. Administrators can still reset anyone's password from Settings → Users.

The Dashboard

The landing page after login. A single at-a-glance view of what needs watching: the currently active race and its countdown, live wind, the marks and any reporting trackers, horn, video and hut-power status, the age of the last off-site backup, and — when the club makes them — how a 3D replay render is getting on. Everything on it is something that changes; the club's standing start-line, finish-line and radio notes are Sailing Instructions and were taken off it in v1.002.



The Dashboard.

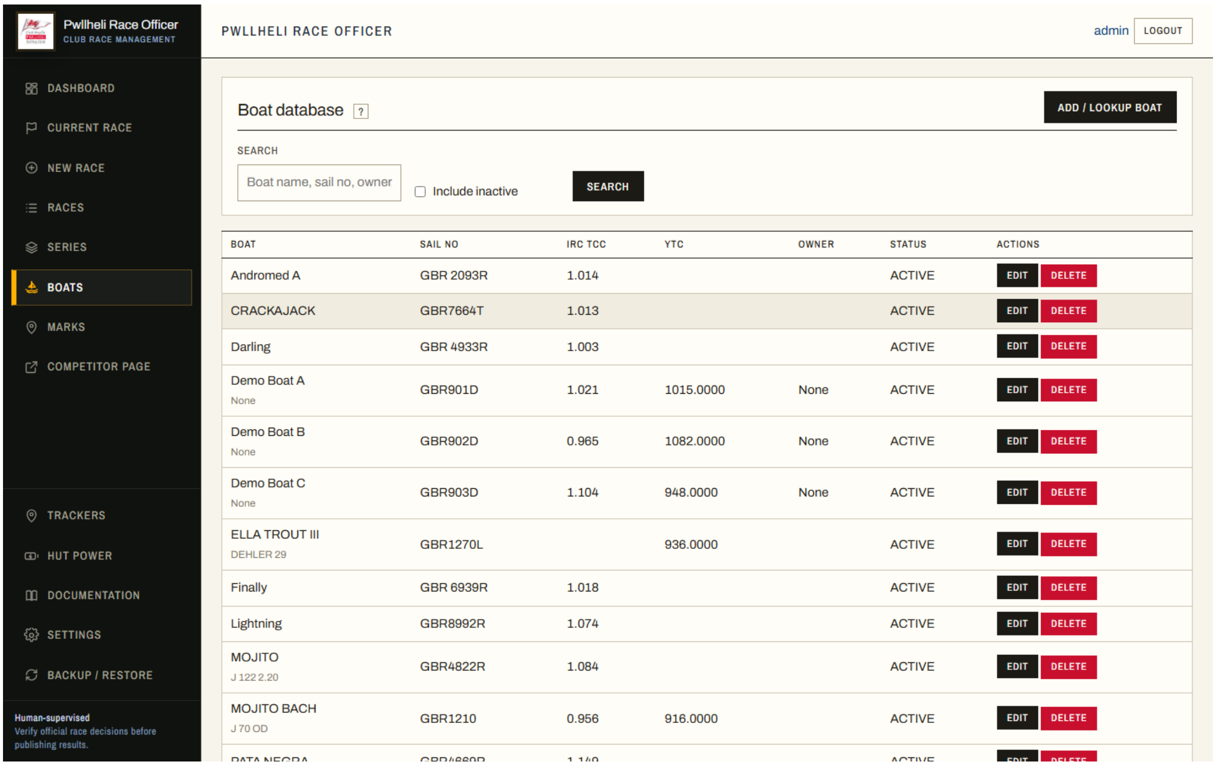
Finding today's race

The side menu carries a **Current race** link between Dashboard and New race, which opens the race sheet for the race the club is running now. The current race is the latest one that still has boats **racing**, and otherwise simply the latest race — so creating a new race sheet does not take the screen away from a race still being sailed. If a display or a page seems stuck on the wrong race, look at the previous race's sheet for a boat nobody finished or retired.

PART II — BOATS, MARKS AND COURSES

The boat database

Boats are stored once, with both IRC and YTC ratings, so they don't need re-entering for every race. A rating is copied into a race entry as a snapshot the moment the boat is added to that race — editing the boat record later only affects races you add it to afterwards (an individual entry's rating can still be corrected on the race's Entries & finish times tab).

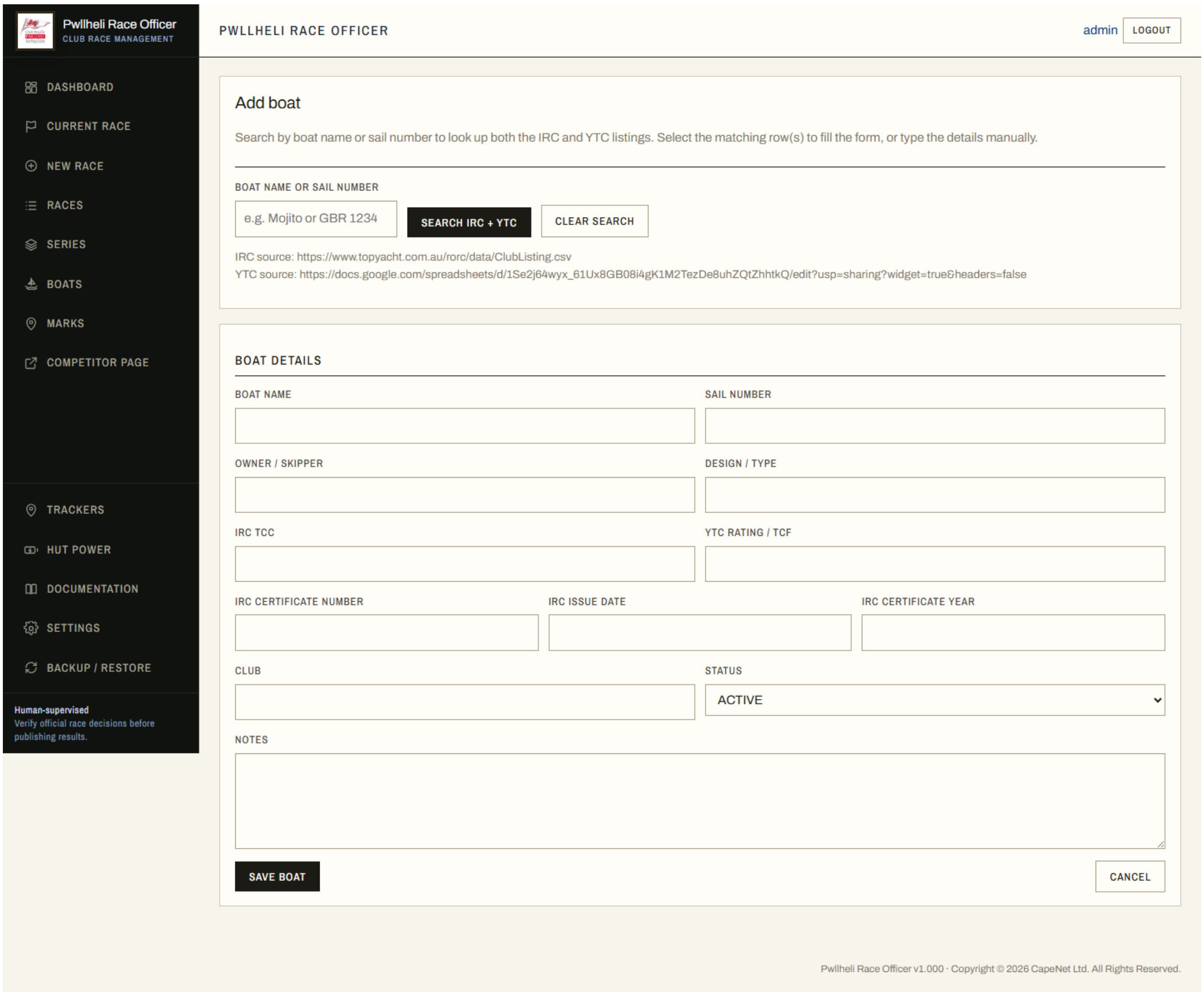


The Boat database: search, and the boat list with IRC/YTC ratings.

Adding a boat

There are two ways to add or update a boat, both from Boats → Add / lookup boat:

- **Search IRC + YTC** — type a boat name or sail number and the app queries both configured rating sources at once (Chapter 18), showing matching rows from each list side by side. Selecting a row copies its fields into the form below.
- **Manual entry** — type the details directly: boat name, sail number, owner, design, IRC TCC, YTC rating, IRC certificate number/issue date/year, club and notes.



Add / lookup boat: search both rating sources, or fill in the form manually.

Two further pages support **bulk** import: importing a page of matches from the configured IRC CSV listing, and importing from the configured YTC Google Sheet/CSV listing — useful for loading a whole fleet at the start of a season rather than one boat at a time.

> **NOTE**
>
> The **Include inactive** checkbox on the Boats list is off by default, so deactivated boats stay out of the way without being permanently deleted.

Marks and courses

The Marks page lists the club's marks. **Administrators** can **add** a simple mark (code, name, position in decimal degrees, buoy and top-mark — the position text is generated automatically) and **delete** a mark that is not in use, directly from this page. A mark cannot be deleted while it is used in a fixed course or as the start/finish seaward mark, or if it is a compound mark or one of its components — the page shows **In use** with the reason instead of a Delete button. Race officers see the list read-only. Fixed courses, the start/finish line and compound marks are still edited by hand in the underlying JSON files (**data/courses.json**, **data/start_finish.json**, and compound entries in **data/marks.json**).

Waypoints

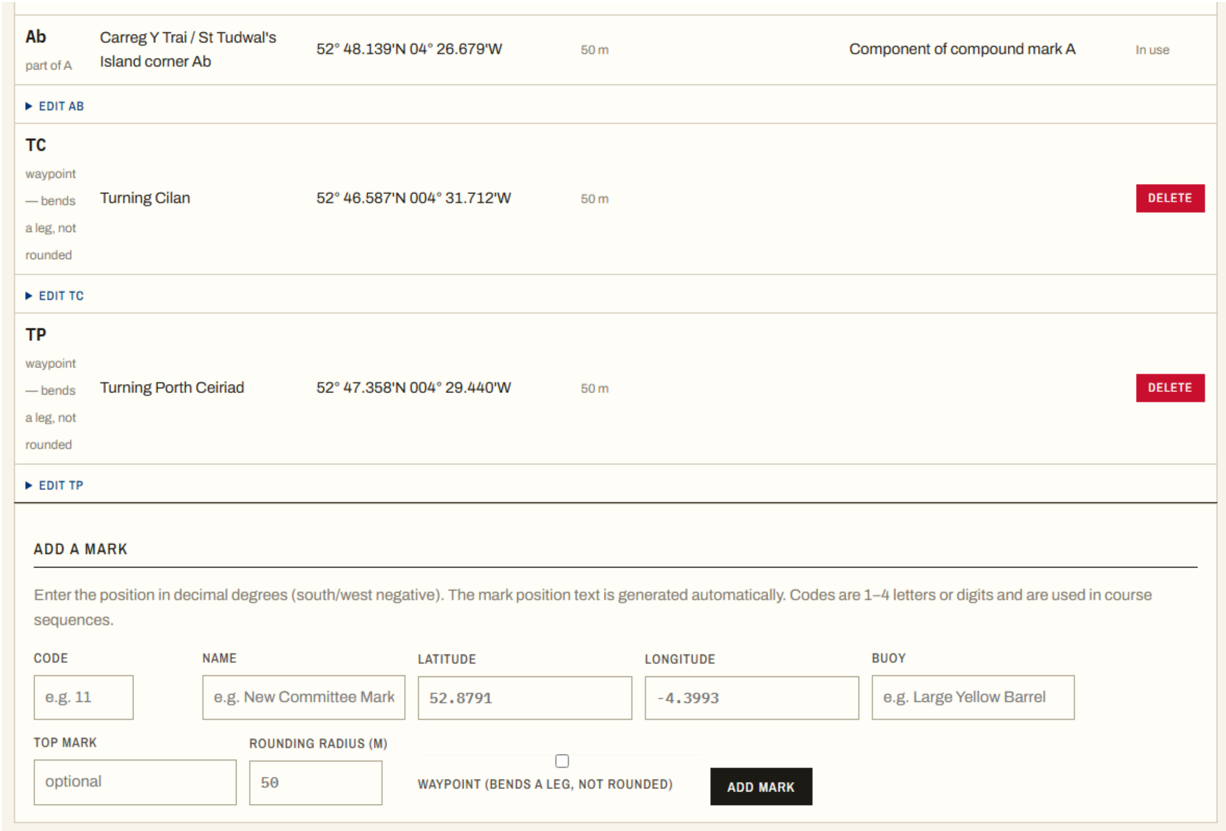
A **waypoint** bends a leg round a headland. It is not a mark: boats are not asked to round it, it is absent from the course board, the announcement, the shortening options and a boat's marks-rounded count, and nothing is drawn for it on the chart. The course changes direction there and that is all.

It exists because a straight line from mark 2 to the Gwylan Islands crosses Mynytho, Llangian and Trwyn Cilan. Drawing a course over land is the visible half; the arithmetic is the rest. That leg measures **10.90 nm** straight against **13.06 nm** round the corner — 20% short, for the whole leg, in distance-to-go and in the projected leaderboard — and a single bearing of 243°T stands in for 211°T then 278°T. With the wind at 145°T that is a 66° TWA reach and a 133° TWA broad reach being modelled as one leg at 98°, so the polar projection is wrong for both halves.

Add one on the Marks page with the **Waypoint** box ticked; it takes no rounding radius. In the manual course builder it offers a single **Add** rather than Port and Starboard, since boats pass a turning point rather than leaving it on a hand, and it appears as a grey **via** chip. It can be used more than once in a course — an out-and-back round the Gwylans wants it on both legs. The club's two are **TC** (Trwyn Cilan) and **TP** (Porth Ceiriad).

> **NOTE**
>
> A boat passes a waypoint by **drawing level with it**, not by coming near it — a gate rather than a circle. A radius cannot do the job: boats beating past a headland pass a long way offshore, so it would have to be a mile or more, and the radius test fires on proximity alone with no departure test to hold it back — a 2 km radius on a waypoint 12.6 km down the leg would advance the boat while it was still 84% short of the corner. The gate cannot fire early and does not care how far off the boat passes.

A waypoint used by a fixed course, or by any race's made-up course, cannot be deleted; the app names the race. Deleting one would straighten a leg back across the land on a race already sailed, changing its recorded length and bearings after the event. Full detail in **docs/WAYPOINTS.md**.



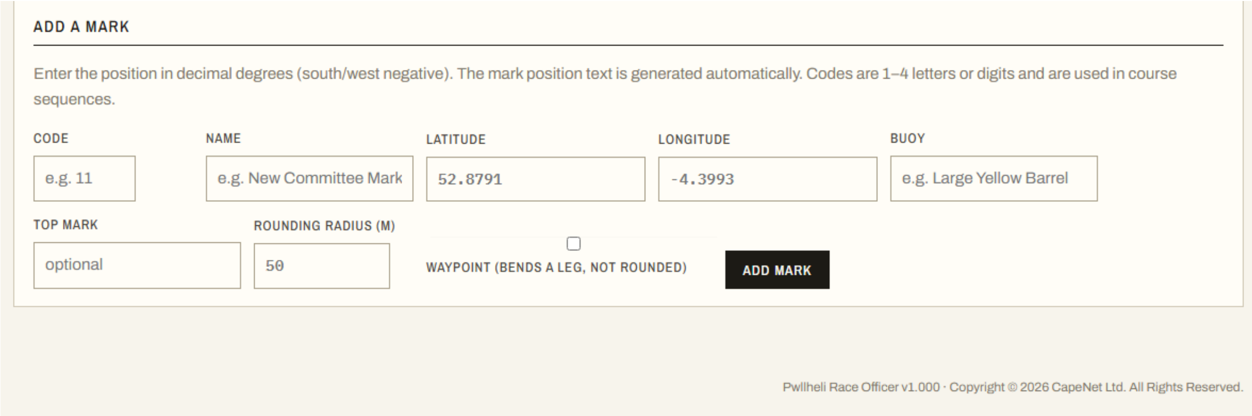
The Marks page (foot of the list): the club's two waypoints, marks that are in use and so cannot be deleted, and the Add a mark form.

Marks move: editing and re-measuring

A laid mark drags in a storm, and the app then looks for boats rounding a buoy that is no longer there. That used to be as bad as it sounds: rounding was judged within a radius of the mark's *recorded* position, and the course walk is **sequential**, so a buoy 60 m adrift made every boat read as never having rounded it and stalled every mark behind it — a fleet stuck at the same mark and no GPS finishes at all. The rounding gate absorbs a drag of that order (Chapter 24): it reaches 750 m along a line through the mark on the hand the course requires, so it asks which side of the mark a boat went, not how close it came.

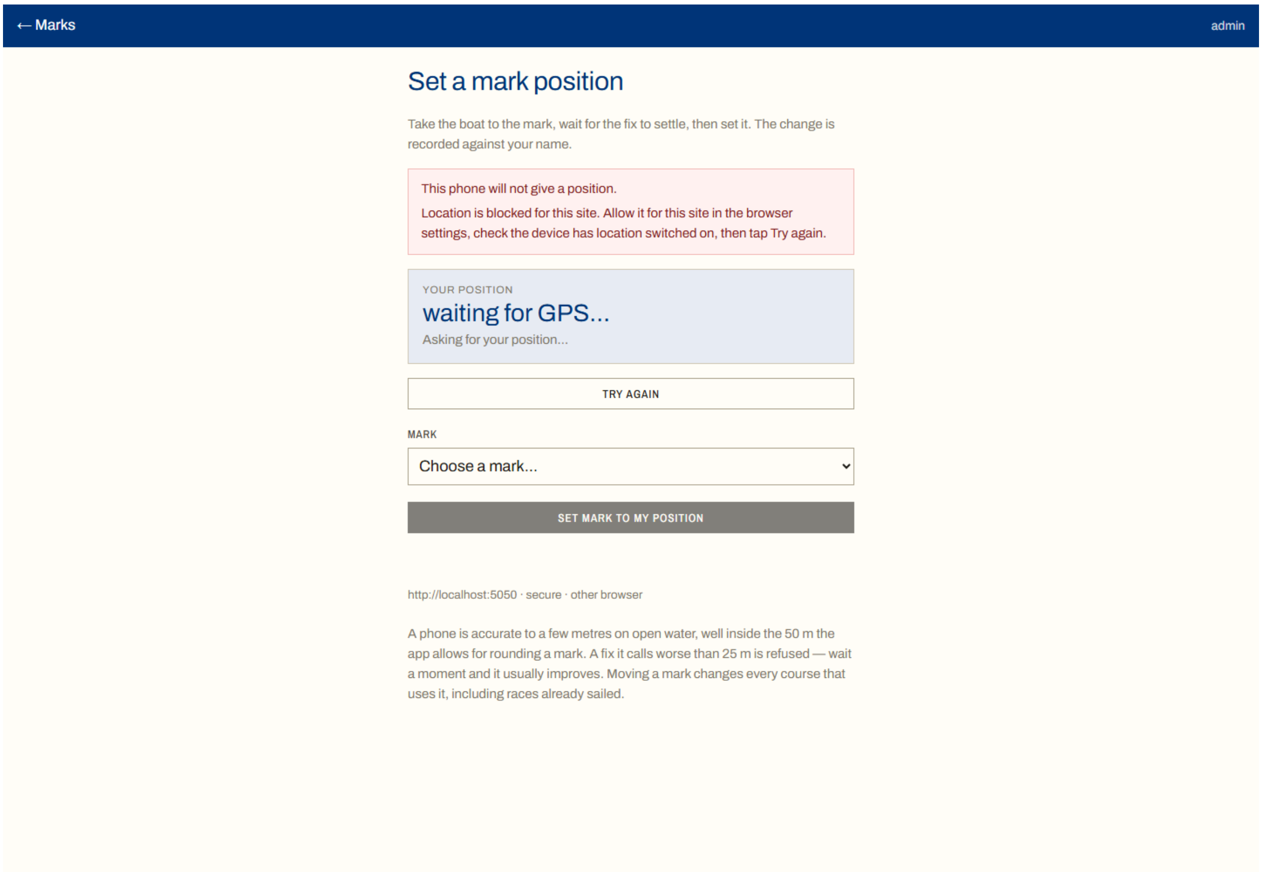
What a wrong position still costs is worth stating, because it no longer announces itself. Distance to go, the leg bearings and the chart all measure to the recorded position. And a boat passing the correct side of the *buoy* but the wrong side of the *recorded* position is on the hand the gate barely reaches, leaving only the radius and the departure test to catch it. So a position is not a constant: it is a measurement that needs retaking after heavy weather. There are two ways.

By typing it in. Marks → Edit on any simple mark, for a position from a chart plotter or a survey. Administrators only. The same panel carries the mark's own **rounding radius**: leave it blank and the mark follows the figure in Settings (50 m by default), including when that figure changes later. Set it only for a mark that needs its own — a buoy on a long scope swings a wide circle, while the same generous figure at a mark rounded twice in one course would start counting roundings that never happened.



Marks → Edit: position, buoy, top mark and the mark's own rounding radius. Blank follows the Settings value.

From a phone on the water. Marks → Set a position from the water is a page of its own, built to be used one-handed in a small boat: take the RIB to the mark, wait for the fix to settle, pick the mark and set it. It shows the position and the accuracy the phone claims, and how far the mark is about to move — the button reads *“Move 4 60 m to here”* rather than “Save”. A fix the phone itself calls worse than 25 m is refused, and a move of more than 2 km must be confirmed; both are enforced on the server, not only in the page.



Set a position from the water: the fix and its accuracy, the mark picker, and how far the mark will move.

It needs the club's **https** address: browsers do not give a position to a page served over plain http, so this cannot work from a bare LAN address however many times the button is pressed. If the page sits on “waiting for GPS…”, the phone has not been asked or has refused — and on an iPhone *each browser* is granted location separately, under its own name in Settings → Privacy & Security → Location Services, so allowing it for Safari does nothing for Chrome. The page names which browser it is in and what to check.

Every change records **who set it, when, from what, and the accuracy claimed**, and keeps the position it replaced. That history is read: a race is drawn, replayed and analysed with the marks as they stood when it was sailed, so correcting a mark today does not redraw races already run. A race is pinned to *when it ended*, which is what makes the usual case work — a drag is normally discovered mid-race, and a correction made while the race is still being sailed counts for that race, including the boats that had already rounded.

Compound marks

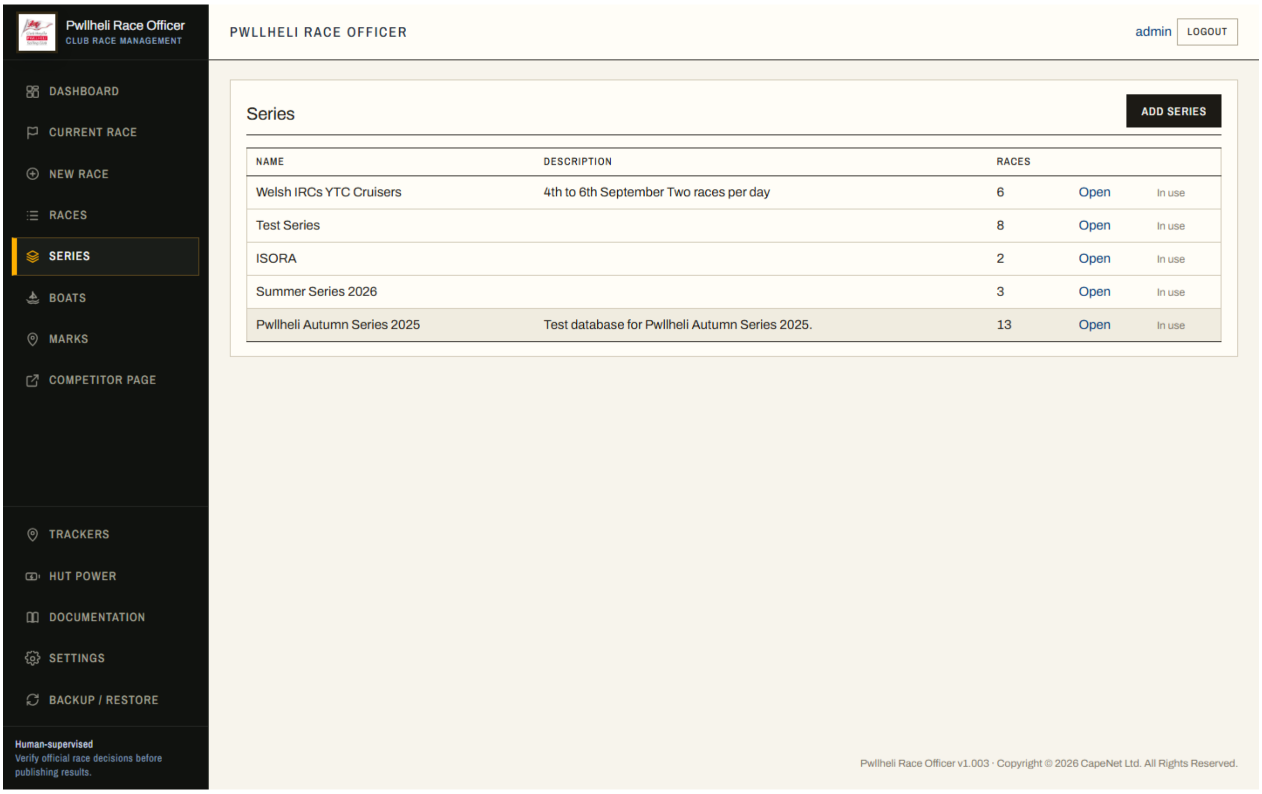
Some marks represent a physical feature — an island group, or a pair of rocks — rather than a single buoy. A compound **parent** mark (e.g. **Y** = Gwylan Islands, **A** = Carreg Y Trai/St Tudwal's Islands) has no coordinates of its own; it lists real-mark **components** (e.g. Ya/Yb) and a port/starboard rounding order. Course boards, the public course sequence and audio announcements only ever name the parent (e.g. “round Yp”), while the course chart, leg analysis and length calculations expand it to the real physical corner points. The manual course builder only offers the parent marks, never the raw corners.

▸ EDIT M					
Y	Gwylan Islands		—	Compound mark: when Y is to port, round Ya then Yb to port; when Y is to starboard, round Yb then Ya to starboard. Yachts must not pass between the two islands. Components: YA, YB	In use
A	Carreg Y Trai / St Tudwal's Island East / St Tudwal's Island West		—	Compound mark: when A is to port, round Aa then Ab to port; when A is to starboard, round Ab then Aa to starboard. Beware rocks between the bell buoy and St Tudwal's Island East. Components: AA, AB	In use
Ya part of Y	Gwylan Islands corner Ya	52° 47.399'N 04° 41.599'W	50 m	Component of compound mark Y	In use
▸ EDIT YA					
Yb part of Y	Gwylan Islands corner Yb	52° 47.070'N 04° 42.027'W	50 m	Component of compound mark Y	In use
▸ EDIT YB					
Aa part of A	Carreg Y Trai / St Tudwal's Island corner Aa	52° 47.707'N 04° 28.323'W	50 m	Component of compound mark A	In use
▸ EDIT AA					
Ab part of A	Carreg Y Trai / St Tudwal's Island corner Ab	52° 48.139'N 04° 26.679'W	50 m	Component of compound mark A	In use
▸ EDIT AB					

Compound marks Y and A, with their real-position components Ya/Yb and Aa/Ab.

Series setup

A series (e.g. "Autumn Series 2025") is a named group of races scored together with discards. **Add series** on the **Series** page opens a form carrying the whole setup — name, discards, rating bands and default start plan — so a series arrives configured rather than blank. It is the same form as **Edit series details** on the series itself, which is where it is changed afterwards.

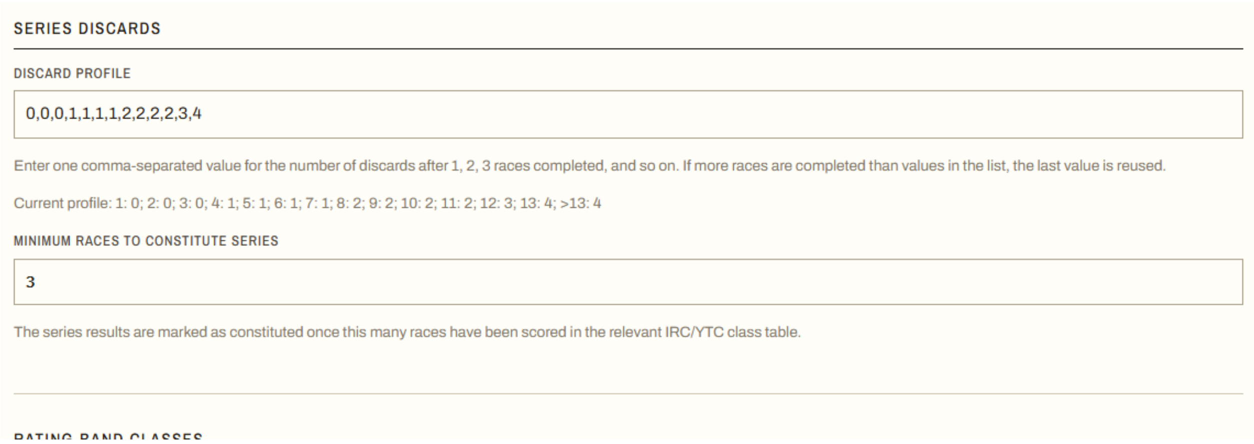


The Series page. Winter Series 2027 has no races in it yet and can be deleted; every series that has been raced shows In use instead.

A series is **deleted** from the same list, or from the bottom of **Edit series details**, and only when nothing is in it: one still holding races shows *In use* and names how many are in the way. Empty it first — each race's series is changed on its own "Course & start" tab, to another series or to none — or delete the races. Neither a cascade nor an orphan is offered on purpose: a series carries the rating bands, start plan and discard profile its races were *scored under*, so deleting it out from under them would drop a season from the standings while every race went on claiming to belong to it.

Discards and constitution

The **discard profile** is a comma-separated list (e.g. **0,0,1,1,1,1,2,2,2**) giving the number of a competitor's worst scores dropped once 1, 2, 3… races are completed; the last value is reused if more races are sailed than the list has entries. **Minimum races to constitute series** sets when a class's results stop being labelled provisional.



SERIES DISCARDS

DISCARD PROFILE

0,0,0,1,1,1,1,2,2,2,2,3,4

Enter one comma-separated value for the number of discards after 1, 2, 3 races completed, and so on. If more races are completed than values in the list, the last value is reused.

Current profile: 1: 0; 2: 0; 3: 0; 4: 1; 5: 1; 6: 1; 7: 1; 8: 2; 9: 2; 10: 2; 11: 2; 12: 3; 13: 4; >13: 4

MINIMUM RACES TO CONSTITUTE SERIES

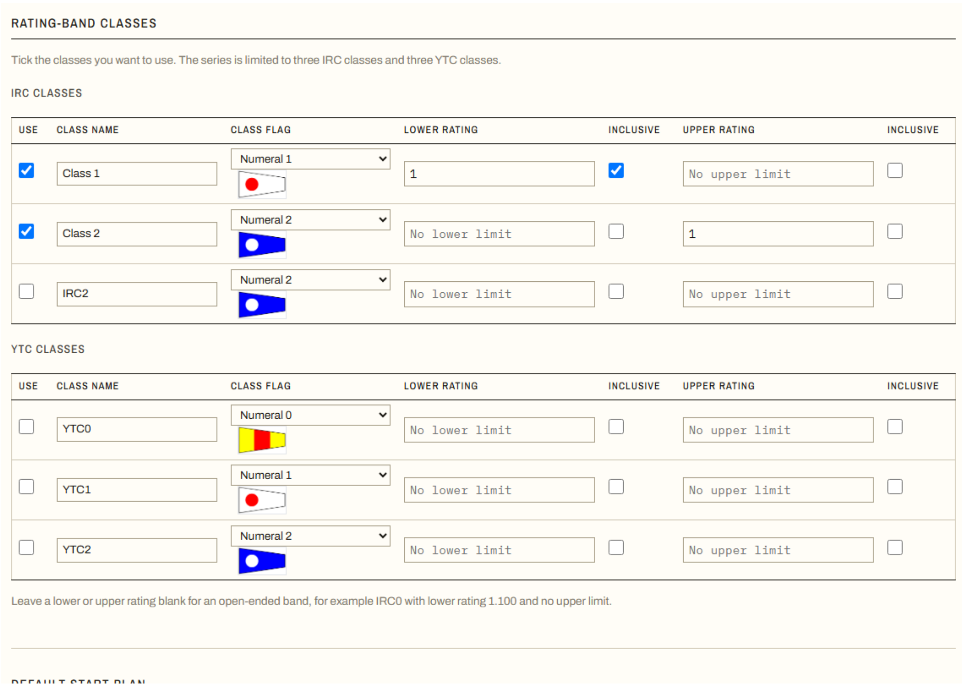
3

The series results are marked as constituted once this many races have been scored in the relevant IRC/YTC class table.

Discard profile, constitution threshold, and the series-level publish/export buttons (Chapter 12).

Rating-band classes

Up to three IRC and three YTC classes, each with a name, a numeral class flag, and an optional rating band (e.g. IRC Class 1 = 1.000 and above; leave a bound blank for open-ended). Boats are grouped into these classes automatically by their rating.



RATING-BAND CLASSES

Tick the classes you want to use. The series is limited to three IRC classes and three YTC classes.

IRC CLASSES

USE	CLASS NAME	CLASS FLAG	LOWER RATING	INCLUSIVE	UPPER RATING	INCLUSIVE
☑	Class 1	Numeral 1	1	☑	No upper limit	☐
☑	Class 2	Numeral 2	No lower limit	☐	1	☐
☐	IRC2	Numeral 2	No lower limit	☐	No upper limit	☐

YTC CLASSES

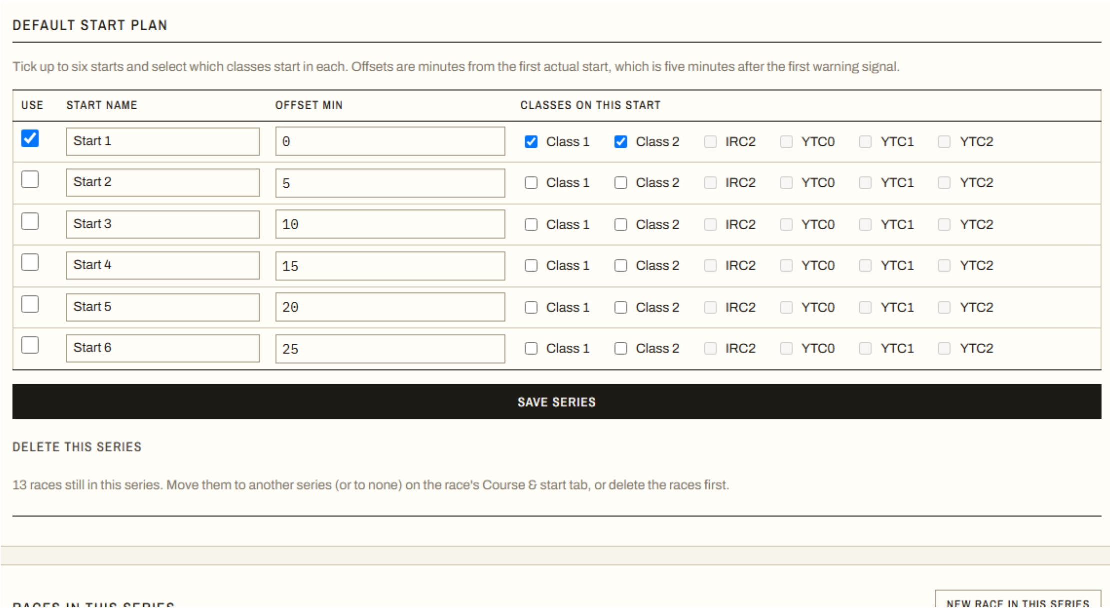
USE	CLASS NAME	CLASS FLAG	LOWER RATING	INCLUSIVE	UPPER RATING	INCLUSIVE
☐	YTC0	Numeral 0	No lower limit	☐	No upper limit	☐
☐	YTC1	Numeral 1	No lower limit	☐	No upper limit	☐
☐	YTC2	Numeral 2	No lower limit	☐	No upper limit	☐

Leave a lower or upper rating blank for an open-ended band, for example IRC0 with lower rating 1.100 and no upper limit.

IRC rating-band classes with numeral flags and rating bands.

Default start plan

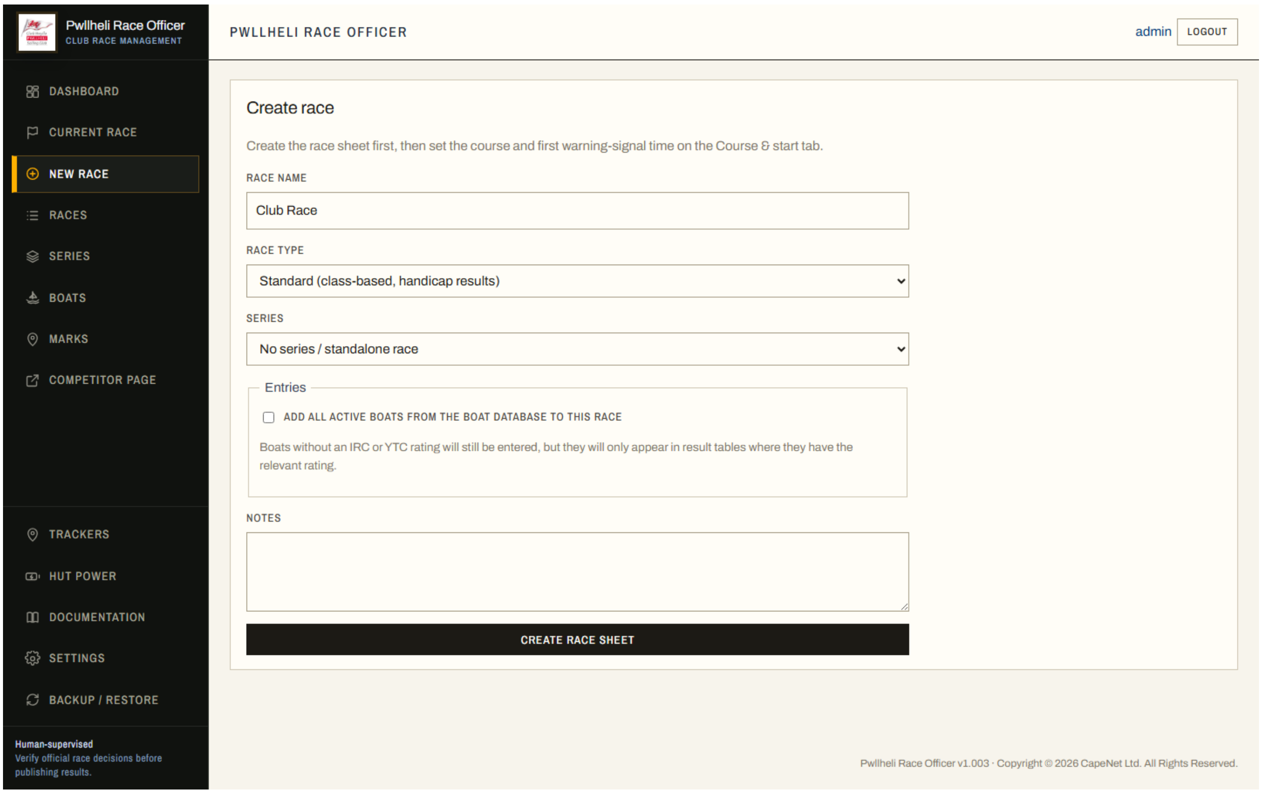
Up to six starts, each with a time offset in minutes from the first actual start (five minutes after the first warning signal) and a choice of which classes start on it. This becomes the default for every new race added to the series; it can still be reviewed per race.



The default start plan.

Creating a race and setting its course

Race creation is deliberately minimal: name, series, and an optional bulk-add of every active boat. A race takes its classes from its series' rating bands, so there is nothing to type about class or fleet. The course and warning time are set next, on the race sheet itself.



Creating a race.

Choosing the finish line

Almost every race finishes where it starts, on the club line between the CHPSC Bridge window and the ODM (mark **O**), and the **Finish line** selector on the Course & start tab can be left alone. Some races are not sailed to it: an **ISORA** passage race finishes on the transit between the Pwllheli Fairway Buoy (mark **F**) and the bridge at Plas Heli, bearing 297° magnetic — a line about 1.6 km long whose shore end is nearly 800 m from the club one. Pick it and everything follows: the chart draws it, GPS finishes are detected on it, and the competitor page and clubhouse display show it.

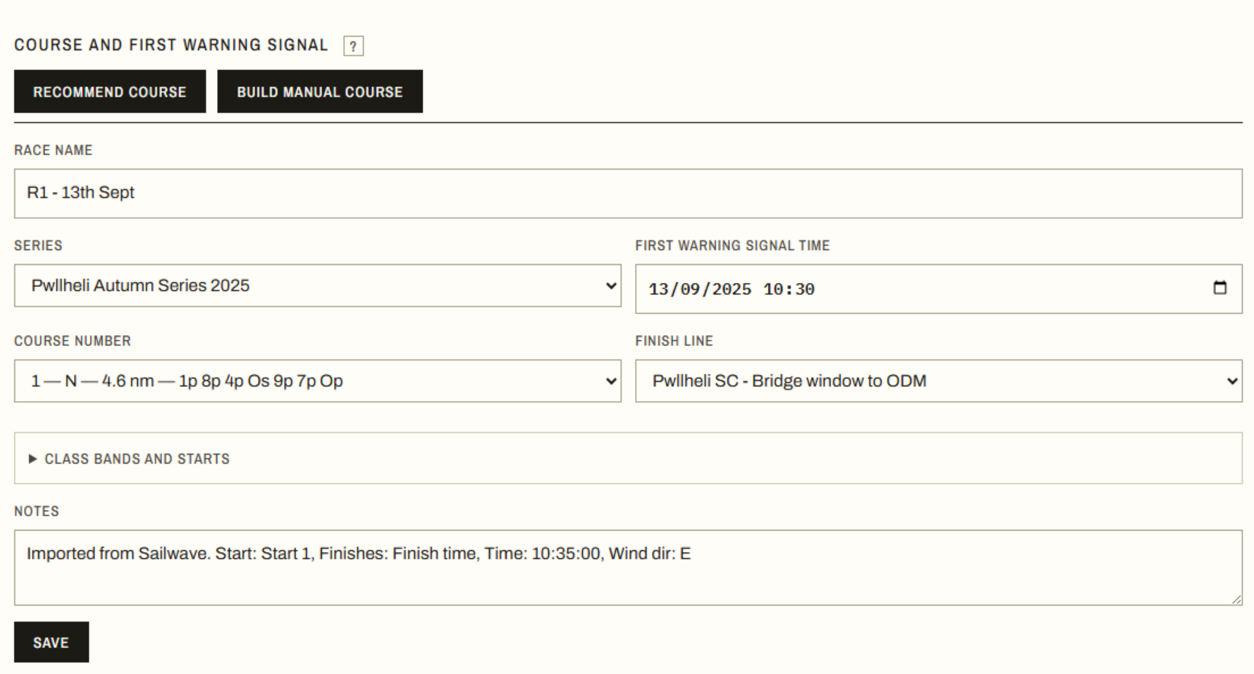
The start line does not change. Races start on the club line whatever they finish on, so with a different finish selected the chart draws two lines, marked *Start line* and *Finish line*. Boats **cross** the finish line rather than rounding the mark at the end of it: when a course's last mark is the finish line's own mark — O on the club line, F on the ISORA one — no rounding is required there. A course ending anywhere else, including a shortened one, still has to reach the line. The seaward end of each line is a mark, so re-measuring it moves the line with it; ping F from the water and the ISORA finish follows.

Getting this wrong does not fail loudly. The crossing test simply never fires, and every boat sits unfinished at the end of a race that has plainly finished — so set it before the start.

Course & start tab

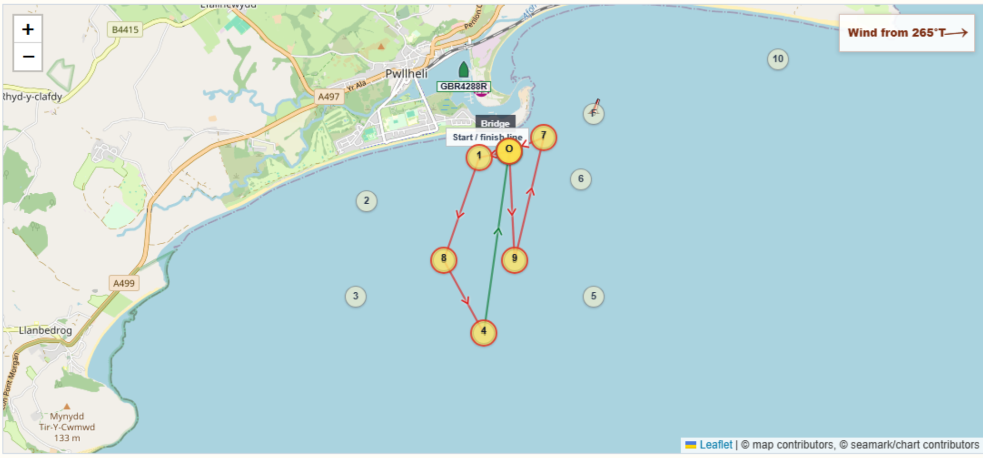
A “?” beside a heading opens the explanation. The race page keeps the controls at the top and the reasoning one click away, so what stays on screen is what changes what you do next — *AP is up since 14:02*, *this race has started*, *whole minutes only*. The *why* and the worked examples are behind the **?**, and everything in those popups is also in this manual.

Set the **first warning signal time** — the actual start always follows five minutes later. Choose a course by picking a fixed course number, using **Recommend course** to have the app suggest a standard course from the live start-hut wind and a chosen boat polar, or **Build manual course** to lay out a one-off course from individual marks.



Course & start: warning-signal time and course selection.

Once a course is selected, the course chart and a predicted leg-by-leg timing table (distance, bearing, suggested sail and target boat speed) are calculated from the chosen polar and latest wind.



▼ POSITION ON THE WATER (GPS)

Every boat entered in the race, ordered by how far round the course it is — this is the order on the water, not corrected for handicap. Tracked boats also appear on the course chart above; the list refreshes every few seconds. Assign trackers. ?

#	BOAT	SAIL	MARKS	NEXT	TO GO	SOG	FIX	SAILING TO
1	Mojito	GBR4288R	Finished	—	—	0.0 kn	51s ago	—
2	Darling	GBR 4933R	—	—	—	—	No tracker	—
3	Mojito Bach	GBR 1210	—	—	—	—	No tracker	—
4	Sgrech	GBR 983	—	—	—	—	No tracker	—
5	Andromed A	GBR 2093R	—	—	—	—	No tracker	—
6	Finally	GBR 6939R	—	—	—	—	Not reporting	—
7	Quattro	GBR 3152	—	—	—	—	No tracker	—
8	Lightning	GBR8992R	—	—	—	—	No tracker	—

▼ LEG ANALYSIS USING ODM O AS START/FINISH REFERENCE

Predicted time: 43 min · First leg: 8° Starboard · Shape: 2 upwind · 5 reach · 0 downwind · Sail changes: 4

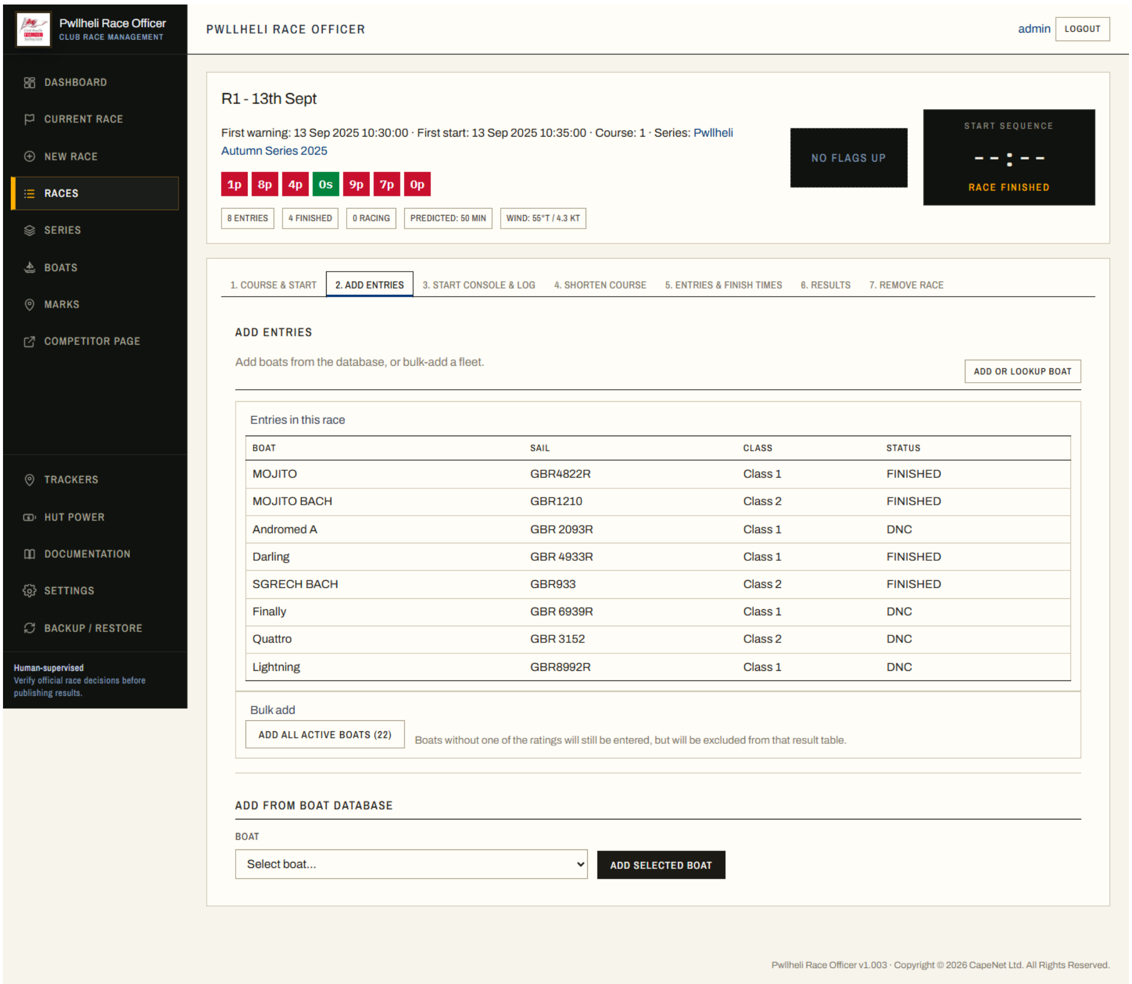
LEG	DISTANCE	BEARING	TWA	SIDE	SAIL	TARGET	TIME	POINT
O → 1	0.21 nm	257°T	8°	Starboard	J1	5.28 kt @ 45°	3 min	Beat/VMG
1 → 8	0.74 nm	199°T	66°	Starboard	J1	6.33 kt	7 min	Close reach
8 → 4	0.57 nm	151°T	114°	Starboard	A2	6.80 kt	5 min	Beam reach
4 → O	1.26 nm	8°T	103°	Port	A2	6.97 kt	11 min	Beam reach
O → 9	0.75 nm	177°T	88°	Starboard	A0	6.92 kt	6 min	Close reach
9 → 7	0.86 nm	13°T	108°	Port	A2	6.97 kt	7 min	Beam reach
7 → O	0.25 nm	248°T	17°	Starboard	J1	5.28 kt @ 45°	3 min	Beat/VMG

The Course & start tab below the form: the course chart, the fleet's position on the water, and the predicted leg-by-leg timing.

Adding entries

On the Add entries tab, boats are added from the boat database, either individually or in bulk:

- **Bulk add** — every active boat, or every boat already in a given class, in one click.
- **Add from boat database** — look up one boat and add it, optionally overriding its class for this race.

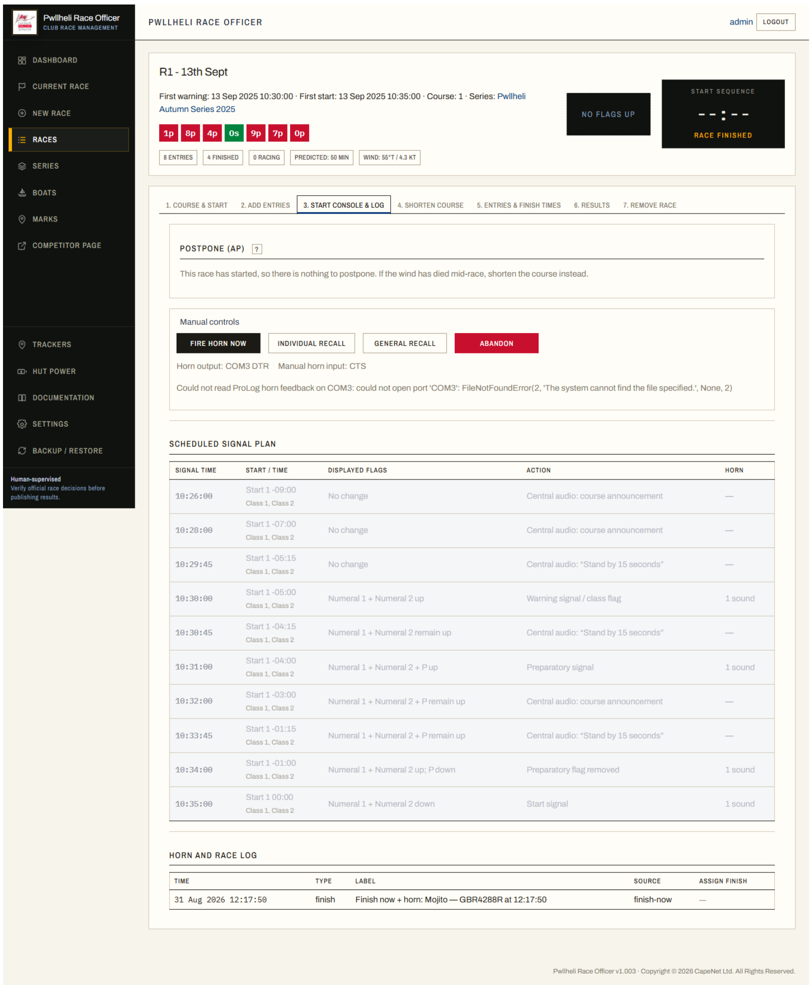


Add entries: current entries, bulk-add and add-from-database controls.

> **NOTE**
>
> Boats without a rating for a given system are still entered — they're simply excluded from that system's result table until a rating is added.

Running the start sequence

The Start console & log tab is the race-day working screen. The scheduled signal plan is generated automatically from the warning time and start plan.



Start console & log: postponing, manual controls, the scheduled signal plan and the horn/race event log.

Manual controls cover situations the schedule can't predict:

- **Fire horn now** — sound the horn immediately, outside the scheduled plan.
- **Individual recall** — log that specific boats were called back over the line.
- **General Recall** — log a full recall of the start.
- **Postpone (AP)** — the real thing, not a note: two horn blasts, the start sequence held, and AP on every page that shows flags. See below.
- **Abandon** — log an abandoned race.

IMPORTANT

The visual flag panel models the normal warning/preparatory/start class-flag sequence, but does not model every recall, postponement or abandonment flag combination — the race officer remains responsible for the actual flags flown and the on-the-water decision.

Postponing a start

If the wind dies, the line is not ready or the fleet is not there, **postpone — do not move the start time**. Moving it signals nothing to the fleet, and if the sequence is already running it drops the rest of it: boats hear a warning, then no preparatory signal and no gun. Postponing is on this tab because this is the tab that is open before a start.

FIELD / CONTROL	WHAT IT DOES
AP	Races not started are postponed. The warning signal will be made one minute after AP is removed.
AP over H	Races not started are postponed. Further signals ashore.
AP over A	Races not started are postponed. No more racing today.

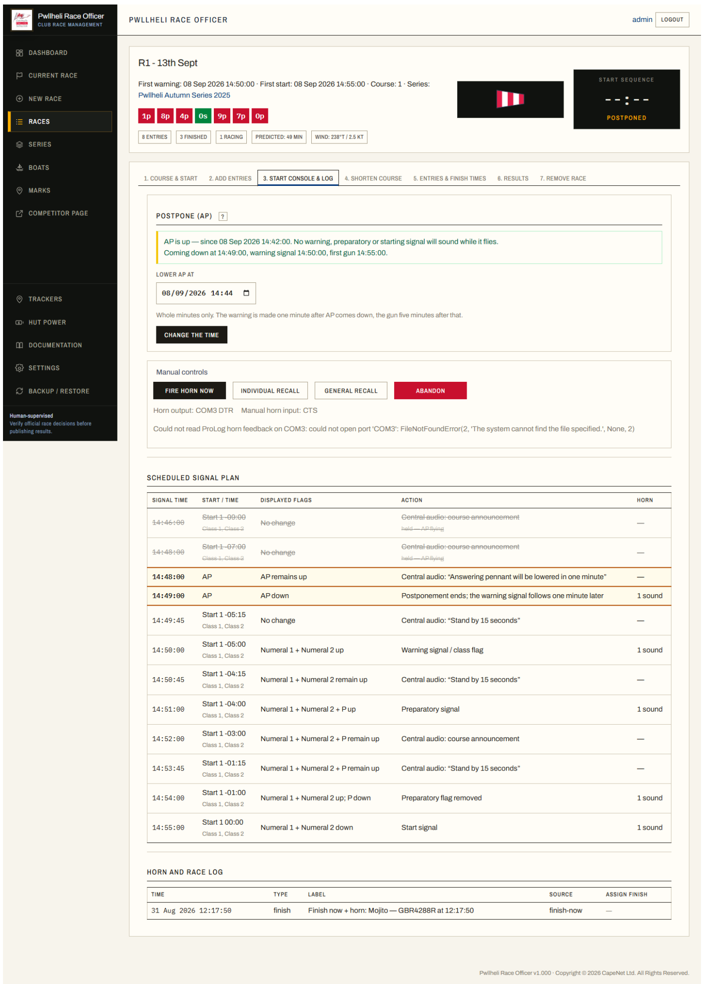
Flying one sounds **two horn blasts**, holds the start sequence — no warning, preparatory or starting signal sounds while the flag is up — shows the flag on the race sheet, the competitor pages and the clubhouse display, and records it in the race log. The race keeps its scheduled time: nothing is rescheduled behind anybody's back.

When the wind is back, say **when AP comes down** — whole minutes, as with the first warning signal, and at least ninety seconds away so the announcement a minute beforehand has room. **The warning signal is one minute after that and the gun five minutes after the warning**, so choosing 14:20 gives a warning at 14:21 and a first gun at 14:26. That is the point of the flag: you decide when you are ready, not what time the wind will return. The fleet is told when you set the time, again if you change it, a minute before the flag moves, and once more as it comes down with its single sound.

IMPORTANT

AP postpones races that have *not started*. After the first gun the button is not offered: stopping a race under way is abandonment (flag N), which this app does not signal — if the wind has died mid-race, shorten the course instead. A race postponed *before* its gun can always still be lowered, so the flag cannot get stuck up.

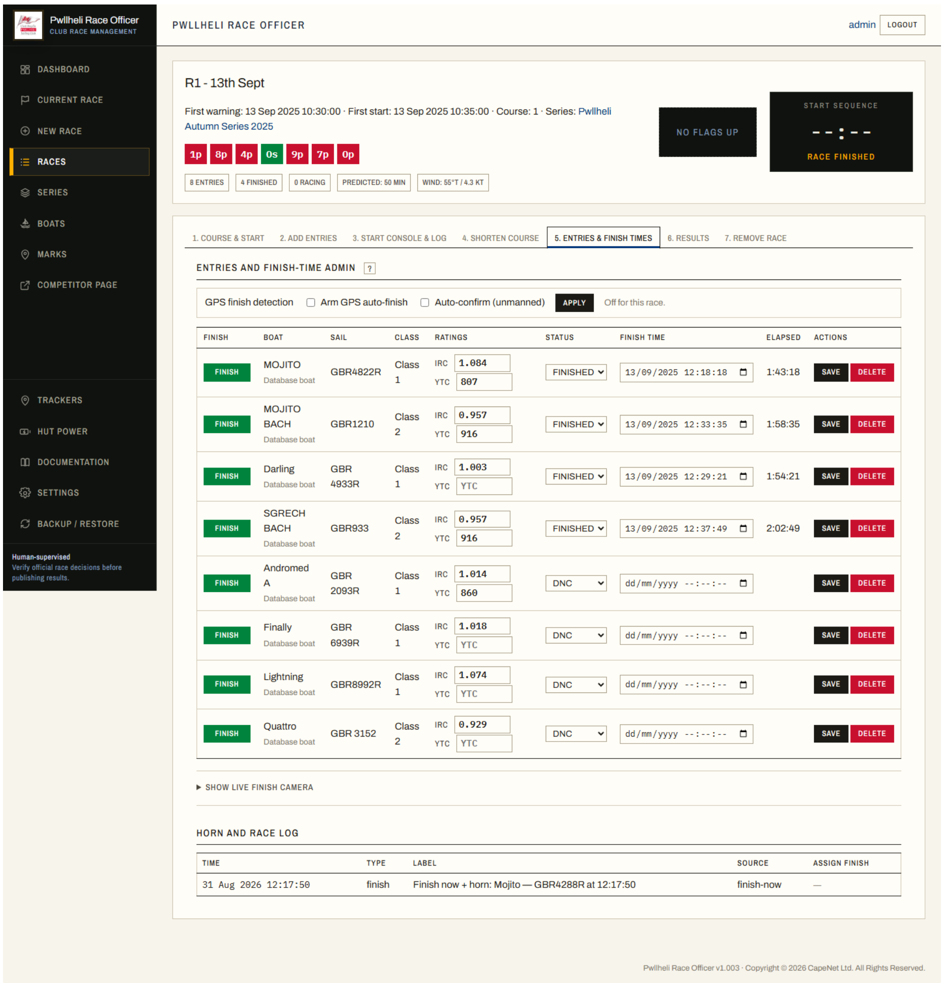
The scheduled signal plan below follows the postponement: signals due before the flag comes down are struck through and marked *held*, because they will not be made, and the announcement and the flag coming down appear as rows of their own. The start video follows it too — the clip booked for the old gun is cancelled, and a new one booked for the gun that actually happens.



AP flying: the flag in the panel, the clock reading Postponed, the minute it comes down and the times that follow from it — and the signal plan with the held signals struck through above the two AP rows.

Recording finishes

As boats cross the line, use the Entries & finish times tab. The blue **Finish** button records the finish time, sounds the horn and adds a log entry in one action. For anything other than a normal finish, set a status instead (see the status-code table in Chapter 21).

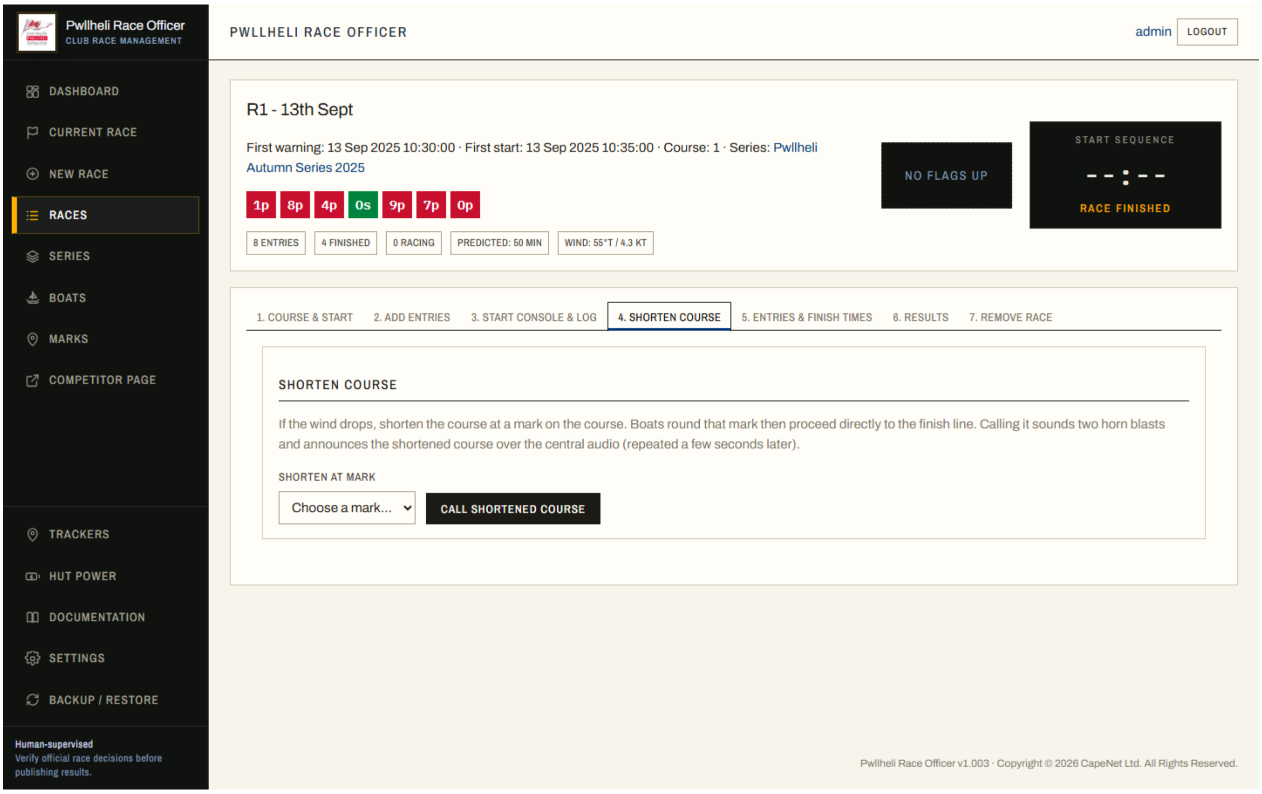


Entries & finish times: one row per boat with its ratings, status and finish/elapsed time.

Each entry's IRC/YTC rating is editable here — useful for a post-entry correction. A live finish-camera view can also be shown from this tab when a camera is configured (Chapter 16).

Shortening the course

If the wind drops or time runs short, the **Shorten course** tab (between *Start console & log* and *Entries & finish times*) shortens the course at a mark: choose a mark from the course sequence and press **Call shortened course**. The app sounds **two horn blasts** and then — once the horns have finished, so it is not drowned out — makes a central-audio announcement — “Shortened course called on mark <mark> — after this mark proceed to finish” — repeated a few seconds later. It flies International Code flag **S** (blue square on white) in the flag panel on both the race and public competitor pages until all boats are no longer racing, records the call in the race log, shows a **Shortened course** banner, and truncates the shown course (and predicted-time/leg analysis) to end at that mark with → **Finish** after it — the course chart drawing a dashed final leg from that mark directly to the finish line. Boats round the chosen mark and sail directly to the finish; finish times are recorded here as usual. **Clear shortened course** undoes a mistaken call. The announcement needs central audio enabled (Chapter 13) to be heard over the VHF/PA.



The Shorten course tab: pick a mark from the course and press Call shortened course.

Race and series results

The Results tab calculates corrected results as soon as finish times/statuses are recorded — there is no separate “compute” step. IRC corrected time = elapsed × IRC TCC; YTC corrected time = elapsed × 1000 ÷ YTC number. Separate tables are produced per rating-band class, with an overall table where classes share a start.

IRC OVERALL RESULTS

Corrected = elapsed × IRC TCC · Start 1: 21 Jul 2026 11:45:00

RANK	BOAT	SAIL	START	IRC TCC	STATUS	ELAPSED	CORRECTED	FINISH VIDEO
1	MOJITO BACH	GBR1210	Start 1 11:45:00	0.957	FINISHED	0:05:26	0:05:12	View
2	SGRECH BACH	GBR933	Start 1 11:45:00	0.956	FINISHED	0:05:28	0:05:14	View
3	Darling	GBR 4933R	Start 1 11:45:00	1.003	FINISHED	0:05:37	0:05:38	View
4	Andromed A	GBR 2093R	Start 1 11:45:00	1.014	FINISHED	0:05:35	0:05:40	View
5	Finally	GBR 6939R	Start 1 11:45:00	1.018	FINISHED	0:05:49	0:05:55	View
6	Lightning	GBR8992R	Start 1 11:45:00	1.074	FINISHED	0:05:50	0:06:16	View
7	MOJITO	GBR4822R	Start 1 11:45:00	1.084	FINISHED	0:05:51	0:06:20	View
8	PATA NEGRA	GBR4669R	Start 1 11:45:00	1.149	FINISHED	0:05:52	0:06:44	View
	QUATTRO	GBR 3152R	Start 1 11:45:00	0.925	RET			—

IRC1 RESULTS

Corrected = elapsed × IRC TCC · Start 1: 21 Jul 2026 11:45:00

RANK	BOAT	SAIL	START	IRC TCC	STATUS	ELAPSED	CORRECTED	FINISH VIDEO
1	Darling	GBR 4933R	Start 1 11:45:00	1.003	FINISHED	0:05:37	0:05:38	View
2	Andromed A	GBR 2093R	Start 1 11:45:00	1.014	FINISHED	0:05:35	0:05:40	View
3	Finally	GBR 6939R	Start 1 11:45:00	1.018	FINISHED	0:05:49	0:05:55	View
4	Lightning	GBR8992R	Start 1 11:45:00	1.074	FINISHED	0:05:50	0:06:16	View
5	MOJITO	GBR4822R	Start 1 11:45:00	1.084	FINISHED	0:05:51	0:06:20	View
6	PATA NEGRA	GBR4669R	Start 1 11:45:00	1.149	FINISHED	0:05:52	0:06:44	View

IRC2 RESULTS

Corrected = elapsed × IRC TCC · Start 1: 21 Jul 2026 11:45:00

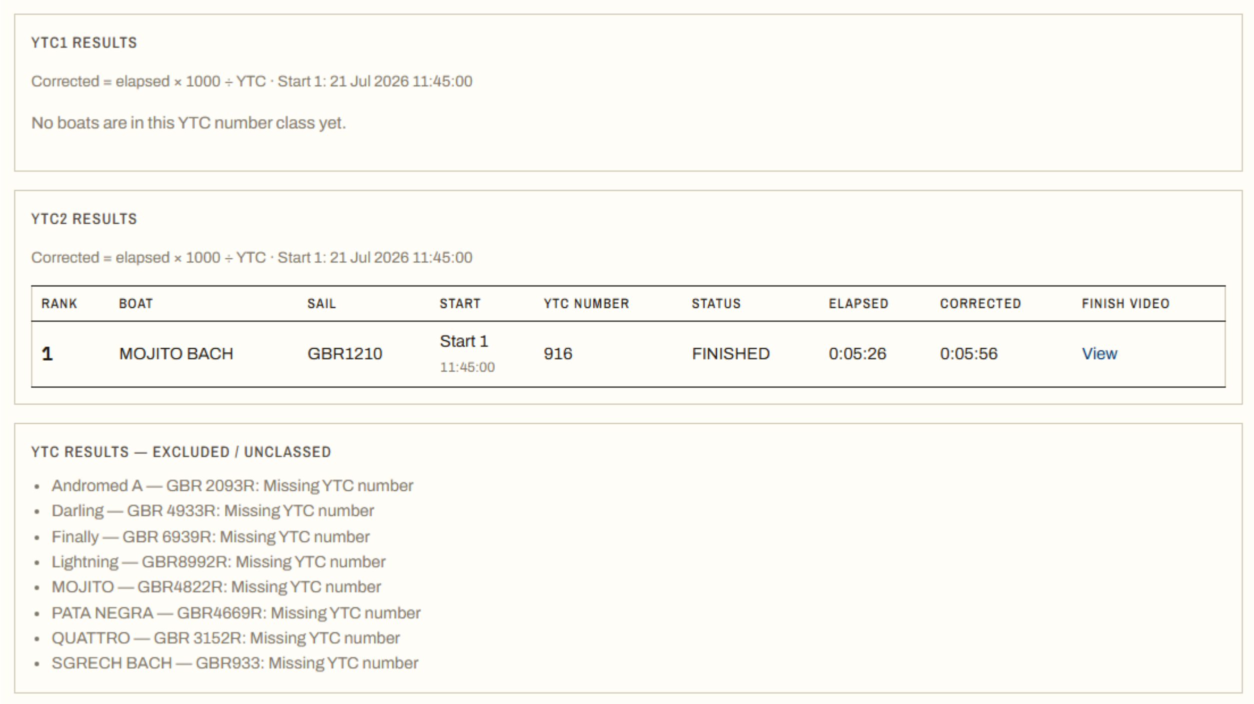
RANK	BOAT	SAIL	START	IRC TCC	STATUS	ELAPSED	CORRECTED	FINISH VIDEO
1	MOJITO BACH	GBR1210	Start 1 11:45:00	0.957	FINISHED	0:05:26	0:05:12	View
2	SGRECH BACH	GBR933	Start 1 11:45:00	0.956	FINISHED	0:05:28	0:05:14	View
	QUATTRO	GBR 3152R	Start 1 11:45:00	0.925	RET			—

YTC OVERALL RESULTS

Corrected = elapsed × 1000 ÷ YTC · Start 1: 21 Jul 2026 11:45:00

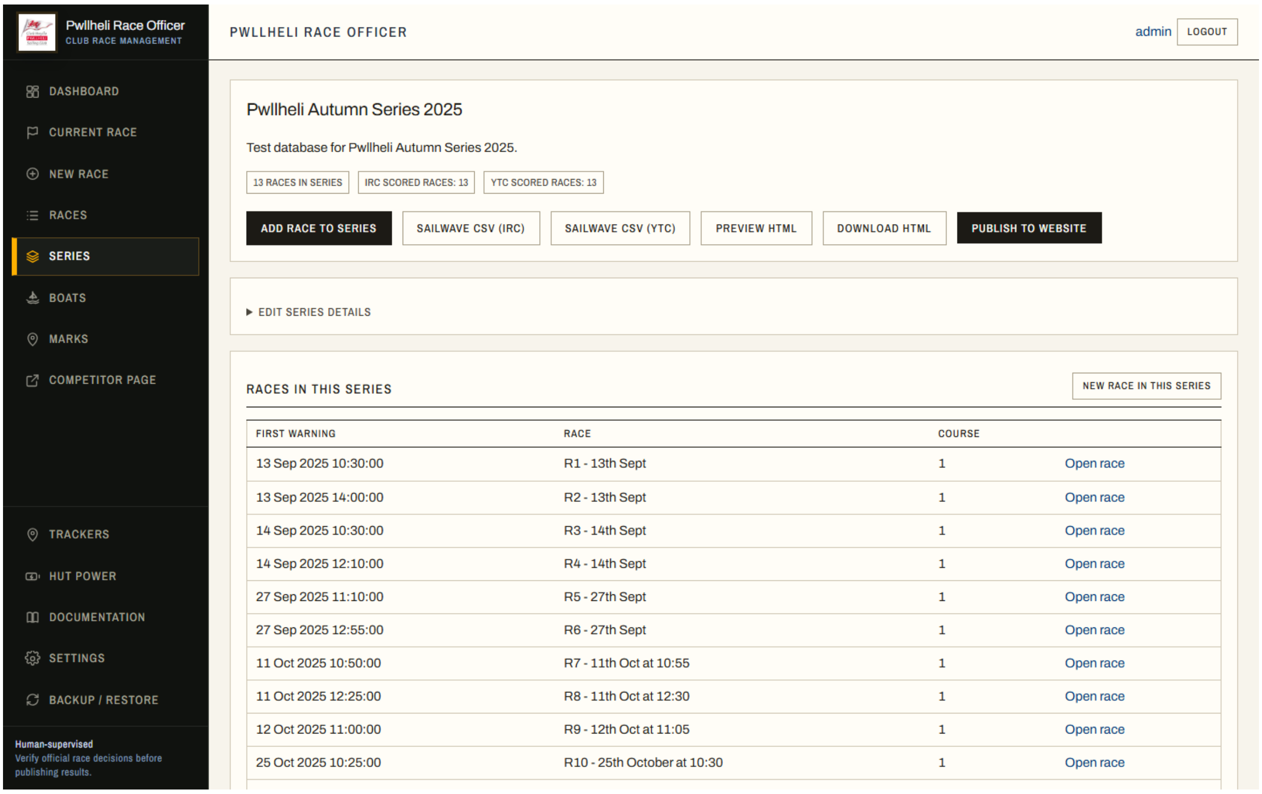
RANK	BOAT	SAIL	START	YTC NUMBER	STATUS	ELAPSED	CORRECTED	FINISH VIDEO
1	MOJITO BACH	GBR1210	Start 1 11:45:00	916	FINISHED	0:05:26	0:05:56	View

Race results (1 of 2): status and the first results table.



Race results (2 of 2): remaining classes and any boats excluded from a table for missing a rating.

Download IRC/YTC CSV exports this race's results. On the series page, **Series results** rolls every scored race into running totals automatically — discarded scores appear struck through, ties are broken per RRS Appendix A8 (race ties use A7 equal-points scoring), and each table shows whether it is yet constituted.



Series results, with running totals and discards.

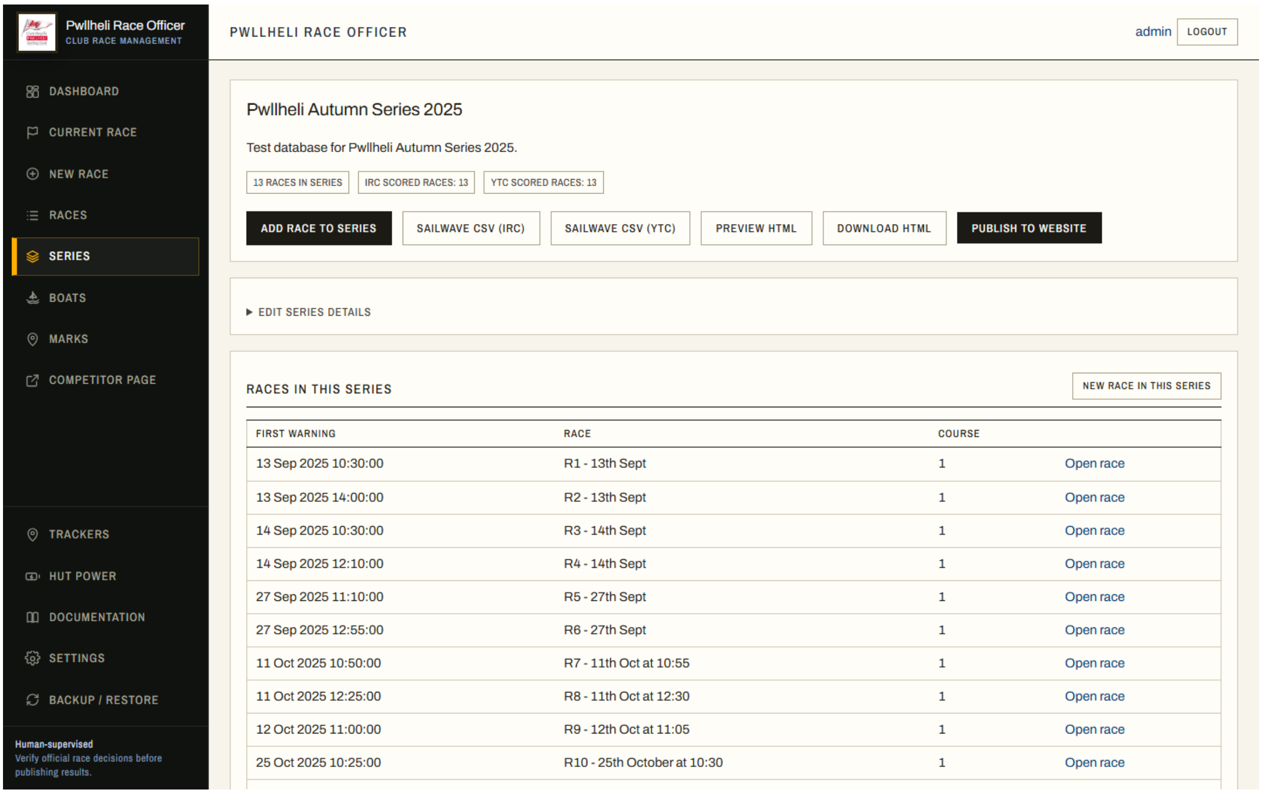
Publishing results

Publishing is done from the **Series** page and always produces a document covering every race in the series: **Download CSV** (spreadsheet), **Preview HTML** (opens it in a tab to look at), **Download HTML** (a single self-contained file to upload by hand) and, from v0.281, **Publish to website**.

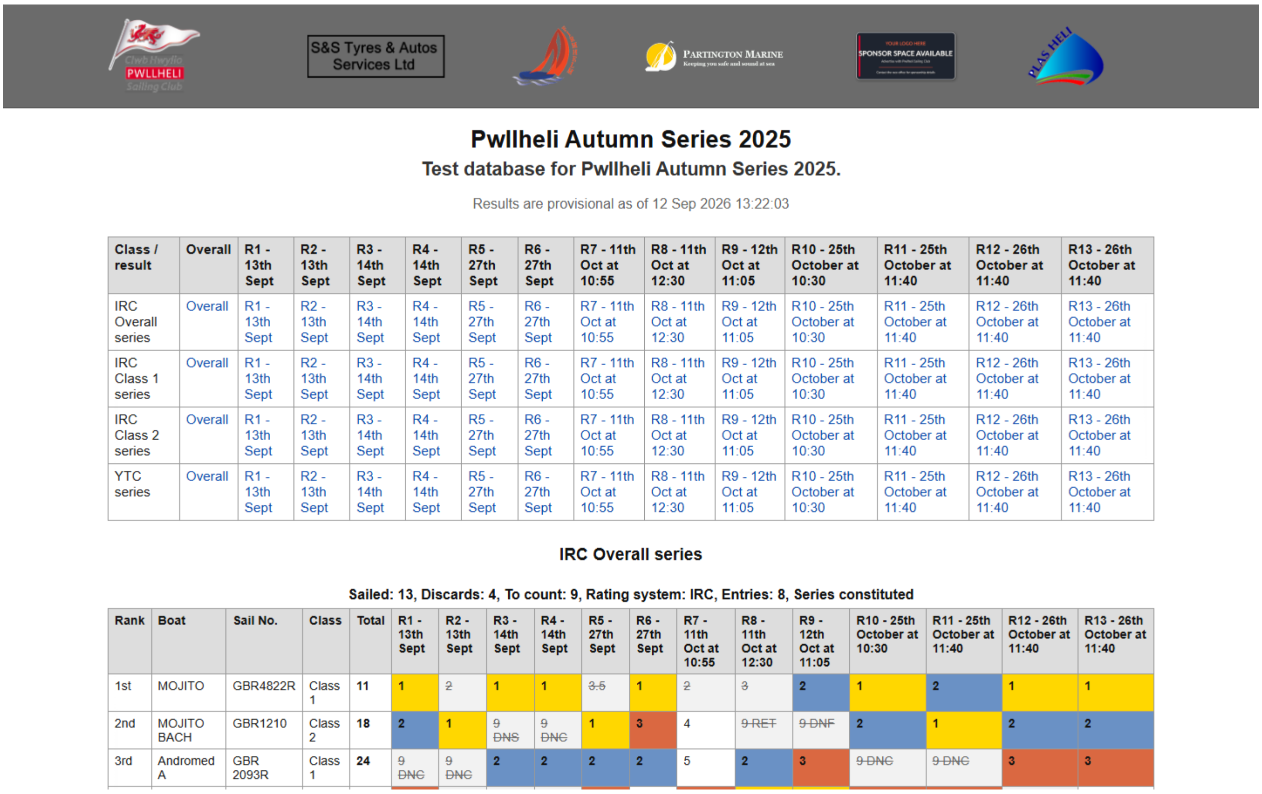
Publish to website uploads the document to the club's public Cloudflare R2 bucket — the same bucket the public race videos use, configured under Settings → Video recording, so there is one set of credentials and one public base URL rather than two. The button is hidden until that bucket is configured, and each series publishes into **results/series-<id>/**.

Every publish writes **two** objects: a dated one, cached for a year and never overwritten, which is the permanent record of that publish; and **latest.html**, overwritten each time and cached for only sixty seconds. The short cache is the point of the pair. The latest link is what goes on the club website, and a long cache there would leave Cloudflare serving Saturday's standings on Wednesday. The dated key carries seconds, so two publishes a minute apart are two records rather than one file with two entries pointing at it.

Published… lists what has gone out, with the club-website link in a field to copy. The app records each publish in its own **published_results** table rather than listing the bucket, so the competitor page can show the latest link without a signed request to Cloudflare on a public page. Nothing is recorded unless the upload succeeded, and a failure reports what went wrong rather than asking you to try again.



The series toolbar: Add race to series, Download CSV, Preview HTML and Download HTML.



Pwllheli Autumn Series 2025

Test database for Pwllheli Autumn Series 2025.

Results are provisional as of 12 Sep 2026 13:22:03

Class / result	Overall	R1 - 13th Sept	R2 - 13th Sept	R3 - 14th Sept	R4 - 14th Sept	R5 - 27th Sept	R6 - 27th Sept	R7 - 11th Oct at 10:55	R8 - 11th Oct at 12:30	R9 - 12th Oct at 11:05	R10 - 25th October at 10:30	R11 - 25th October at 11:40	R12 - 26th October at 11:40	R13 - 26th October at 11:40
IRC Overall series	Overall	R1 - 13th Sept	R2 - 13th Sept	R3 - 14th Sept	R4 - 14th Sept	R5 - 27th Sept	R6 - 27th Sept	R7 - 11th Oct at 10:55	R8 - 11th Oct at 12:30	R9 - 12th Oct at 11:05	R10 - 25th October at 10:30	R11 - 25th October at 11:40	R12 - 26th October at 11:40	R13 - 26th October at 11:40
IRC Class 1 series	Overall	R1 - 13th Sept	R2 - 13th Sept	R3 - 14th Sept	R4 - 14th Sept	R5 - 27th Sept	R6 - 27th Sept	R7 - 11th Oct at 10:55	R8 - 11th Oct at 12:30	R9 - 12th Oct at 11:05	R10 - 25th October at 10:30	R11 - 25th October at 11:40	R12 - 26th October at 11:40	R13 - 26th October at 11:40
IRC Class 2 series	Overall	R1 - 13th Sept	R2 - 13th Sept	R3 - 14th Sept	R4 - 14th Sept	R5 - 27th Sept	R6 - 27th Sept	R7 - 11th Oct at 10:55	R8 - 11th Oct at 12:30	R9 - 12th Oct at 11:05	R10 - 25th October at 10:30	R11 - 25th October at 11:40	R12 - 26th October at 11:40	R13 - 26th October at 11:40
YTC series	Overall	R1 - 13th Sept	R2 - 13th Sept	R3 - 14th Sept	R4 - 14th Sept	R5 - 27th Sept	R6 - 27th Sept	R7 - 11th Oct at 10:55	R8 - 11th Oct at 12:30	R9 - 12th Oct at 11:05	R10 - 25th October at 10:30	R11 - 25th October at 11:40	R12 - 26th October at 11:40	R13 - 26th October at 11:40

IRC Overall series

Sailed: 13, Discards: 4, To count: 9, Rating system: IRC, Entries: 8, Series constituted

Rank	Boat	Sail No.	Class	Total	R1 - 13th Sept	R2 - 13th Sept	R3 - 14th Sept	R4 - 14th Sept	R5 - 27th Sept	R6 - 27th Sept	R7 - 11th Oct at 10:55	R8 - 11th Oct at 12:30	R9 - 12th Oct at 11:05	R10 - 25th October at 10:30	R11 - 25th October at 11:40	R12 - 26th October at 11:40	R13 - 26th October at 11:40
1st	MOJITO	GBR4822R	Class 1	**11**	1	~~2~~	1	1	~~3.5~~	1	~~2~~	~~3~~	2	1	2	1	1
2nd	MOJITO BACH	GBR1210	Class 2	**18**	2	1	~~9 DNS~~	~~9 DNC~~	1	3	4	~~9 RET~~	~~9 DNF~~	2	1	2	2
3rd	Andromed A	GBR 2093R	Class 1	**24**	~~9 DNC~~	~~9 DNC~~	2	2	2	2	5	2	3	~~9 DNC~~	~~9 DNC~~	3	3

The published results document: branding banner, jump-to-race toolbar, and colour-coded top-three places.

> **IMPORTANT**
>
> The published file is a snapshot, provisional as of when it was published. After any correction, press **Publish to website** again — the app does not keep the club website in sync on its own. The link itself never needs changing; only the publishing does.

CSV exports are Sailwave import files

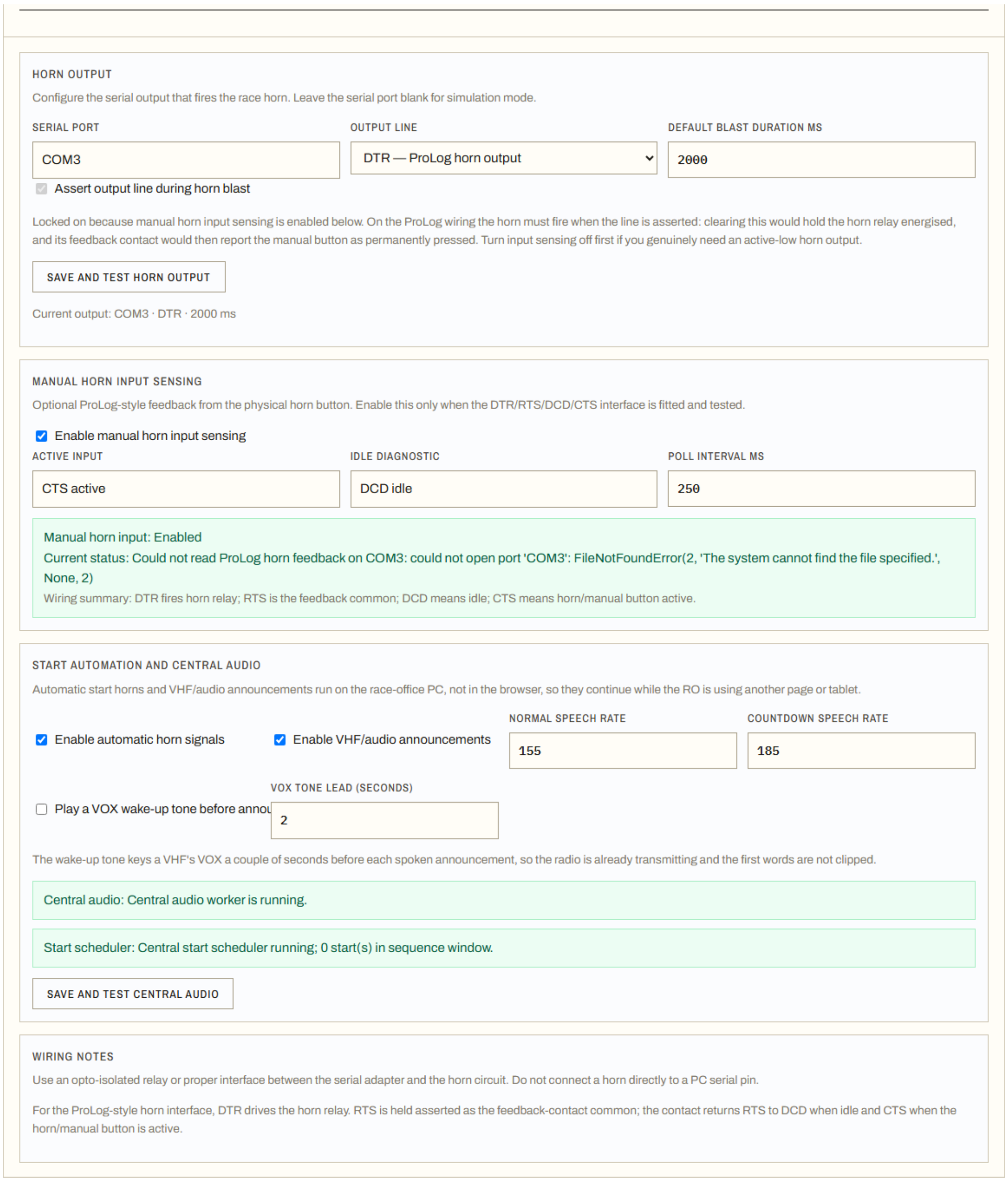
The club scores and keeps its published history in **Sailwave**, so the CSV downloads on the race and series pages exist to be imported rather than read. Each is one header row of field names Sailwave recognises, then **one row per competitor per race**, races told apart by **RaceNo** — the shape its importer reads, needing no column mapping.

- **One file per rating system.** A Sailwave series is scored under one system and a competitor carries one **Rating**, but this app produces IRC and YTC from the same finish times — so each page offers both, formatted the way each system is written (IRC TCC to three decimals, YTC whole). Mixing them in one file would silently rescore a fleet.
- **Finishing places are deliberately not exported.** Sailwave scores from elapsed time and rating; sending an order as well would import this app's arithmetic and then ask Sailwave to redo it, with nothing to say which wins if they disagreed. **Elapsed** is sent alongside **Start** and **Finish**, because a race here can have per-class start times.
- Races are numbered by their **position in the series**, so importing week by week lands each race in its own Sailwave race rather than overwriting race 1 every time. The series export is every race in one file; the standings are not exported, because working out totals and discards is the reason for importing.

The readable version of the same results is still the HTML publish above. The full column reference and import steps are in **docs/WEBSITE_PUBLISHING.md**.

Horn, manual input sensing and central audio

Configured in **Settings** → **Horn settings & Test**, split into three subsections: horn output, manual horn input sensing, and start automation/central audio.



Settings → Horn settings & Test: all three subsections.

Horn output

The app uses **pyserial** to assert **DTR** (or, in legacy/non-ProLog setups, RTS) on a serial adapter for a configured blast duration (50–5000 ms) — this matches the ProLog-style race-office interface. Fields: **Serial port** (e.g. COM3 on Windows, /dev/ttyUSB0 on Linux — leave blank for simulation mode, which logs events without firing hardware), **Output line** (DTR/RTS), **Default blast duration**. **Save and test horn output** fires a real test blast.

IMPORTANT

Never drive a horn directly from a serial adapter, and never wire horn voltage or a horn switch directly to CTS/DSR/DCD/RI. Use a relay, opto-isolated relay board or transistor driver, with proper isolation and voltage limiting. A robust arrangement keeps the existing horn button/circuit working independently of the PC, with the PC's horn output driving an isolated relay contact in parallel.

Manual horn input sensing

Optional feedback that timestamps the moment the physical horn button is pressed, using the relay's feedback contact: **DTR** (pin 4) drives the horn relay; **RTS** (pin 7) is held asserted as the feedback-contact common; the relay returns RTS to **DCD** (pin 1) when idle and to **CTS** (pin 8) when the horn/manual button is active. The app treats CTS-asserted as authoritative for “active”, using DCD only as an internal idle diagnostic. Status is polled via an authenticated JSON API; if unreachable the UI shows “Input status unavailable” rather than a raw error. Only manual horn events — not automatic sequence horns — can be assigned as a boat's finish time from the race log.

NOTE

Enabling manual horn input sensing forces the horn output line to DTR regardless of the Output-line selector, since RTS is needed as the sensing common in that wiring.

Central audio (VHF announcements)

Speech is generated on the race-office PC itself (via **pyttsx3**/SAPI on Windows, with OS command-line fallbacks elsewhere), not in the browser — so VHF announcements keep running while the race officer uses another page, a tablet, or the split-screen view. Route the PC's audio output through a mixer, isolation transformer or approved radio audio interface into the VHF/PA input; keep levels conservative and test before racing, and keep the official horn circuit independent of the audio feed. Settings fields: **Enable automatic horn signals**, **Enable VHF/audio announcements**, **Normal speech rate** (default 185) and **Countdown speech rate** (default 285, used for the final ten-second count and “Start.”). **Save and test central audio** speaks a test phrase.

If the VHF is keyed by **VOX** (voice-activated transmit), the radio can take a moment to open, clipping the first word. Enable **Play a VOX wake-up tone before announcements** and set the **VOX tone lead (seconds)** (default 2): a short tone is played that many seconds before each spoken announcement — standard and pursuit — so the radio is already transmitting when speech begins. The tone is scheduled ahead of the announcement, so it does not shift the countdown or signal timing.

How the signals line up

Per configured start, relative to that start's own start signal:

TIME	SIGNAL	FLAGS AFTER
–5:00	Warning signal (1 sound)	Class numeral pennant(s) raised
–4:00	Preparatory signal (1 sound)	Pennant(s) + P raised
–1:00	One-minute signal (1 sound)	P lowered; pennant(s) remain up
0:00	Start signal (1 sound)	Pennant(s) lowered

Spoken announcements are layered around these: a course announcement at –9:00/–7:00/–3:00, a “stand by 15 seconds” call 15 seconds before each horn, spoken “Five minutes / Four minutes / One minute / Thirty seconds / Twenty seconds”, then a spoken countdown from ten to one, and “Start.” at zero. If two starts' signals coincide (e.g. Start 2's warning falls on Start 1's start), the scheduler coalesces them into a single horn sound rather than sounding twice.

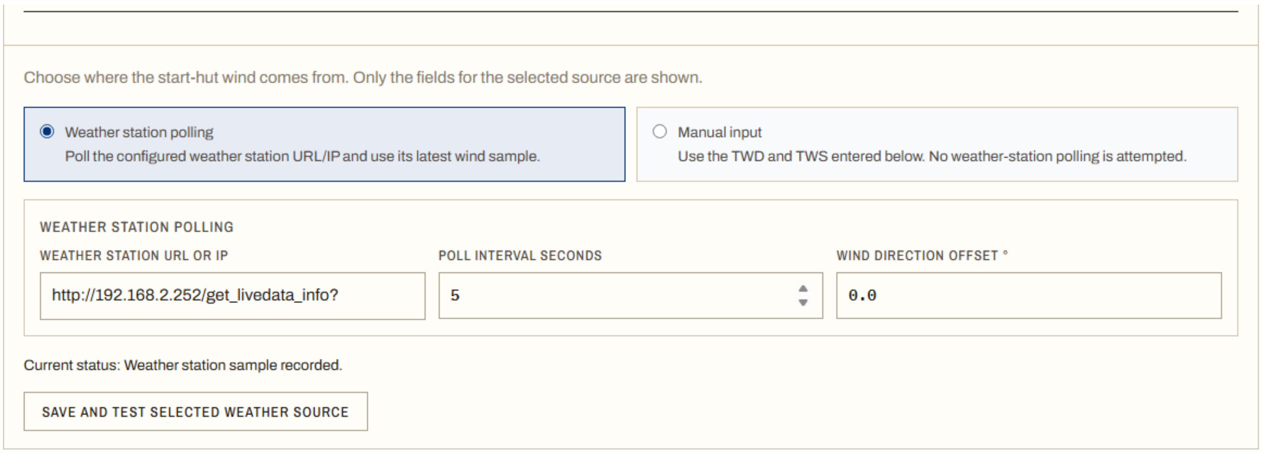
> **NOTE**
>
> There is no built-in protest or redress workflow, and the current flag panel does not yet model every AP/X/First Substitute/N/S combination. The horn remains the authoritative signal — browser pages only display the countdown and flags.

PART IV — HARDWARE INTEGRATION

Weather station

Configured in **Settings → Weather Station**. Two modes, chosen by radio button:

- **Weather station polling** — poll a configured URL/IP and use its latest wind sample.
- **Manual input** — use a TWD/TWS typed in below; no polling is attempted.



Settings → Weather Station: pick the source, then only that source's fields are shown.

Fields and behaviour

FIELD / CONTROL	WHAT IT DOES
Weather station URL or IP	Default format is http://<ip>/get_livedata_info? (a bare IP has this path appended automatically). Response is expected as JSON.
Poll interval seconds	1–120 seconds; the background poller runs continuously regardless of which page is open, with a 4-second request timeout.
Wind direction offset °	Corrects for a weather station not aligned to true north; applied before the reading is stored.
Manual TWD ° / TWS kt	Used only when Manual input is selected.

The app reads three fields from the station's **common_list** JSON array by id: **0x0A** (wind direction), **0x0B** (wind speed) and **0x0C** (gust), converting units (kt, mph, m/s, km/h all recognised; unitless values are assumed m/s). On any failure the status line reads “Could not read weather station: …” rather than silently going stale.

> **NOTE**
>
> The configured URL is validated before use: it must resolve to a routable address — **localhost, link-local, multicast and unspecified addresses are rejected** by default (normal club-LAN private addresses are fine). **RO_WEATHER_ALLOWED_HOSTS** (Chapter 2) can further restrict accepted hostnames. If validation fails, the app falls back to the built-in default URL rather than erroring.

Every successful poll is stored as a wind-history sample, feeding the wind-history chart, course recommendation, race leg analysis and the public course-analysis view.

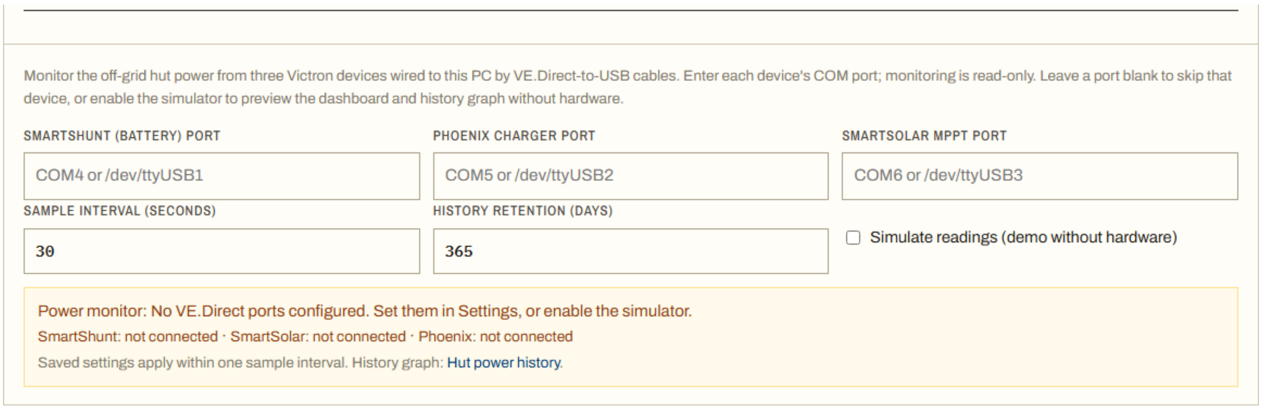
Ambient samples are pruned after 24 hours, but a sample falling within an hour either side of a race is kept indefinitely. The wind a race was sailed in is part of that race's record — the replay's wind gauge reads it back years later, and the Chart and Course analysis tabs show the average over the race once everyone has finished. Trimming it to the last day would have left every race older than yesterday with a gauge and no wind to put in it.

PART IV — HARDWARE INTEGRATION

Hut power monitoring (Victron VE.Direct)

For an off-grid hut, the app can monitor the 12 V power system from three Victron devices over their **VE.Direct** serial ports: a **SmartShunt** battery monitor, a **Phoenix** IP43 charger, and a **SmartSolar** MPPT solar controller. Each device connects to the race-office PC with its own VE.Direct-to-USB cable, so each appears as a separate COM port. Monitoring is **read-only** — the app reads the telemetry the devices broadcast and never sends commands to them.

Configuration (Settings → Hut power)



Settings → Hut power: the three device COM ports, sample interval, history retention and the simulator toggle.

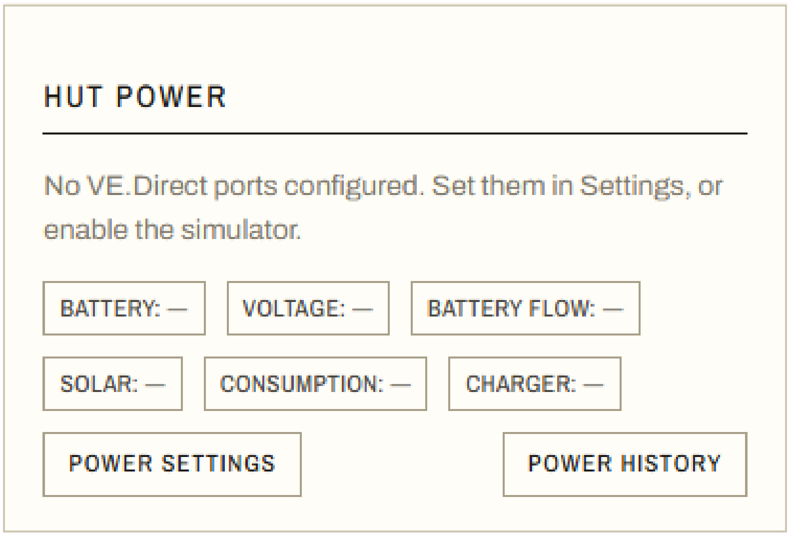
FIELD / CONTROL	WHAT IT DOES
SmartShunt / Phoenix / SmartSolar port	The COM port for each device (e.g. COM4). Find them in Windows Device Manager or the VictronConnect app. Leave a port blank to skip that device.
Sample interval (seconds)	How often a combined reading is stored, 5–3600 s (default 30).
History retention (days)	Samples older than this are purged, 1–3650 (default 365). History lives in a separate database, so this does not affect the race data or its backups.
Simulate readings	Generates plausible battery/solar values so the dashboard card and history graph can be demoed before the cables are wired. Any real configured port overrides the simulator for that device.

NOTE

History is stored in **data/power_history.db**, separate from the race database. It is **not** included in the Backup / restore sections and is excluded from release ZIPs, so continuous telemetry never bloats the race data.

Dashboard card

The **Hut power** card on the Dashboard shows the battery state of charge, voltage and current, solar input, charger state, and the estimated hut consumption. It refreshes every few seconds.



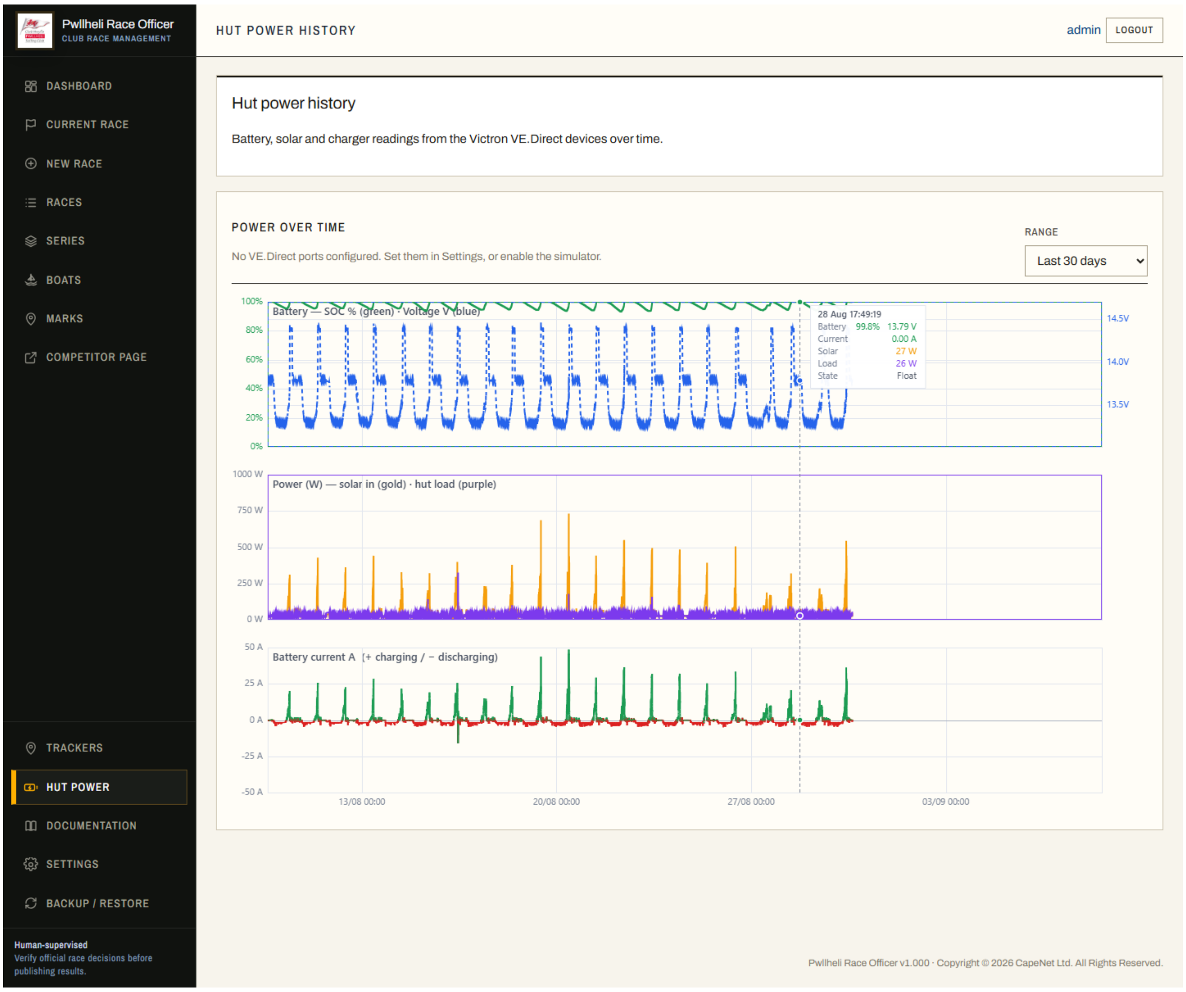
The Dashboard Hut power card (shown here with the simulator on).

Hut consumption (derived load)

The SmartShunt measures only the battery's net current, so the hut's actual DC load is worked out from all three devices: **load = solar current + charger current** – **battery current**, times the bus voltage. A source that is off or not connected counts as zero, and the figure is clamped at zero against measurement noise. The consumption reading is therefore only fully accurate when all three devices are reporting.

History graph

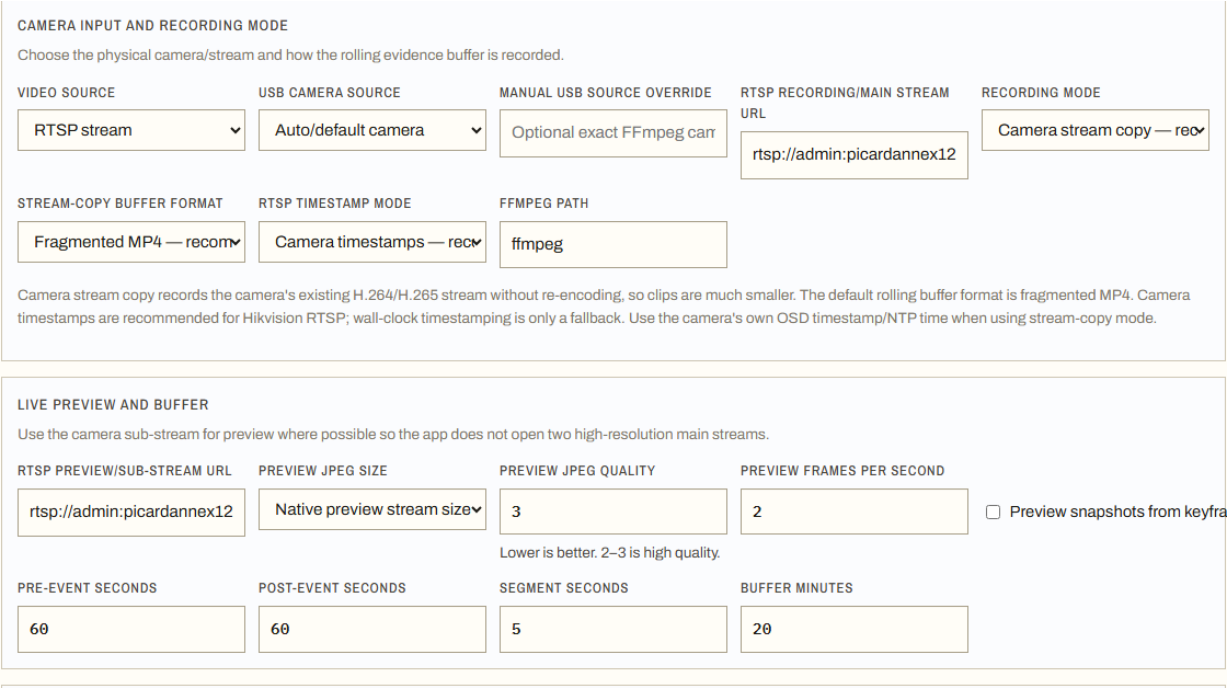
The **Hut power** item in the sidebar (grouped with Documentation and Backup) opens a history page with a selectable time range. It plots battery state of charge and voltage, solar input with hut consumption overlaid for comparison, and battery charge/discharge current.



The Hut power history page: battery, power (solar in vs hut load) and battery current over the selected range.

RTSP camera and video recording

Configured in **Settings** → **Video Recording**. This chapter covers the **RTSP IP camera** path (select **Video source** → **RTSP stream**) — the app also supports a USB webcam, not covered here.



Camera input and recording mode, and the rolling live-preview buffer.

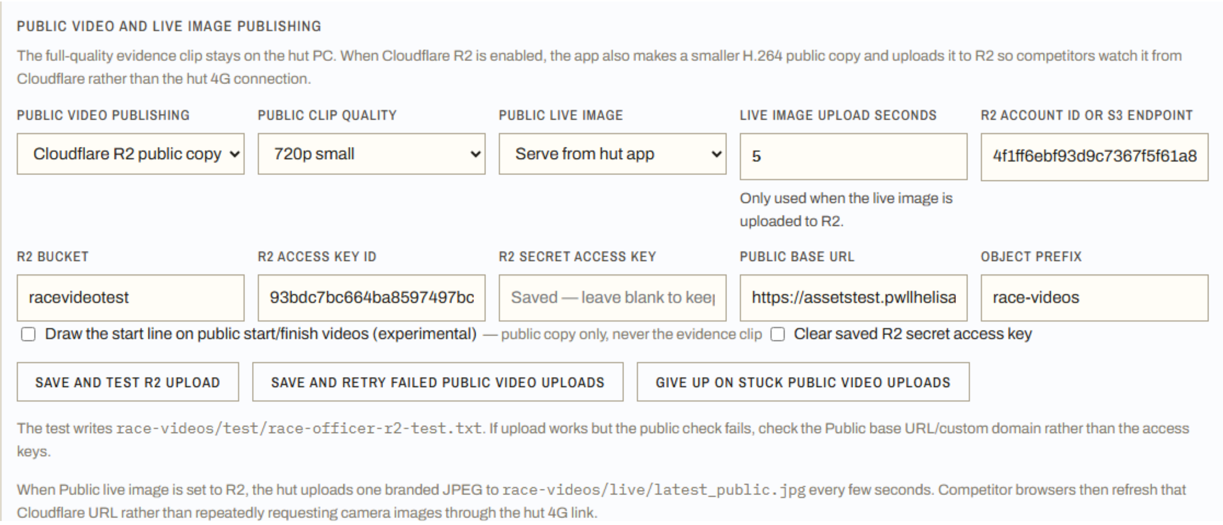
RTSP setup

FIELD / CONTROL	WHAT IT DOES
RTSP recording/main stream URL	e.g. rtsp://user:pass@192.168.1.157:554/Streaming/Channels/101 (channel 101 = main stream on a Hikvision-style camera). Credentials are embedded directly in the URL.
Recording mode	Camera stream copy (recommended) copies the camera's existing H.264/H.265 stream without re-encoding — much smaller CPU load and file overhead; the alternative re-encodes with an app-drawn timestamp overlay.
Stream-copy buffer format	Fragmented MP4 (recommended) or MPEG-TS as an advanced fallback.
RTSP timestamp mode	Camera timestamps (recommended for Hikvision RTSP) vs wall-clock arrival time (fallback only — can show pause/jump artefacts if the camera's own timestamps are unusable).
FFmpeg path	Defaults to relying on ffmpeg being on PATH; set an explicit path if it isn't.
RTSP preview/sub-stream URL	Optional second, lower-resolution stream (e.g. channel 102) so the live preview doesn't compete with the main recording stream for bandwidth/CPU.

FFmpeg keeps a rolling buffer in **runtime/video/buffer/**; clips are cut from it for start signals, the Finish button, and manual horn events, using **Pre-event/Post-event seconds**, and are saved to **data/video_clips/**

(back this up — Chapter 19). **Buffer minutes** and **Segment seconds** control the rolling buffer's size and granularity.

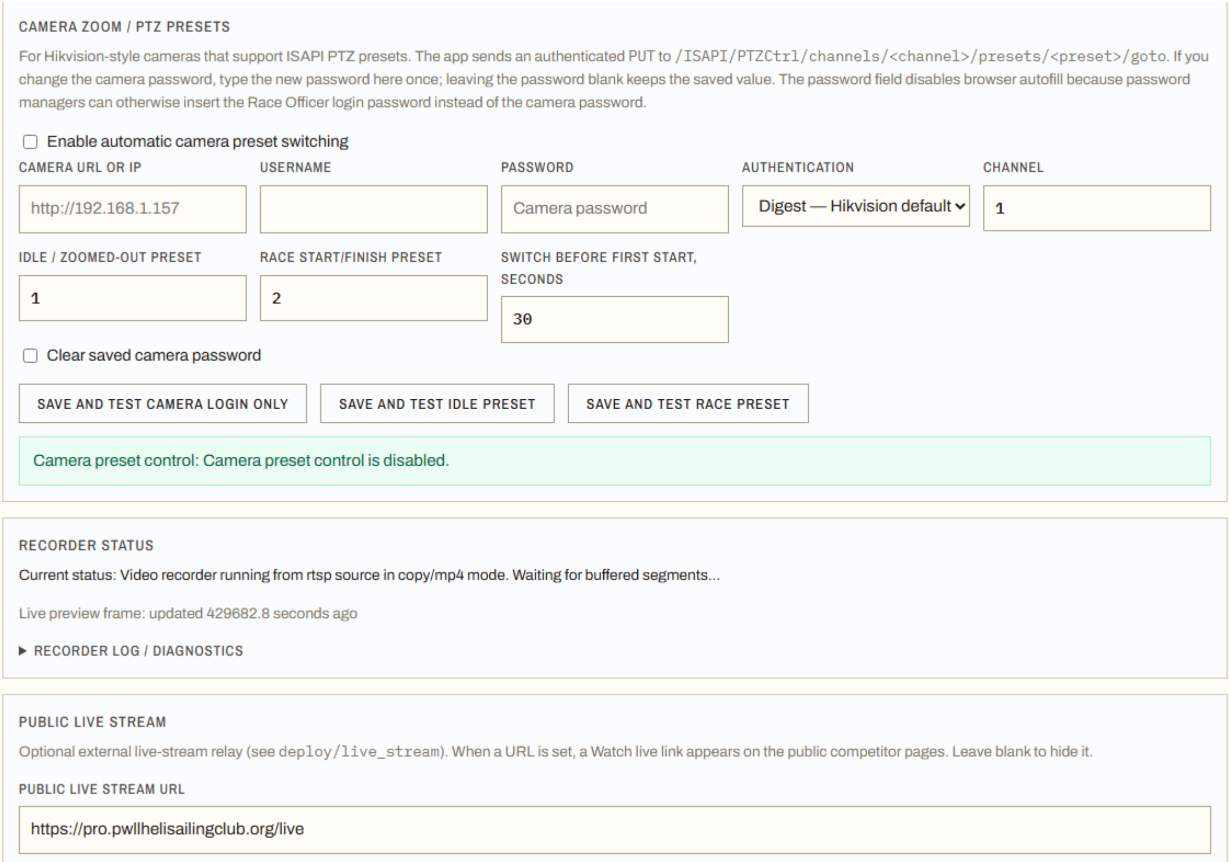
> **NOTE**
>
> For Hikvision-style stream-copy: use standard H.264/H.265 (not Smart Codec/H.264+/H.265+), constant bitrate, and an I-frame interval around one second. Low-bitrate or smart-codec streams can look like periodic freezes, since the app faithfully copies whatever sparse frames the camera actually sends.



Public video and live-image publishing (Cloudflare R2) — see Chapter 17.

Camera PTZ presets

For Hikvision-style cameras with ISAPI PTZ presets, the app can automatically switch between an **idle/zoomed-out** preset and a **race start/finish** preset — switching to the race preset a configurable number of seconds before the first actual start, and back to idle once no entry is racing. Requests are authenticated PUT calls to **/ISAPI/PTZCtrl/channels/<channel>/presets/<preset>/goto**.



Camera zoom/PTZ presets, recorder status, and the public live-stream link.

NOTE

A successful manual preset test (**Save and test idle preset** / **…race preset**) holds that preset for 30 seconds before the automatic scheduler can switch it back — giving you time to actually see whether the camera moved. If a camera login fails, automatic switching pauses itself to avoid repeatedly locking the camera's admin account.

Cloudflare Tunnel, public video and live stream

Cloudflare Tunnel lets competitors reach the public pages (and the race officer reach /admin) from outside the club network, without opening inbound ports on the race-hut router.

Setting up the tunnel

1. Run the app on the race-office PC (ideally as the Windows Scheduled Task from Chapter 2) and confirm **http://localhost:5050/admin** works locally first.
2. In the Cloudflare dashboard, configure a **public hostname** that forwards to **http://localhost:5050** with the path left **blank** — forward the whole app, not just /public.
3. Give competitors the bare hostname, e.g. **https://pro.yourclub.org**.
4. Race officers use the same hostname's **/admin** route and log in as normal.

> **IMPORTANT**
>
> Do not tunnel only **/public**. The public pages also load CSS, JavaScript and images from **/static/...** — tunnelling only /public makes the page render as unstyled plain text. This warning appears in three places in the app's own documentation and code comments, because it's the single most common setup mistake.

Security model

The Tunnel exposes the **whole** app, including **/admin** — security relies entirely on Flask login, not on restricting which routes the tunnel forwards. Set a strong, explicit **RO_SECRET_KEY** (Chapter 2) for any internet-facing deployment. Public routes are a deliberately narrow allow-list (static assets, the public competitor pages, public video/live-frame routes, and read-only course/weather APIs) — horn control, finish-time entry, settings, boat management, rating import and user administration are never reachable without logging in.

> **NOTE**
>
> There is no rotatable “public share link”/token feature in the current version — the public URLs (**/public/current**, **/public/race/<id>**) are plain, keyless and always readable by anyone who reaches them. Don't put anything sensitive in a public-facing race name or note field.

One responsive page serves phones, tablets and PCs. On a phone the wide tables become one card per boat. The **competitor home page** carries the analog wind dial (the same instrument and script as the dashboard, from **templates/partials/wind_gauge.html**) and a countdown to the current race in its header, then three tabs — **Races** (the whole year grouped into rolled-up series, current series open and current race highlighted), **Wind** and **Live camera** (which starts on opening the tab, with no checkbox). A **race page** is three tabs — **Entries** (or **Start times** for a pursuit), **Chart** and **Course analysis** — gaining a fourth, **Leader board**, once every boat has stopped racing; the leader board then opens first and the Chart/Course analysis tabs switch to the **average wind over the race** (warning signal to last finish, TWD vector-averaged) as a record of the conditions. The older **/public/mobile/...** addresses still work and redirect to the responsive page, so links already shared with competitors keep working.

Cloudflare R2 (public video and live image)

A separate, optional feature — configured alongside video recording (Chapter 16), not part of the Tunnel setup — that offloads competitor video/image traffic from the hut's own connection onto Cloudflare R2 storage instead.

FIELD / CONTROL	WHAT IT DOES
Public video publishing	Off (serve the local evidence clip) or Cloudflare R2 public copy. The full-quality clip always stays local in data/video_clips/; R2 only ever gets a smaller, branded H.264 copy.
Public clip quality	720p small or 1080p normal — for the R2 public copy only.
Draw the start line on public start/finish videos	Experimental, off by default. Detects the orange ODM buoy and draws the start line (pole base → buoy) on the public copy only — never the evidence clip. Start videos: red before the start, green after; finish videos: green. Best-effort (skipped if the buoy isn't found); tune in data/startline_config.json. Needs Pillow + numpy.
Public live image	Serve from hut app, or upload the latest branded JPEG to R2 every few seconds (Live image upload seconds).
R2 account ID / S3 endpoint, bucket, access key ID, secret access key	Standard S3-compatible credentials for the R2 bucket.
Public base URL	The public/custom domain competitor browsers actually load from.
Object prefix	Folder-style prefix inside the bucket, e.g. race-videos.

Save and test R2 upload writes a small test file and checks it's publicly reachable; if the upload succeeds but the public check fails, check the **Public base URL** rather than the access keys. **Save and retry failed public video uploads** re-attempts any clips that didn't make it to R2 the first time.

Public live camera stream

Separately from the still-image live preview and finish clips, the club can publish a continuous, branded, low-bandwidth **live view** of the hut camera to competitors and spectators. The live stream is produced by an external relay (below); the app's only part is a single setting that surfaces the public **Watch live** link.

FIELD / CONTROL	WHAT IT DOES
Public live stream URL	Settings → Video Recording → Public live stream. The public address of the live view, e.g. https://pro.pwllhelisailingclub.org/live (only http/https accepted). When set, a **Watch live** link appears on the public competitor pages — a button in the home page's live-camera panel and a link in the race-page footer. Leave blank to hide it everywhere.

NOTE

Setting the URL does not itself start any streaming — it only shows the link. The stream must actually be served by the relay for the link to work. The full-quality finish video and the still-image live preview are unaffected and independent of this.

From v0.182 the URL does more than show a link: the competitor home page's **Live camera** tab and the race sheet's **live finish camera** both show their refreshing snapshots while the relay's on-demand stream starts, then embed that stream once it reports it is playing. The relay's watch page posts its state to the embedding page (**{source: 'psc-live', state: …}**); an older relay build that posts nothing still works, the panels just switch on a timer instead. Snapshot refreshing stops when the video takes over, and closing the tab or panel removes the player so the relay can drop the stream.

> **IMPORTANT**
>
> The relay stream lags the water by a few seconds and uses the hut connection in both directions. Time finishes from the horn and the recorded clips, and keep the race sheet's camera panel closed on a metered connection when it is not being used.

The camera on the club's own website

The club's main site can carry the same view. **deploy/embed/club_website_camera.html** is a small self-contained page to upload to that site and drop into an iframe; its README covers the two lines of HTML and the one Caddy rule the relay needs. It plays the relay's live stream, so the first visitor starts it and Cloudflare fans the same copy out to everybody after them — the hut's uplink serves one stream however many people are watching — with a still from the hut filling the gap while it starts.

> **NOTE**
>
> The page checks where it is embedded. On a site the app does not recognise it takes a single still and stops, so a copy left on some other page cannot sit there pulling pictures off the hut all day.

The live-stream relay (deploy/live_stream)

The stream itself runs on a separate always-on machine — **not** the hut PC — so the off-grid hut sends only one outbound stream, and only while someone is watching. The complete setup, configuration and troubleshooting guide ships with the release in **deploy/live_stream/README.md**; the essentials are:

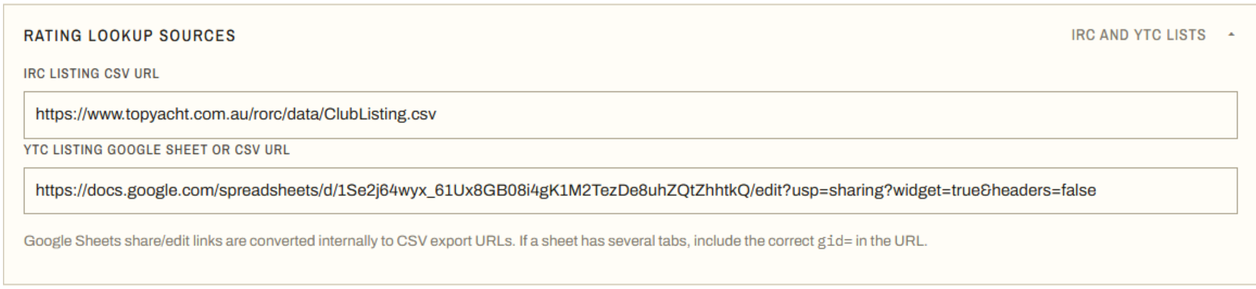
- **On-demand and shared:** the relay pulls a single stream from the hut camera only while at least one viewer is watching, and drops it about 20 seconds after the last viewer leaves. Viewer count does not change the load on the hut.
- **Cloudflare fan-out:** Cloudflare caches the video segments, so the relay and the home uplink stay flat no matter how many people watch.
- **Branding is automatic:** the relay burns the club logo (top-left) and rotating sponsor logos (top-right) into the video, reading them live from the app's public branding manifest **/api/branding/live** — change the logos in Settings → Public branding and the stream follows, with nothing to edit on the relay.
- **It is also the club's front door:** the same relay proxies the club's public hostname to the hut race app and shows a holding page when the hut is offline. The trade-off is that if the relay itself is down, the public hostname is unreachable.

> **NOTE**
>
> The relay (MediaMTX + Caddy + cloudflared, typically on a Proxmox LXC) is infrastructure a club IT volunteer sets up once. Day-to-day race operation needs none of it — only the **Public live stream URL** setting above. Follow deploy/live_stream/README.md for the build, and keep any tokens/secrets out of version control (the bundled relay env files are placeholders only).

Rating sources, polars and branding

Rating lookup sources

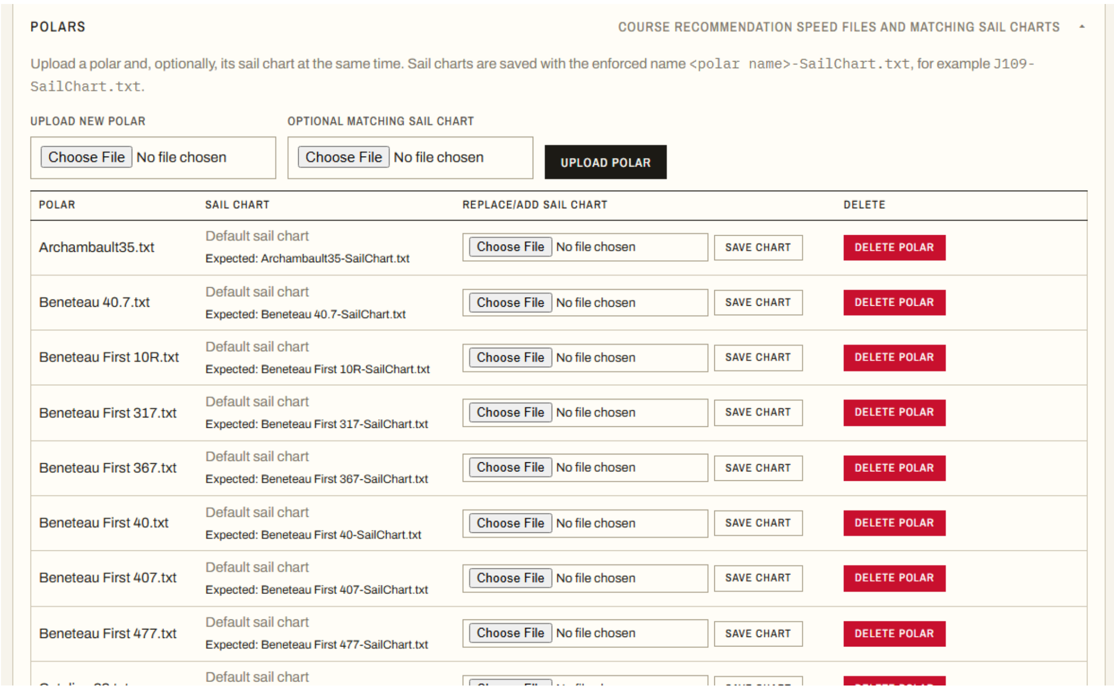
Two URLs feed both the Boats add/lookup search and the bulk import pages (Chapter 4): an IRC listing CSV URL and a YTC listing Google Sheet/CSV URL (share links are converted to CSV-export links automatically; include the correct **gid=** if the sheet has multiple tabs). Both lists are cached for an hour to avoid slow repeated downloads. Only http:// and https:// URLs are accepted.



Settings → Rating lookup sources.

Polars and sail charts

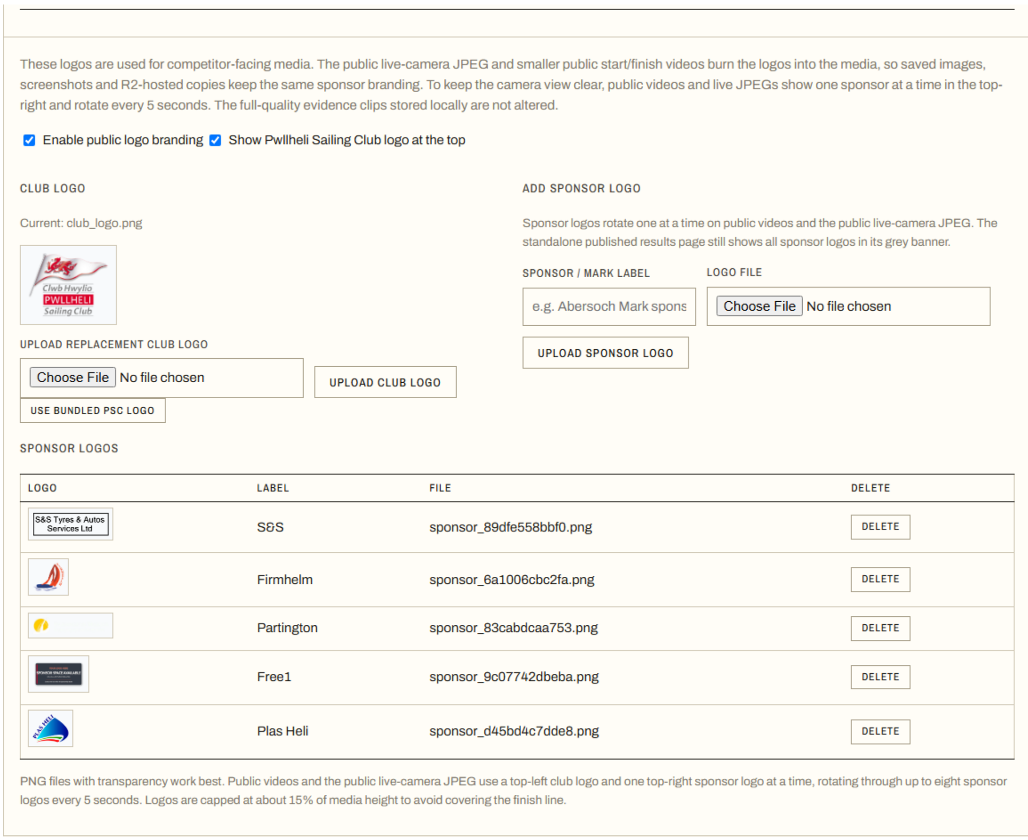
A **polar** is a boat-speed-vs-wind file used by course recommendation, race leg analysis, the manual course builder and public course analysis. Upload a polar (.txt/.pol/.csv) with an optional matching **sail chart** (a TWA/TWS-to-sail-choice table) in one step; the sail-chart filename is enforced from the polar's name, e.g. polar **J109.txt** expects sail chart **J109-SailChart.txt**. Without a matching chart, analysis falls back to the bundled default sail chart.



Settings → Polars: upload form and the installed-polars table (the club's full library runs to dozens of boat classes).

Public branding

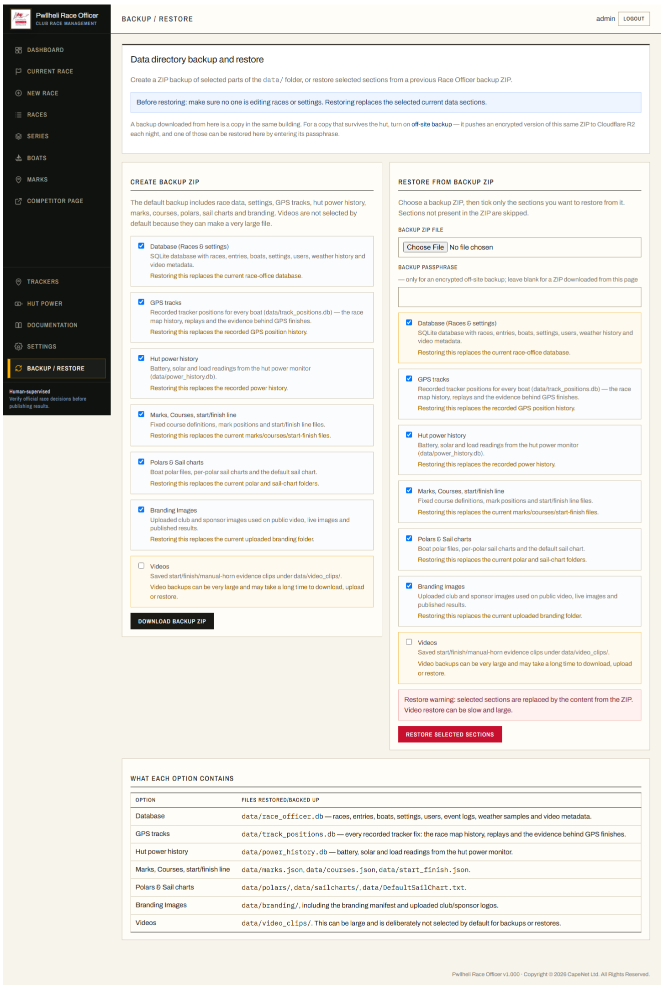
Club and sponsor logos shown on competitor-facing media: the public live-camera JPEG and the smaller public R2 video copy burn the logos in directly (club logo fixed top-left, one of up to **eight** sponsor logos top-right, rotating every 5 seconds, capped at about 15% of media height so the finish line stays visible); the full-quality local evidence clip is never branded. The standalone published results page instead shows the full club-plus-all-sponsors strip in its banner (Chapter 12), since it's a static page with nothing to rotate.



Settings → Public branding: club logo and sponsor logos.

Backup and restore

The Backup / restore page (bottom of the side menu) creates or restores a ZIP covering selectable parts of the **data/** folder.



Backup / restore: the selectable sections, and what each contains.

The app keeps race data in **three** SQLite databases, and all three have their own section: **Database** (races, entries, boats, settings, users), **GPS tracks** (**data/track_positions.db** — every recorded tracker fix, and so the evidence behind GPS finishes) and **Hut power history**. Those, plus **Marks/Courses/start-finish**, **Polars & Sail charts** and **Branding Images**, are selected by default; **Videos** is not, because clips can make backups very large.

> **IMPORTANT**
>
> Before v0.192 only the race database was backed up, so a PC rebuilt from a backup came back with every boat's recorded track gone. If you hold older backup ZIPs, they do not contain the tracks.

Every database is copied through SQLite's **online backup API** rather than as a file, so a backup taken while the tracker poller and power monitor are writing cannot catch a half-written transaction — nothing has to be stopped first. A database that does not exist yet (a hut that has never used tracking) is reported as nothing to back up rather than an error. Every backup includes a manifest recording app version, creation time and file counts. Restoring validates every ZIP entry against path-traversal (must live under **data/**, no **..** segments, no absolute/drive paths) before writing anything to disk, and puts each database back where its module looks for it, reopening it at the current schema.

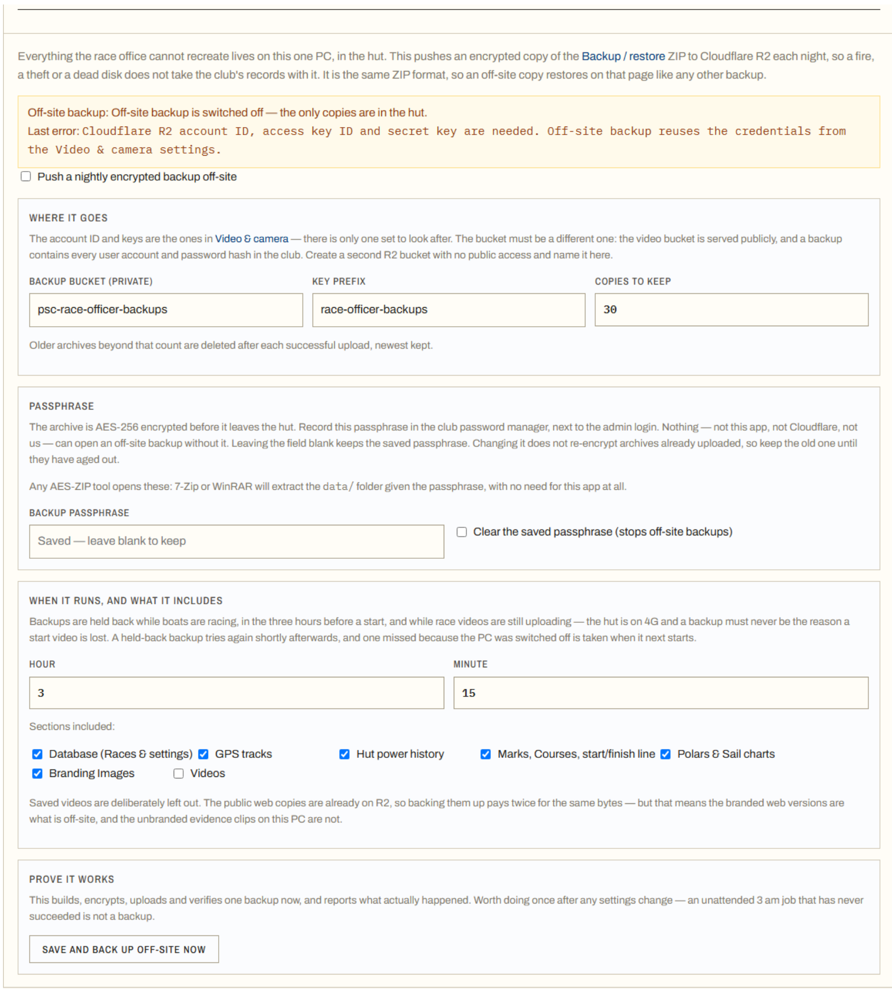
> **IMPORTANT**
>
> Restoring replaces the selected sections outright with the ZIP's content. Make sure no one is editing races or settings while a restore runs, and keep the pre-restore backup until you've confirmed races, boats, users and settings all loaded correctly.

> **NOTE**
>
> Restoring is **admin only**, because it overwrites current data — race officers see the Restore panel replaced by an “administrator access required” note. Creating and downloading a backup is available to race officers too (see the user roles in Chapter 3).

Off-site backup

A backup made on the page above is a copy in the same building as the original. From **v0.238** the app can also push an **encrypted copy off-site**, every night, to Cloudflare R2 — so a fire, a theft or a dead disk in the hut does not take the club's records with it. Settings → **Off-site backup**.

It is the *same ZIP*, with the same **data/...** member names, so an off-site copy restores on the Backup / restore page like any other backup. There is no second format to keep working.



Settings → Off-site backup: the bucket, the passphrase, the schedule and the sections that go off-site.

Setting it up

FIELD / CONTROL	WHAT IT DOES
Backup bucket	A **second, private** R2 bucket, created in the Cloudflare dashboard with no public access. It must not be the bucket that serves public race videos.
Key prefix	Folder the archives are written under. Default **race-officer-backups**.
Copies to keep	Older archives beyond this count are deleted after each success, newest kept. Default 30.
Backup passphrase	Encrypts the archive. Record it in the club password manager, beside the admin login.
Hour / minute	When it runs. Default 03:15.
Sections	What goes in. Everything except the saved videos, by default.

The account ID and access keys are the ones already entered for public video publishing (Chapter 16) — there is only one set of credentials to look after. The **bucket**, however, must be a different one, and the app **refuses to run** if it is not.

> **IMPORTANT**
>
> The public video bucket is served publicly. A backup placed in it would be one guessed object key away from being anybody's download — and a backup contains every user account and password hash in the club. That is why a shared bucket is refused rather than merely warned about.

After any change, press **Save and back up off-site now** and read the result. It builds, encrypts, uploads and verifies one backup immediately and reports what actually happened, including a wrong bucket name or a key the bucket will not accept. An unattended 3 am job that has never once succeeded is not a backup.

> **IMPORTANT**
>
> If an R2 **API token is scoped to specific buckets**, a token created for the video bucket cannot write to the backup bucket. Either widen the token's scope to both buckets, or issue a token that covers the backup bucket, in the Cloudflare dashboard under R2 → Manage API tokens.

Encryption, and getting the data back

The archive is **AES-256 encrypted before it leaves the hut**, in the WinZip AES format. That choice is deliberate: **7-Zip or WinRAR will extract the data/ folder given the passphrase**, with this app not installed and no script to run. A format only this app understood would have made the off-site copy depend on the very thing it exists to survive.

To restore one: download the **.zip** from R2, then use the Backup / restore page as normal and enter the passphrase in the **Backup passphrase** field. A wrong passphrase is refused *before* anything is deleted — the restore clears the target folders before extracting, so that check is what stops a typo costing you the branding folder. If the archive is larger than the upload limit (**RO_MAX_UPLOAD_MB**, 64 MB by default), raise it for that restore, or extract the ZIP with 7-Zip and copy the **data/** folder in by hand with the app stopped.

> **IMPORTANT**
>
> Nothing can open an off-site backup without the passphrase — not the app, not Cloudflare, not the developer. Changing it does not re-encrypt archives already uploaded, so keep the old passphrase until those have aged out of retention.

When it runs, and when it stands aside

Nightly at the configured time, and **held back** while boats are still racing, while a start is within the next three hours, and while race videos are still uploading. The hut is on 4G shared with a caravan park, and start videos have already been lost to Saturday-evening contention — a backup must never be the reason one is lost. A held-back run tries again about twenty minutes later; a failed one waits about forty-five. A backup missed because the PC was switched off at 03:15 — the normal state of a hut PC — is taken when the app next starts, rather than skipped for the day.

Checking it is still happening

The **dashboard** carries the age of the last success, and says so plainly when the answer is “never” or “40 days”. This is the point of the card: a backup job that quietly stopped months ago is worse than no backup job at all, because the club believes it has one. Settings also lists **what is actually in the bucket**, rather than what the app believes it put there.

Each upload is **verified with a HEAD request** against the expected byte count, because an upload that reports success without the bytes arriving is the failure nobody notices until a restore. An unencrypted sidecar manifest (**....manifest.json**) is stored beside each archive holding the sections, file counts, size and SHA-256 — so the club can see what is off-site, and check a downloaded archive, without decrypting anything first. It contains counts only, never race data.

> **NOTE**
>
> **Videos are deliberately left out.** The public web copies are already on R2, so backing them up would pay twice for the same bytes. The consequence is worth stating: those R2 copies are branded, re-encoded web versions, so the **unbranded evidence clips on the hut PC are not off-site**. If a particular clip matters as evidence, copy it off by hand.

Encrypted backups need the **pyzipper** package from **requirements.txt**. Without it the app refuses to run an off-site backup rather than uploading a readable archive.

Reading the bundled documentation in the app

The **Documentation** page lists the PDF guides and, beside them, every one of the bundled **docs/*.md** files — installation, tracking, video, the relay, troubleshooting and the rest — which open rendered in the app with their cross-links working. Before this they could only be read as files on the race-office PC, which is no use when **TROUBLESHOOTING** is the one you want *while* something is going wrong.

Web server sizing

Settings → **Web server** sets how much the app takes on at once: **worker threads** (4–64, default 8), the **connection limit** (50–512, default 100) and the **idle timeout** (20–600 s, default 120). Waitress reads them once at startup, so they take effect on a **restart** — the card shows what the running process actually started with and says plainly when a restart is owed, because "I changed it and nothing happened" gets changed twice more during a race.

> **NOTE**
>
> Raise the **connection limit** first when phones on the water get "cannot connect" while the race office is fine. It counts open sockets, not people, and a browser holds up to six per origin — so the old default of 100 is about sixteen viewers, reached whether or not anything is slow. That is what filled it during a race: every request fast, and no sockets left. Threads are cheap here because they are mostly waiting on disk and network rather than working.

The same section carries the **public address**, which is not about sizing: it is the address competitors type in, and it is what the app uses for any address it builds for the outside world. Two things consume those addresses today: the logo addresses the live-stream relay fetches from the branding manifest, and the same manifest read by a **3D replay render machine** when it brands a film. Reached from outside, the app is behind the relay and sees only the tunnel's own internal hostname, so without this those addresses name a machine nobody can reach. Leave it empty on the hut network, where the address a request arrived on is already the right one. Scheme and host only, and unlike the three above it takes effect immediately rather than on a restart. Neither failure is loud — the stream simply carries no logos and a film comes out unbranded — which is why it is worth setting when the club is first published rather than diagnosing later.

Slow-request log

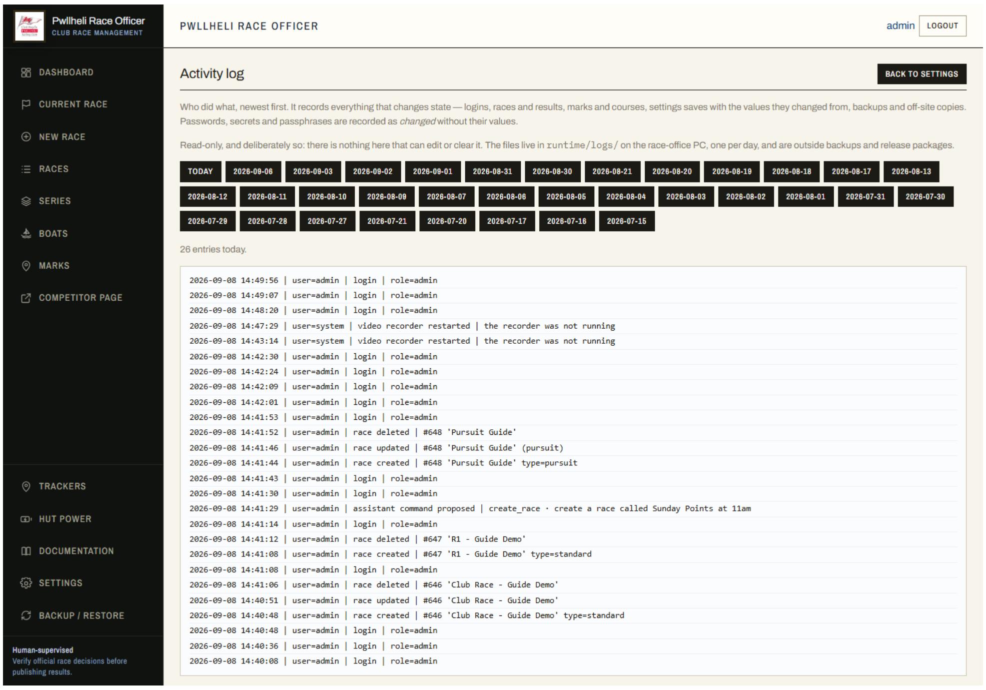
Any request over **RO_SLOW_REQUEST_MS** (default 250 ms) is written to **runtime/logs/slow.log** with its method, path, status and duration — one file per day, kept a fortnight, and set the threshold to 0 to switch it off. It exists because when the server started refusing connections there were four plausible causes in the code and nothing recorded how long a request took. Guessing there means fixing three things that were fine and leaving the one that was not; if it happens again, this file names the endpoint.

Activity log

The app keeps a plain-text history of **everything that changes state**: logins, failed logins, logouts and password changes; races created, updated and deleted; boats added to and removed from races; finishes and pursuit positions recorded; an entry edited by hand, with the old and new finish time and status; course changes, shortenings and mark edits; rating imports; series edits; backups downloaded and restored; and user-account changes. It is written to **runtime/logs/activity.log** on the race-office PC, one file per day (yesterday rolls over to **activity.log.YYYY-MM-DD**). It lives under **runtime/**, so it is not part of backups or release packages.

A **settings save lists the keys that changed, and what they changed from — horn_active: 1 -> 0** is the kind of line that explains a horn fault a fortnight later. Only the differences are listed, and a save that altered nothing is still recorded so "who was in Settings" stays answerable. Passwords, secrets, tokens and passphrases are recorded as *changed* or *set* **without their values**: this is a plain-text file, and it must not become somewhere to read a credential out of.

It can be read in the browser: **Settings** → **Recent hardware events** carries a link to it, newest entry first with a button per day. Administrators only, since it names who did what, and **read-only** — there is no route that can edit or clear it, which is what makes an audit trail worth keeping.



The activity log: two mark corrections, and a settings save with what each key changed from.

NOTE

One line is worth knowing on sight: **finish direction skipped an earlier crossing**. It means a boat crossed the line with every mark rounded and that crossing was refused by the finishing-direction test, so the finish recorded is a later one. Either the boat did cross the wrong way first — the finish video settles it — or the line geometry needs looking at.

Environment variables reference

FIELD / CONTROL	WHAT IT DOES
RO_HOST	Waitress bind address. Default 0.0.0.0.
RO_PORT	Waitress port. Default 5050.
RO_THREADS, RO_CONNECTION_LIMIT, RO_CHANNEL_TIMEOUT	Waitress worker threads (8), connections allowed at once (100) and idle-connection timeout in seconds (120). Defaults for the Settings → Web server values, which override them once saved; raise the connection limit first when phones cannot connect.
RO_DB_TIMEOUT_S	How long a database write waits for another writer before giving up (default 30 s). Waiting beats failing here: a finish that is not recorded is the one thing this app cannot afford to lose.
RO_SECRET_KEY	Flask session signing key; auto-generated and persisted if unset. Set explicitly for internet-facing/Cloudflare deployments.
RO_INITIAL_ADMIN_PASSWORD	First-run admin password (only effective before the DB/first admin user exists).
RO_ALLOW_DEFAULT_ADMIN	Set to 1 to keep an existing insecure admin/admin credential instead of having it auto-replaced.
RO_WEATHER_ALLOWED_HOSTS	Comma-separated hostname allow-list for the weather-station URL, in addition to the built-in localhost/link-local/multicast rejection.
RO_SERIAL_PORT, RO_HORN_LINE, RO_HORN_ACTIVE, RO_HORN_DURATION_MS	Optional startup defaults for the horn output configuration normally set from Settings → Horn settings & Test.
RO_HORN_INPUT_ENABLED, RO_HORN_INPUT_LINE, RO_HORN_INPUT_ACTIVE, RO_HORN_INPUT_POLL_MS	Optional startup defaults for manual horn input sensing, normally set from the same Settings section.
RO_COOKIE_SECURE	Session cookies are marked Secure by default, so they only work over HTTPS. Set to 0 for local http testing — otherwise a login over http://localhost:5050 silently fails to stay signed in.
RO_MAX_UPLOAD_MB	Maximum request/upload size in MB (default 64). Raise it only to restore a very large backup ZIP containing video clips.
RO_TRACK_ENABLED, RO_TRACCAR_BASE_URL, RO_TRACCAR_TOKEN	First-run defaults for the GPS-tracking connection normally set from Settings → GPS tracking.
RO_TRACK_INGEST_SECRET	Shared secret letting Traccar's forwarder POST fixes to /api/track/ingest. Must match forward.header on the relay; without it the endpoint stays closed and positions arrive only on the poll.

FIELD / CONTROL	WHAT IT DOES
RO_TRACK_SIM, RO_TRACK_POLL_SECONDS, RO_TRACK_RETENTION_DAYS, RO_TRACK_ROUNDING_RADIUS_M, RO_TRACK_GATE_REACH_M, RO_GPS_FINISH_HORN	Startup defaults for the simulator, poll interval (5), retention (90 days), mark-rounding radius (50 m), rounding gate reach (750 m) and the auto-finish horn (off).
RO_VEDIRECT_SMARTSHUNT_PORT, RO_VEDIRECT_SMARTSOLAR_PORT, RO_VEDIRECT_PHOENIX_PORT	Serial ports for the three Victron hut-power devices, normally set from Settings → Hut power.
RO_POWER_SAMPLE_SECONDS, RO_POWER_RETENTION_DAYS, RO_POWER_SIM	Hut-power sample interval, history retention, and a simulator for previewing the dashboard card without hardware.

Values set from the Settings UI are persisted to the database and take precedence after first save; the environment variables mainly matter for the very first run or for scripted/headless deployments.

PART VI — ADMINISTRATION AND REFERENCE

Status codes and glossary

Entry status codes

CODE	MEANING
PRESTART	Entered and waiting: a display label shown until the race's warning-signal time (stored as RACING throughou
RACING	Started and still racing
FINISHED	Crossed the finish line — has a finish time
DNC	Did Not Compete — never entered/came to the start
DNS	Did Not Start
OCS	On Course Side — over the line early at the start
RET	Retired from the race
DNF	Did Not Finish
DSQ	Disqualified
DNE	Disqualification that is not excludable as a discard (RRS 90.3(b))
DGM	Discretionary penalty for gross misconduct

Glossary

First warning signal vs. first start

The warning-signal time is set on a race; the actual start is always five minutes later. Every other start in a multi-start plan is an offset from that first start, not from the warning signal.

IRC / YTC

Two independent handicap rating systems scored side by side. A boat only appears in a system's table if it has a rating for it.

Discard profile / constitution

See Chapter 6.

Rating-band class

A named group of boats sharing a rating system and range, with its own numeral class flag.

Compound mark

A named course mark that expands to real physical marks for chart/analysis purposes — see Chapter 5.

Stream copy vs re-encode

Recording modes for the RTSP camera — see Chapter 16.

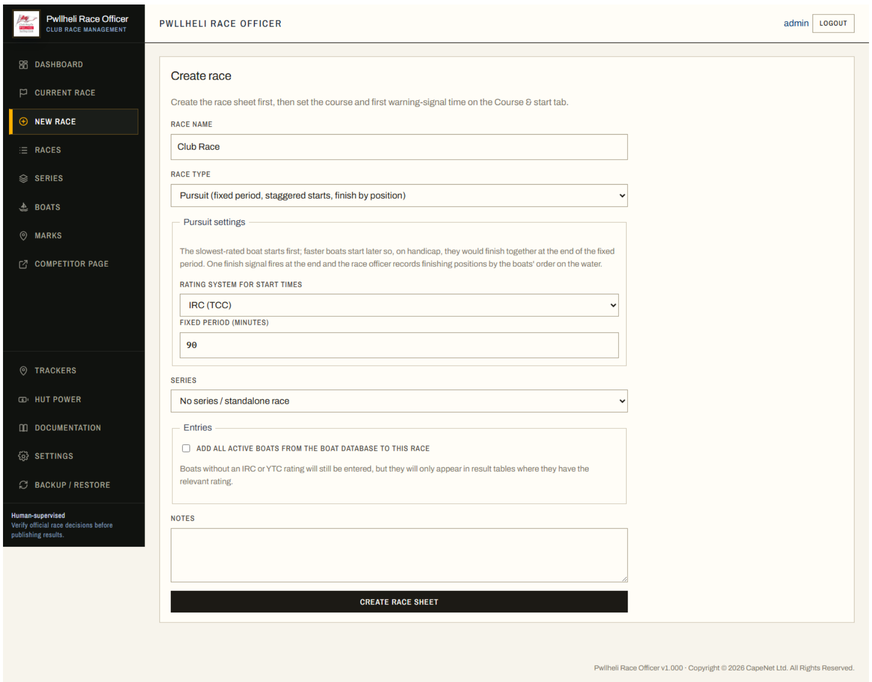
Good practice checklist

- Keep the race-office PC's clock accurate (NTP/GPS-disciplined) and visible; every signal and finish time depends on it.
- Don't let the PC sleep during racing — the start scheduler and horn/audio need to keep running.
- Before race day: check horn wiring and isolation, test manual horn input sensing if fitted, test central audio to the VHF/audio path, and confirm horn/audio/weather/video status in Settings.
- During the start sequence: supervise automatic horn/audio, confirm displayed flags match the physical flags hoisted, and use manual controls for recall/postpone/abandon as needed.
- Prefer the blue **Finish** button so horn, log, finish time and video clip stay linked; if using a manual horn event for a finish, assign it from the horn/race log afterwards.
- Handle protests and redress outside the app, then apply the outcome by editing the affected entry directly.
- Back up **data/** before using “Remove Race” to delete a race sheet — it cannot be undone.
- Re-publish (Download HTML / Download CSV) after any post-race correction so the club website and your records stay in step.
- For a public/Cloudflare deployment, set an explicit **RO_SECRET_KEY** and treat every public URL as world-readable — there is no access-key gate on them.

For narrative, step-by-step walkthroughs rather than reference lookup, see the **Race Officer's Guide** (running a series day-to-day) and the **Competitor's Guide** (what competitors see on the public pages).

Running a pursuit race

A **pursuit race** is a different kind of race, chosen with the **Race type** option when you create a new race. It runs for a fixed period; the slower boats start first and the faster boats later, so on handicap they would all finish together. You pick **IRC** or **YTC** as the single rating system that sets the start times. There are no classes, and there is one finish signal rather than individual finish times.



New race: choosing the Pursuit type, its rating system and the fixed period.

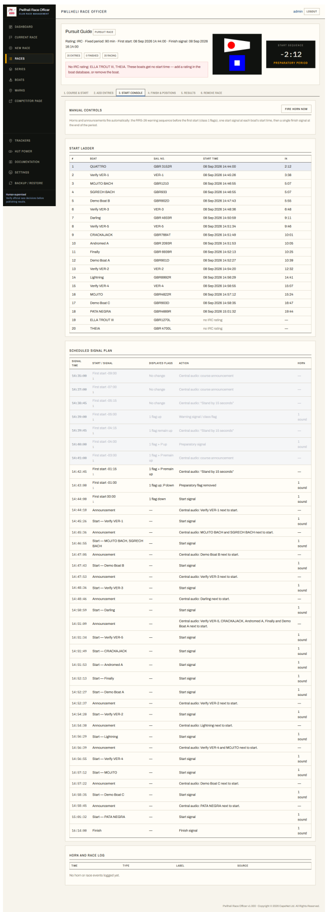
Setting up

1. On **New race**, set **Race type** → **Pursuit**, choose the rating system (IRC or YTC) and the fixed period in minutes, and optionally add all active boats.
2. On the race sheet's **Course & start** tab, set the first warning-signal time (the first, slowest boat starts five minutes later), confirm the period and rating system, and save.
3. Choose a course the same way as a standard race — a fixed numbered course, **Recommend / change course**, or **Build manual course**.
4. Start times are computed automatically and recomputed whenever you add or remove a boat. Until the first warning time and period are set, boats show “set start time” rather than a time — that is different from a genuinely missing rating, which is flagged separately.

> **NOTE**
>
> The slowest-rated boat starts first; each faster boat is delayed by **period × (1 – slowest speed ÷ boat speed)**, derived from the chosen rating (IRC TCC, or 1000 ÷ YTC). A boat with no rating in the chosen system is still entered, but gets no start time and is flagged.

Running the start

The **Start console** tab shows the start ladder: every boat and its start time, with the next boat to start highlighted and a live countdown. Horns fire automatically — the five-minute warning sequence before the first start, one start signal at each boat's start time, then the single finish signal at the end of the period. **Fire horn now** is available for a manual signal. There is a start video for each start time and no finish video. The first start flies the normal flag sequence, defaulting to class 1 (the numeral-1 pennant).



The pursuit start console: the start ladder with the next boat to start highlighted.

Finishing and results

At the finish signal, judge the boats by their order on the water and record it on the **Finish & positions** tab — a position number and status for each boat. There is no time correction: the finishing order is the result, shown on the **Results** tab. A pursuit race that belongs to a series scores into the standings by finishing position, exactly like a normal race's places. Competitors see each boat's start time (next to start highlighted) on the public race page, and the finishing order once you have recorded it.

GPS yacht tracking and automated finishes

Boats carrying low-cost **Queclink GL521MG** GPS trackers can be shown live on the race course chart, with a live position-on-the-water list, and the app can detect finish-line crossings to help record finishes. Positions are shown to the race officer on the race sheet and, since v0.177, on the **public competitor race page** (course chart plus tracking columns in the entries list). The trackers report to a **Traccar** server on the live-stream relay, and no port is opened on the hut PC. Relay/Traccar setup is in **deploy/live_stream/README.md** §7.

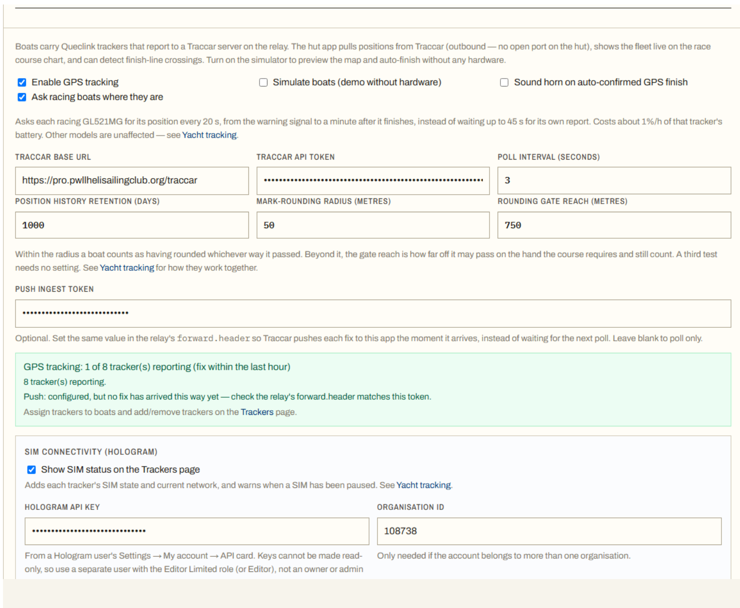
> **NOTE**
>
> GPS tracking is cellular, so coverage thins offshore, and the GL521MG is a battery asset tracker built for very long standby — its default reporting is far too coarse for finish order. Set a **frequent reporting interval on race days**, expect to **charge the trackers between race days** because a fast rate costs a large multiple of the standby drain, and treat GPS finishes as proposals to confirm — the finish video remains the arbiter.

Push or poll

Positions reach the app two ways, and both are wanted. **Push** (v0.189) is the fast path: Traccar's position forwarder POSTs each fix to the app the moment it decodes it, and finish detection runs on receipt — measured at 0.02–0.32 s, against up to a full poll interval of waiting. This is what keeps the automatic horn and the competitors' view prompt. **Polling** stays on as the safety net: it covers a relay that is not forwarding and a stalled forwarder, and it **back-fills gaps**, asking Traccar for the range missed while the app was down rather than leaving a permanent hole in the track. With push working, a poll interval of about 30 seconds is plenty.

> **NOTE**
>
> A recorded finish **time** does not depend on how quickly the app hears about a fix — crossings are interpolated between fix timestamps, so those were always right. What push removes is the delay in the things you can see: the automatic horn, and the positions competitors are shown.

Connection (Settings → GPS tracking)



Settings → GPS tracking: the Traccar connection, poll interval, retention, the mark-rounding radius, the rounding gate reach and the simulator toggle.

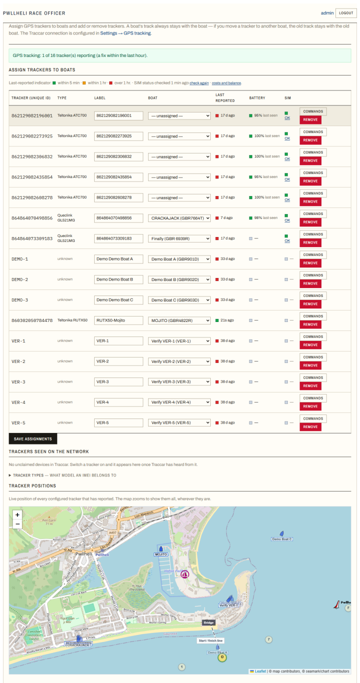
FIELD / CONTROL	WHAT IT DOES
Enable GPS tracking	Master on/off for the position poller.
Traccar base URL / API token	The Traccar server and an API token. The token needs device-management rights so the app can add and remove devices for you.
Poll interval (seconds)	How often positions are pulled from Traccar (default 5). With push working this can go to about 30 — polling is then only the safety net and the back-fill.
Push ingest token	Shared secret that lets Traccar POST each fix straight to the app. Set the same value here and in the relay's forward.header. Leave blank to poll only.
Position history retention (days)	How long stored positions are kept (default 90); a separate database from the race data.
Mark-rounding radius (metres)	Close enough to a mark that it counts as rounded whatever else the geometry says (default 50) — one of the three tests below, not the only one. Inside this distance no side is asked about, on either hand: a boat nearer the mark than we know where the mark is has no determinable side.
Rounding gate reach (metres)	How far past a mark, on the hand a boat is supposed to pass it, still counts as rounding it (default 750). Wide on purpose: it is what catches a boat that gives a mark a berth. On the wrong hand only the rounding radius counts, and that asymmetry is the only thing in the app that knows a mark has a required side.
Simulate boats	Runs a built-in simulator so the map, positions list and finishes can be previewed without any hardware.
Show SIM status on the Trackers page	Off by default. Turns on the SIM column and the top-up warnings described under *What the SIM is doing* below. Needs a Hologram API key.

FIELD / CONTROL	WHAT IT DOES
Hologram API key	From a Hologram user's Settings → My account → API card. Hologram keys cannot be scoped, so limit the key by limiting its owner: make a separate user with the **Editor** role rather than using an owner or admin login. Editor reads SIM state, usage, plan pricing and the balance, and cannot activate SIMs or change billing.
Organisation ID	Only needed when the key's owner belongs to more than one Hologram organisation.

> **NOTE**
>
> The status box below the fields reports whether push is actually arriving, because a token that does not match the relay's fails **silently** — the app carries on polling, every tracker reports, and the only symptom is a late horn. It reads **Push: working — N fixes received, last just now**, **configured, but no fix has arrived this way yet** (go and check forward.header), or **not configured**. Counts are since the app last started.

The Trackers page

Tracker management lives on the **Trackers** page (bottom group of the side menu, with Documentation and Settings). Unlike Settings it is available to **race officers**, since assigning trackers is a race-day task. Each tracker shows how long ago it last reported with a RAG dot — **green** within 5 minutes, **amber** within the hour, **red** after — and the whole page updates live without a refresh.



The Trackers page: each tracker's boat assignment and a live last-reported indicator.

There are two ways to add one. **Trackers seen on the network** (v0.190) lists the devices Traccar has heard from that are not set up here yet, most recently heard from first — switch a tracker on, pick its boat from the dropdown and press **Add**, with no 15-digit IMEI to read off the device. The page notices new arrivals between refreshes and says so. Adopting claims the device in Traccar for the app's token, and removing a tracker deletes it there too. This is the only way in: a form for typing an IMEI by hand was removed in v0.270, because a tracker that has never reached Traccar cannot be tracked whatever the app records, and a 15-digit number read off a label is a number waiting to be mistyped.

> **NOTE**
>
> For a tracker to appear by itself, the relay needs **database.registerUnknown** enabled (see traccar-forward.xml). A device Traccar registers by itself belongs to no Traccar user, which is why Traccar's own web UI hides it until you switch on *All Devices* — and why its positions are not returned to the app's API token at all until adopting it **claims** it. Without that the poller would never see that boat even after it was assigned to one.

> **NOTE**
>
> A boat's track always stays with the **boat**: if you move a tracker to a different boat, the earlier fixes remain with the first boat. The Trackers page is the **only** place a pairing is set — the race sheet used to carry a second, per-race dropdown for a loaner tracker and it was removed in v0.207, because two places to set the same thing is two places to get it wrong on a race morning. To lend a tracker for one race, re-assign it here and move it back afterwards; nothing recorded is lost either way.

What kind of tracker it is (v0.269)

A **Type** column names each tracker's model — Queclink GL521MG, Teltonika ATC700, and so on. It is worked out from the first eight digits of the IMEI, the *type allocation code* that every unit of a model shares, checked against the protocol Traccar decoded. Both are needed: the code belongs to whoever certified the radio, and a tracker built around a bought-in cellular module often carries the module maker's codes, while the protocol alone is far too coarse — an asset tracker that flattens in a day and a mains-powered router are both simply **teltonika**. A model the app does not recognise shows as *unknown* with a **name it** button, which adds it to the **Tracker types** catalogue on the same page. Naming a model is open to race officers; it records a fact about the kit rather than touching the hardware.

Sending a command to a tracker (v0.269)

Commands beside each tracker opens a console that sends a command straight to the device through Traccar and shows what comes back, with a list of prebuilt commands chosen to suit that tracker's protocol. This one is **administrators only** — the rest of the page decides which boat carries which tracker, while this reconfigures hardware at sea. A tracker that is awake answers in seconds; one that sleeps between reports answers on its next connection, up to five minutes on the ATC700.

What you can read back depends on the tracker. Teltonika units send their answer as text and it appears in the console, so a reporting interval can be read and changed there and then. Queclink units acknowledge the command and send their answer as a separate message that Traccar does not keep, so for those the acknowledgement is all the app can show — those commands are labelled *answer not visible here* rather than left looking broken. The exception is a position request, whose answer is a fix and arrives on the map about a second later.

> **NOTE**
>
> **Factory resets are refused outright.** In Queclink's command set a factory reset and a reboot differ by a single digit, and a tracker that forgets its server address stops being findable at all — it has to come off the boat and onto a USB cable to be set up again. A reboot is allowed but asks first, and says what it costs: about **5 minutes 45 seconds with no fixes**, measured, then a few minutes at reduced accuracy. Never do that to a boat approaching the finish, where being blind is worse than being slightly out of position. Every command is written to the activity log, including the refused ones.

Asking racing boats where they are (v0.270)

A Queclink reports once a minute and holds each fix until the top of the minute, so its position reaches the app about **45 seconds old** — 185 m behind the boat at 6 knots, which is the gap a finish is interpolated across. Asked directly it answers in about a second. So while a race is on the app asks: from the **warning signal** until **a minute after each boat stops racing**, every tracked boat carrying a GL521MG is sent a position request every 20 seconds. The extra minute is not padding — a finish is interpolated between the fixes either side of the line, so the one after the crossing is what pins it down.

Twenty seconds is a floor the tracker sets rather than a choice: asked faster it returns the same cached fix over again. The result is fixes about 62 m apart arriving 4 seconds old, against 185 m arriving 45 seconds old. It costs roughly **+1.1%/h**, about double a Queclink's idle draw, so even a nine-hour race polled throughout is under a fifth of its battery. Only the GL521MG is asked, matched on its IMEI type code rather than its protocol: the request carries that model's own password, so another Queclink would simply reject it. Teltonika trackers are left out for a different reason — an ATC700 already reports every 10 seconds while moving and answers in a second, and at about 10%/h it is the one unit that cannot spare the asking.

> **NOTE**
>
> On by default, and switched off in **Settings** → **GPS tracking**. It does nothing unless Traccar is configured, and a race left open by mistake stops being polled after 30 hours, so a forgotten one cannot quietly flatten a tracker.

Live map and position on the water

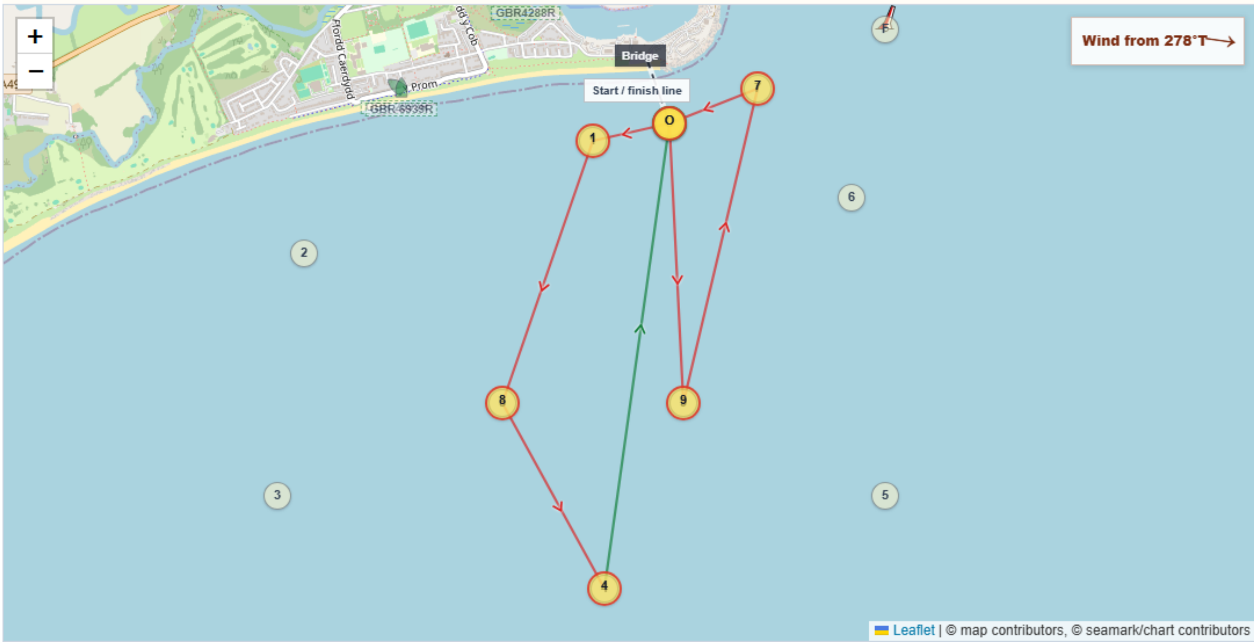
On the race sheet's **Course & start** tab, tracked boats appear on the course chart as a hull seen from above, turned the way the boat is heading, with its sail number beside it, and in the **Position on the water** list, ordered by how far round the course each boat is — marks rounded (e.g. 6/10), the next mark, and the distance still to sail, plus speed and last-fix age. It refreshes every few seconds; a boat whose last fix is over a minute old is dimmed.

The boats are compared as at the same instant. Trackers do not report together - a fleet can be running at 2, 10 and 61 seconds at once - so putting each boat's last known position against the others would compare them at different moments. At 6 knots, two minutes of that is 400 m, which is enough to put an order the wrong way round, and it showed as the list swapping boats about while nothing on the water was changing. Each boat is carried forward from its last fix on its last known course and speed, to the moment being shown. One unheard from for three minutes is left where it last actually was, and marked stale.

Every boat entered is listed, and one with no position says why. Its row is dashed across and the last column reads **No tracker** when there is no tracker on the boat, or **Not reporting** when there is one and nothing has come from it in the last hour — the same hour that turns a tracker red on the Trackers page. Those boats sort after the tracked fleet. The chart and the list will show a boat that reported shortly before the warning signal, so it does not vanish in the minutes around the gun, but not one whose last fix is older than that hour: a boat is not at where it was three weeks ago, and the distance-to-go worked out from such a fix was the length of a passage race.

Until you set the course there is no progress to report. A new race stores the first fixed course as a fallback so the chart and the leg analysis have something to draw, and **Marks**, **Next** and **To go** were being measured against it on a race sheet whose own header said *Course: not set yet* — as was GPS finish detection. Those three columns stay blank until you choose a course. Where each boat is, how fast it is going and when it last reported are the tracker's own report and are unaffected.

▼ COURSE CHART



The live fleet on the course chart — a heading arrow and sail number for each tracked boat.

▼ POSITION ON THE WATER (GPS)

Every boat entered in the race, ordered by how far round the course it is — this is the order on the water, not corrected for handicap. Tracked boats also appear on the course chart above; the list refreshes every few seconds. Assign trackers. ?

#	BOAT	SAIL	MARKS	NEXT	TO GO	SOG	FIX	SAILING TO
1	Mojito	GBR4288R	Finished	—	—	—	66s ago	—
2	Darling	GBR 4933R	—	—	—	—	No tracker	—
3	Mojito Bach	GBR 1210	—	—	—	—	No tracker	—
4	Sgrech	GBR 983	—	—	—	—	No tracker	—
5	Finally	GBR 6939R	1/7	8	4.58 nm	0.0 kn	13124m ago	← →
6	Andromed A	GBR 2093R	1/7	8	4.59 nm	0.0 kn	13975m ago	← →
7	Quattro	GBR 3152	—	—	—	—	No tracker	—
8	Lightning	GBR8992R	—	—	—	—	No tracker	—

Position on the water: every entered boat, ordered by progress round the course.

How a rounding is decided

Three tests decide whether a boat has rounded a mark, and **any one of them is enough**:

- **The rounding radius.** A fix inside it, or the path between two fixes passing inside it. Default 50 m, and any mark can carry its own. Inside this distance no side is asked about, on either hand: a boat nearer the mark than we know the mark's own position has no determinable side.
- **The gate.** A line through the mark at right angles to the leg arriving at it, reaching the **rounding gate reach** (default 750 m) on the hand the course requires and only the rounding radius on the hand it does not. Crossing it is rounding. That asymmetry is the only thing in the app that knows a mark has a required side.
- **Closest approach and departure.** The boat came within the mark's neighbourhood and has since opened up again, with the next mark nearer than it was. The catch-all, and it earns its place on *tight* roundings rather than wide ones: at a coarse reporting rate a close rounding can happen entirely between two fixes, leaving no fix inside the radius and no crossing of the gate to find.

The walk through the course is **sequential**, which is why a missed rounding is expensive: the boat shows several marks behind where it actually is for the rest of the race, and because the finish line is only watched once every earlier mark has been rounded, **its GPS finish never arrives either**.

> **NOTE**
>
> A suspected wrong-side rounding is **reported, never refused**. Refusing it would stall that boat for the rest of the race and take its automatic finish with it, to enforce a rule this app does not adjudicate - the race officer does, on a protest, with the video and the track. So the gate can only ever add a rounding the other tests missed; it cannot take one away.

Correcting the mark a boat is sailing to

A wide rounding usually counts on its own, so the arrows are for what the geometry still cannot see - a mark that has dragged a long way, a tracker silent through the rounding, or a boat sent round something the course does not describe.

The **Sailing to** column at the end of each *Position on the water* row carries a left and a right arrow. Forward treats the mark the boat is heading for as rounded; back puts it on the previous one. The row then shows **set**, so a boat corrected by hand is not mistaken for one the GPS worked out.

- A correction applies **from the moment you press it**, never retrospectively. The club's finish-line mark O is also a mid-course rounding mark on most courses, so a boat nudged on to the line could otherwise be finished by a crossing it made on an earlier lap, at the wrong time.
- It reaches the finish detection, not just the display — which is most of the point.
- The arrows stop at the ends: back from the first mark and forward past the line are both refused, because finishing a boat is the finish button's job. A boat that has finished shows no arrows.
- Every press is recorded in the activity log with the marks either side of it, and appears in the race log.

Tracker batteries

These are battery *asset* trackers run at a reporting rate far above the standby they were designed for, so they need charging between race days. The **Trackers** page shows each tracker's battery beside *Last reported*, with the same red/amber/green dots: **green above 25%**, **amber at or below 25%** (charge it before Saturday), **red at or below 10%** (may not last a race). A dash means that tracker does not report a level, which is not the same as a flat one — it is deliberately not shown as 0%.

The **Dashboard** warns when a tracker *assigned to a boat* is at or below 25%, flattest first and naming the boat. It appears only when there is something to say, and only when GPS tracking is on. A spare in the drawer being flat is not warned about.

What the SIM is doing

A tracker that goes quiet tells you nothing about why. A flat battery, a boat behind a headland and a SIM the network has stopped all look identical from the app — but only the last will still be silent tomorrow, and it is the only one no other part of the app can see. With a Hologram key configured, the Trackers page carries a **SIM** column: a dot and one word, **OK**, **Check** or **Error**, and a link to the detail.

- Read a healthy answer carefully. It says the **SIM** is fine — that nothing at the network end is stopping the tracker — and **not** that the tracker is alive. Only the bad answers mean anything.
- The popout names the state the way Hologram's own dashboard does: **Connected** when a data session is open at that moment, **Ready** when the SIM is live but idle. Both are healthy. The club's Queclinks hold

a long-lived connection and usually read *Connected*; the ATC700s sleep between reports and flip between the two.

- **Pausing** means somebody is switching the SIM off and the change has not landed yet. It is reported as paused, because it is about to stop carrying data — better a minute early than a page that says the SIM is fine while it is being switched off.
- **Error** is a SIM that cannot carry data. *Paused by system* means it has hit a usage limit or a low balance and will not recover on its own; it needs resuming in the Hologram dashboard.
- **Check** is a state this app has not been taught. Not a claimed fault, and not a clean bill either.
- The detail popout also shows which carrier the tracker is actually on. That is not trivia: the club's SIM is multi-network, and on a Teltonika half of every setting is live or dead depending on whether the device counts as roaming.

> **NOTE**
>
> Hologram is asked at most every two minutes, and the line above the table says how old the answer you are reading is. If you have just changed something in the Hologram dashboard, press **check again** rather than waiting.

Data, cost and topping up

costs and balance, beside *check again*, totals the fleet: each tracker's data and cost, the account balance, and the date it runs out. The account is prepay, and when it empties **every tracker stops at once**, with nothing on the water to see it coming.

- The top-up date is not the balance divided by a monthly cost. Hologram bills each SIM on its own 30-day cycle from the day it was activated, so the fees arrive in **clusters** that an average would hide. The app walks forward a day at a time instead, taking the recent daily data spend off each day and each SIM's fee on its own renewal date.
- The dashboard says so too, once a month or less is left and more loudly inside a fortnight.
- Every money figure is worked out here, not read from an invoice: Hologram returns **no cost** on any usage endpoint, only a per-MB rate and a recurring fee. Figures are US dollars.
- There is no single time period on that page. Each SIM's *this period* is a different window, so the table shows how far into its own cycle each one is and carries one figure — the last 28 days — on a window they all share.

Automated finishes

Arm GPS auto-finish and **Auto-confirm** are both ticked on a new race, on the **Entries & finish times** tab, so a finish is caught whenever it can be; untick either there if you would rather not. The app follows each boat through the course marks in order and only records a finish once every mark has been rounded and the boat then crosses the line — so a course that passes the ODM mid-course (e.g. O-F-O-1-O), or a shortened course, finishes correctly on the final crossing, not an early one.

Both boxes sit on a single **GPS finish detection** strip above the list of entries, with **Apply** and the current state — *Armed. No finishes detected yet.*, or *Off for this race.* With **Auto-confirm** unticked, each detection waits on that strip instead, as a row offering **Confirm** or **Dismiss** beside the time it caught and the finish video.

- Each detected finish is **proposed** — boat and interpolated time, with a link to the finish video to check it against. Confirm or dismiss it; a confirmed finish is tagged **gps-confirmed**.

- With **Auto-confirm** left on, each detected finish is recorded straight away (tagged **gps-auto**) and remains reviewable and editable afterwards. Untick it and detections wait as proposals instead.
- The horn only sounds on an auto-confirmed GPS finish if that option is enabled in Settings (off by default, because GPS latency makes an automatic horn late).

When the line’s mark has moved

A finish line has two ends and they are not the same kind of thing. The **shore** end is a surveyed transit — a bridge window — and does not move. The **seaward** end is a laid buoy on a mooring: it swings, it drags, and it gets re-laid. A boat that passes *inside* the physical buoy but *outside* the position the app holds for it falls off the end of the line segment, and its finish is never seen. The finishing-direction test does not save you — that measures against the infinite line and is perfectly content; only the segment bound refuses the crossing. An ISORA night race lost the third boat’s automatic finish exactly that way.

So each line projects its seaward end outward, by **seaward_extension_m** in **data/start_finish.json**: 150 m on the club line (347 m, tested as 497 m) and 250 m on the ISORA line (1510 m, tested as 1760 m). Only the seaward end — extending the shore end would project the line inland over the harbour wall and cover nothing that can move.

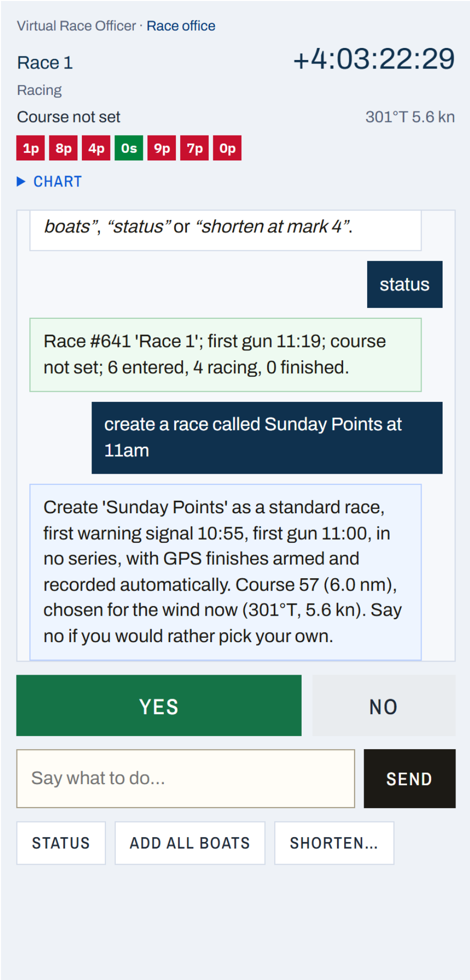
> **NOTE**
>
> Safe because the extension is **collinear**: the infinite line is unchanged, so the finishing-direction test cannot be affected by it, and a finish still requires every mark rounded first. The extended line is used **only** for the crossing test — the chart still draws the line where it really is, and distance-to-go still measures to the real mark. It is a tolerance, not an open end: a boat well beyond it still does not finish. Set it to the widest the buoy plausibly swings.

A tolerance can hide a wrong position rather than fix one, so this is worth knowing: the stored position for **F**, the Pwllheli Fairway Buoy, was **112 m inshore** of the position ISORA’s own sailing instruction gives — which had been sitting in **start_finish.json** all along as **si_seaward_position**, unread. On a 1510 m line that is 7% of its length, before the buoy had moved at all. It is corrected, with the old position kept in the mark’s history so races already scored still replay against the line they were sailed to. **A drifting line mark is worth re-measuring, not just tolerating**: the extension is for the drift between measurements, not a substitute for making one.

PART IX — THE VIRTUAL RACE OFFICER

Running racing from the water

Everything in this manual so far is done on the race sheet, in the hut. It can also be done from a boat. The **Virtual Race Officer** at ***/vro*** is one page — a box to type in, a thread of conversation and a **Yes** button — for a race officer with a phone in one hand and a tiller in the other. It creates races, sets starts and courses, enters boats, shortens courses and answers questions about the racing, and it does all of it by calling the same code the race sheet calls. There is no second way to run a race.



The on-the-water page: the race and the countdown to the gun above, the conversation below, and a read-back waiting for Yes.

Who may use it

Being an administrator is not enough. Each account has a **VRO** tick box in **Settings** → **Users**, and without it the page answers 403 and names the permission that is missing. Both command endpoints check it as well as the page, because a POST is a POST.

> **NOTE**
>
> The separation is deliberate. The club has settled that a race may start with **nobody watching the line** — a boat later judged over by video takes an alternative penalty — but **who** may drive the racing from a phone is a different decision, and it is made one account at a time. **Arming the start sequence is not offered on this page at all**, because that decision has not been made.

How a command works

1. You type a sentence — *“create a race called Sunday Points at 11am, in the summer series”*.
2. The app says back exactly what it would do, in full: *“Create ‘Sunday Points’ as a standard race, first warning signal 10:55, first gun 11:00, in series Summer Series 2026. Course 29 (4.0 nm), chosen for the wind now (131°T, 12.0 kn).”*
3. Nothing happens until you press **Yes**, or type one — *yes*, *ok*, *go ahead*, *do it*.

The read-back is the whole safety model. It is the only check that catches the class of mistake where the app understood something perfectly and it was not what was meant: a race at 11 tomorrow rather than today, the wrong fleet, the right command aimed at yesterday’s race. Questions are answered straight away, because there is nothing to undo. A sentence that begins with yes but carries a change — *“yes, but make it half past”* — is an instruction, and is read as one.

> **NOTE**
>
> A proposal lapses after **five minutes**, only one is live at a time, and sending the same command twice does it once — a phone that retries after a dropped reply is the same command, not a second race. Every command, read-back and result goes to the activity log with the account that gave it.

What it will do

FIELD / CONTROL	WHAT IT DOES
create a race	A race sheet with its first gun, its type and its series. A course is suggested for the wind at that moment and set if you agree.
set the start / the course	*“use course 4”*, *“put the start back ten minutes”*, *“bring it forward five”*.
add entries	Every active boat, one fleet, or one named boat — *“add Mojito”*.
shorten the course	At a mark the fleet is already sailing to. Two horn blasts and the spoken announcement, after you confirm.
status / results	How the race is going, or the finishing order of a race already sailed.
make up a course	A course built from the club’s marks instead of a numbered one, read back mark by mark. Say *“change the second mark to 2, starboard”* and it reads the whole course back again.
look something up	The club’s own data: where a mark is, a course’s legs and their distances, how many courses there are, the boat database, the series, recent races, where the fleet has got to, the polar and its sail chart.
a boat’s rating	IRC and YTC, by name or sail number — from the boat database, and from the RORC IRC listing and the YTC sheet when the boat is not in it. It offers to add a boat the club does not have, and **never overwrites an existing record**.

NOTE

Three places hold a rating and they are not the same thing. The **boat database** is what a race actually scores on, because entering a boat snapshots its rating at that moment; the **IRC listing** and the **YTC sheet** are where that figure came from and either may have moved since. All three are reported, because somebody asking what a boat is rated is usually asking because something does not look right. The import screens on the Boats page do update an existing record from a listing — correctly, with the two side by side in front of you. From a boat they are not, so there the existing record wins: taking a listing figure over the club's own record is how a season gets scored on the wrong number.

It will also answer questions in words, from what the app itself knows: the wind now and how it has shifted over the last hour, which boats are entered, how long a course should take round, which course would give more reaching, and whether the trackers are reporting.

What it will not do

- **Sound a horn on command.** There is no *arm* step and no *fire now*. With start automation switched on in Settings the app fires the warning, preparatory and start signals itself, off the race's stored warning signal — so **setting or moving a start time from the water moves the horn with it**, and the read-back says so. With start automation off, nothing sounds by itself whatever time is set.
- **Signal a recall.** The club starts races with nobody watching the line and reviews the video afterwards, so there is nothing for it to signal.
- **Abandon a race, or record a finish.**

Asked for any of these it says so plainly and offers the nearest thing it can actually do. It never picks a nearby command to be helpful: an unasked-for command is far worse than an unanswered question.

Which interpreter is answering

A **model** reads what you type — one configured in **Settings** → **Virtual Race Officer**. There is nothing behind it: **without one the page is unavailable and says so**, offering no box to type in, and both command endpoints refuse in the same words.

NOTE

The app does have a small built-in grammar — about six sentence shapes — and until v0.264 it answered when no model was configured or one could not be reached. On a page whose whole premise is *type what you want to do*, that reads as an app that understands nothing, and somebody on the water cannot tell a sentence it will not understand from one it has misunderstood. The club's decision is that the feature does not exist until an interpreter is configured and answering. What it costs: the hut's outbound internet can fail while the page itself is perfectly reachable, and the racing then goes back to the race sheet. The page and the Settings card both say which state it is in, and if the last request failed they give the provider's own reason — a 402 for an empty account, a 400 for a model id that does not exist.

The conversation

- It remembers the **last half hour** with the same person, so *"make it a pursuit"* or *"add the fleet to that one"* has something to refer to.
- A question it asks can be answered in a word. Asked *"standard race or a pursuit?"*, answering **standard** re-issues the whole command with the time, the name and the length it already had.

- While a proposal is waiting it is what the conversation is *about*, so asking for a longer course changes that proposal rather than something else.
- **A refresh does not lose it.** On a boat that is a dropped signal or a locked phone, not a decision to start again — the page reloads the conversation, and a proposal still waiting comes back with its Yes button live.

The strip at the top

Two lines and a rolled-up chart, because every line there is a line of conversation pushed off a phone screen: the race the conversation is about and what it is doing; the **countdown to the first gun**, which turns red inside the last five minutes; the course with its length and about how long it should take round in this wind, and the wind itself; and the **course board** — the same marks and hands the race sheet and the clubhouse display show, and the thing you read out over the VHF, sized for a phone. **Chart** stays closed until you open it and then draws the marks and legs from the page itself — no map tiles and no requests, over the same 4G the hut is using. It moves with every command, so a start put back twenty minutes moves the countdown that was the reason for moving it — and so does the board, the chart and the expected time when the course changes.

> **NOTE**
> A command names one or two things, and everything else about the race is left exactly as it was. That is worth stating because it was not always true: the Course & start *form* saves every field on it, which is right for a form, and a command built from only what it named was clearing the rest — “use course 29” wiped the race’s start time, its series (so it scored in no standings), its class and its finish line, while the read-back said “course 29”.

Finishes record themselves

A race created from this page has **Arm GPS auto-finish** and **Auto-confirm (unmanned)** both switched on, and the reply says so — it is a real difference in how the race will be run that the sentence did not ask for. The premise of the page is that nobody is in the hut to press **Finish**, so the app follows each boat through the course marks and records its finish as it crosses the line. Both are the ordinary tick boxes on the **Entries & finish times** tab and can be turned off there. GPS gives the approximate time and order; the finish video remains the arbiter, and every recorded finish stays editable (Chapter 24).

A course nobody has chosen

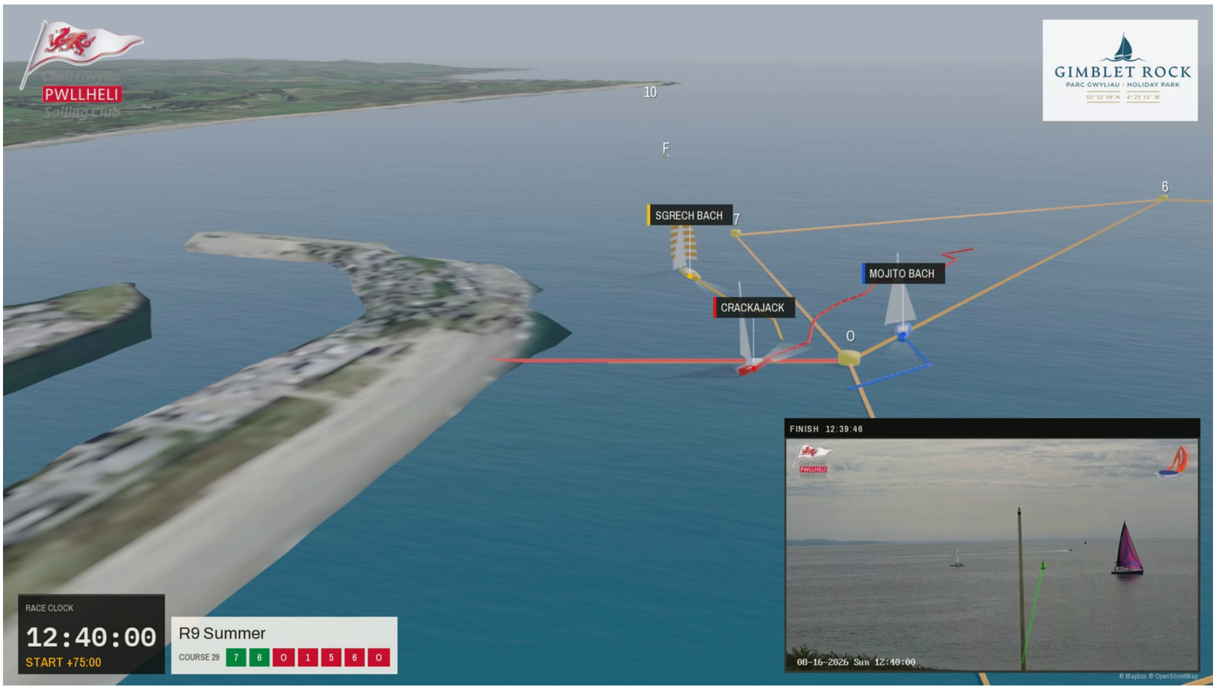
A new race stores the first fixed course as a fallback so the chart, the leg analysis and the shortening options have geometry to work with. That is not a decision, and the app does not treat it as one. Until somebody chooses a course the race sheet and the competitor page both read **Course not set**, and **the spoken VHF course announcement stays silent** — a fleet sent round a course the race officer never picked is a general recall at best.

From v0.276 that is the whole page and not just the header. The course picker itself reads **Course not set** with nothing selected; there is no course board, no leg analysis and no predicted time; and the chart draws the marks and the start/finish line but no course. The list of races and a series' own race table say *not set* in their Course column. Before this the header said one thing and the board directly beneath it drew the fallback's seven marks, which is a contradiction a race officer has to resolve on race morning.

Choosing a course from the picker and saving is what marks it chosen; so are the recommender, the manual builder, and agreeing to the course the on-the-water page suggests when it creates a race. Saving **Course & start** with the picker left on **Course not set** saves everything else and leaves the race without a course, so a first warning signal can be set before the course is known. It only ever goes one way: a course already chosen stays chosen.

Making a film of a race

The app can turn a sailed race into a **film**: the fleet on the water in three dimensions, sailing the course they actually sailed, over the real coastline, with the hut camera cut in at the start and at the finishes. It is built from what the race office already recorded — GPS tracks, the course as sailed, the wind log, the results and the published start and finish videos — so there is nothing extra to do on the water to get one.



A finish, from a film of a club race: the fleet in three dimensions, the hut camera cut in against the same race clock, and the club's own branding.

The hut PC does not make the film

It cannot. A single replay is around eleven thousand frames, which is a couple of hours of work for a machine with a graphics card, and the race-office PC is a fanless box that is also running the race. So it does not try. Pressing **Render a 3D film** writes a *job* into the same Cloudflare R2 bucket the club's race videos already use. A separate **render machine** — a desktop at home, switched on when there is something to make — picks the job up, makes the film, and puts it back in that bucket beside the clips it is made from.

> **NOTE**
>
> Neither machine connects to the other. The hut only ever pushes and the render machine only ever polls, so the render machine needs no route into the club and can sit behind any router. It also means nothing happens until a render machine is switched on: a job queued against one that is off waits patiently, which is what the dashboard card is for.

A render machine is built by unzipping the same release as the clubhouse PC — the renderer ships inside it — onto a machine with a graphics card. The full procedure is in **deploy/render_machine/README.md**, which is also on the Documentation page. A club with no render machine simply never sees the button do anything, and nothing else in the app is affected.

Asking for a film

On a race's **Results** tab, beside *Replay this race in the bar*, there is **Render a 3D film**. Any signed-in user may press it: it costs a little storage and somebody else's computer, and the person who wants the film after a race is not always an administrator.

What it tells you when you press it:

- **How many boats, and how long the racing was** — enough to see it picked up the race you meant.
- **A warning if race videos have not been published yet.** The film cuts the hut camera in at the start and at each finish and can only use clips that have reached the bucket. This is not a failure: the film is made either way, it simply has no inset for the missing clips. Publish the videos first if you want them in it.
- **Whether any render machine has checked in.** If none has, the job is queued and will be picked up whenever one appears.

The same strip then shows progress and finally **Watch the film**. **Render it again** replaces the film everywhere, including on links already shared. Treat a render as *this evening* rather than *in a minute*: an eight-minute film of a three-boat race took two hours on a modest graphics card and about twenty minutes on a good one.

The dashboard card

The **3D replay** card is not really about progress. It is about whether there is a render machine at all, because a job queued against a machine that is switched off looks exactly like one being worked on until somebody notices hours later. It names the race being worked on, says when no render machine has checked in, and flags a render that stopped reporting part way through. It hides itself completely when the club has not set this up.

What competitors see

Once a film exists a link appears by itself in three places: a **3D replay** button beside *Open* in the public races list, **Watch the 3D replay** in the public race page header, and a line in each race's section of a published results document. Nothing appears until the film is actually in the bucket, so an unrendered race looks exactly as it did before. Every one of those links goes straight to the storage bucket rather than through the hut, so a film plays whether or not the clubhouse PC is switched on and a large download never crosses the hut's 4G connection.

What is in the film

- **The boats**, each in its own colour, heeling and trimmed to the wind of that moment and setting a spinnaker when the angle calls for one. The wind is the race's own log sampled through the race rather than averaged — on a day that went from 122° to 196°, an average would have shown a fleet trimmed to a wind that was only briefly true.
- **The course actually sailed**, including a shortened course, with the marks where they stood on the day rather than where they are now.
- **The start line and the finish line.** For an ISORA passage race these are not the same line, and the film uses both.
- **The hut camera**, cut in at the start and at each finish, locked to the same clock as the 3D view so the picture and the boats agree.
- **A race clock, the course board and the true wind**, in the app's own typefaces.

- **Each boat's name and its speed over the ground** on a plate above the rig, with a line down to the boat. The speed is the tracker's own, the same figure the race office sees.
- **The club burgee and the sponsors**, from the same source as the club's start and finish videos, so changing a sponsor in Settings changes the films too.
- **A title card and a results card**, at the front and the end. The results card carries the corrected times in both rating systems, so the film stands on its own away from the app.

The replay runs at 30× real time, dropping to real time at the start and at each finish so those can be watched properly.

Before it will work

- **Public video (Cloudflare R2)** configured under **Settings** → **Video**. The films go in the same bucket as the race videos, and both ends find each other through it.
- **Settings** → **Web server** → **Public address** set to the address competitors use. The render machine reads the club's logos from there; without it the film still renders, just unbranded.
- **A render machine**, per **deploy/render_machine/README.md**.

Fuller operating notes, including what to check when a film comes out without land or without logos, are in **docs/RACE_REPLAY_3D.md** on the Documentation page.